

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination

Lab Course CS-308-MJ-P

Data Science and Analytics

Slip 1

Q1. Create a dataframe containing information about different products such as Price, Rating, and Number of Sales. Generate a Dendrogram to group similar products based on their characteristics.

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage

products = {
    "Product": ["Laptop", "Mobile", "Tablet", "Headphones",
                "Smartwatch", "Camera"],
    "Price": [60000, 25000, 30000, 5000, 8000, 45000],
    "Rating": [4.5, 4.2, 4.3, 4.0, 4.1, 4.4],
    "Sales": [120, 250, 180, 400, 350, 100]
}

df = pd.DataFrame(products)

print("Product DataFrame:")
print(df)

data = df[["Price", "Rating", "Sales"]]

linked = linkage(data, method="ward")

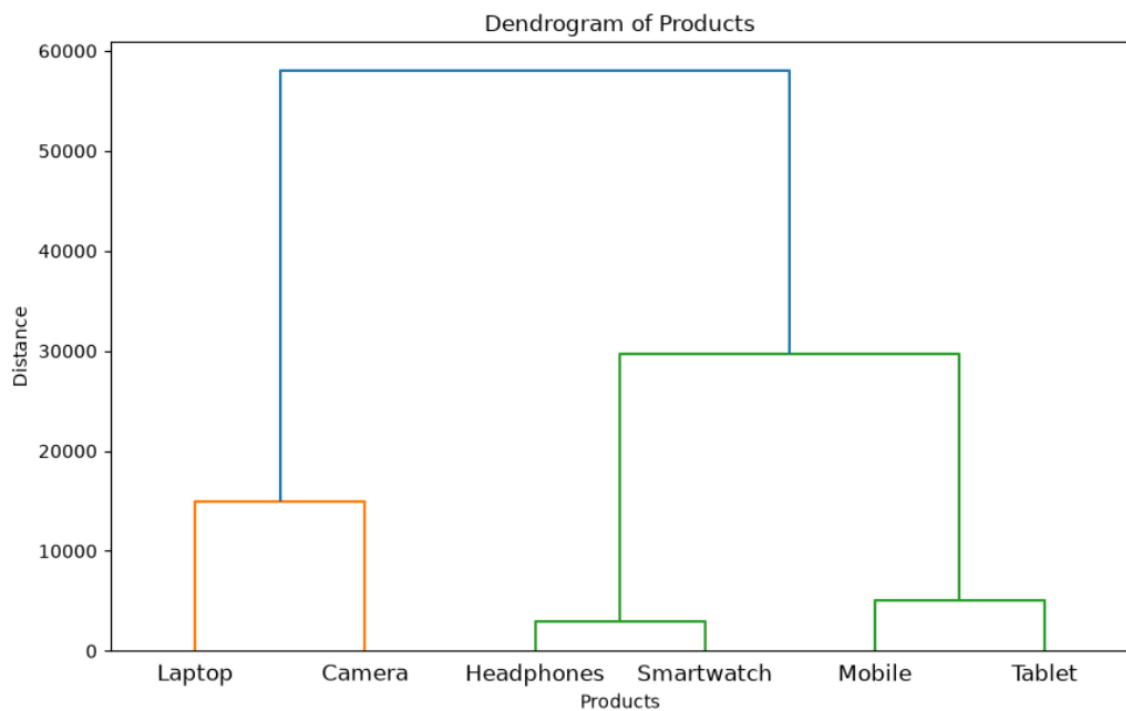
plt.figure(figsize=(10, 6))
dendrogram(linked, labels=df["Product"].values)
plt.title("Dendrogram of Products")
```

```
plt.xlabel("Products")
plt.ylabel("Distance")
plt.show()
```

Output

Product DataFrame:

	Product	Price	Rating	Sales
0	Laptop	60000	4.5	120
1	Mobile	25000	4.2	250
2	Tablet	30000	4.3	180
3	Headphones	5000	4.0	400
4	Smartwatch	8000	4.1	350
5	Camera	45000	4.4	100



Q2. Create a suitable dataset named `Loan_application.csv` containing at least 10 records with the attributes `Application_ID`, `Age`, `Income`, `Credit_Score`, `Loan_Amount`, `Employment_Years`, `Debt_Ratio`, and `Loan_Status`, where `Loan_Status` represents whether the loan application is `Approved` or `Rejected`. Write a Python program to perform the following data reduction techniques: a) **Feature Selection: Identify the important features for predicting `Loan_Status`. b) **Feature Extraction**: Reduce the number of**

numerical attributes using a suitable technique such as Principal Component Analysis (PCA).

```
import pandas as pd
```

```
data = {  
    "Application_ID": ["LA001", "LA002", "LA003", "LA004", "LA005",  
                      "LA006", "LA007", "LA008", "LA009", "LA010"],  
    "Age": [25, 35, 42, 29, 51, 31, 45, 27, 38, 48],  
    "Income": [30000, 55000, 70000, 40000, 85000, 50000, 75000,  
35000, 60000, 90000],  
    "Credit_Score": [650, 720, 780, 680, 800, 710, 760, 640, 730,  
790],  
    "Loan_Amount": [200000, 400000, 600000, 250000, 700000, 350000,  
500000, 180000, 450000, 650000],  
    "Employment_Years": [2, 8, 15, 4, 20, 6, 12, 1, 9, 18],  
    "Debt_Ratio": [0.45, 0.30, 0.20, 0.40, 0.15, 0.32, 0.22, 0.50,  
0.28, 0.18],  
    "Loan_Status": ["Rejected", "Approved", "Approved", "Rejected",  
"Approved",  
                    "Approved", "Approved", "Rejected", "Approved",  
"Approved"]  
}
```

```
df = pd.DataFrame(data)
```

```
df.to_csv("Loan_application.csv", index=False)
```

```
print("Loan_application.csv created successfully!")
```

```
print(df)
```

Feature Selection

```
import pandas as pd
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
df = pd.read_csv("Loan_application.csv")
```

```

X = df[["Age", "Income", "Credit_Score", "Loan_Amount",
        "Employment_Years", "Debt_Ratio"]]

y = df["Loan_Status"].map({"Rejected": 0, "Approved": 1})

model = RandomForestClassifier(n_estimators=100, random_state=42)

model.fit(X, y)

importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
})

importance = importance.sort_values(
    by="Importance",
    ascending=False
)

print("Feature Selection:")
print(importance)

```

Output

Feature Selection:

	Feature	Importance
1	Income	0.185567
4	Employment_Years	0.185567
3	Loan_Amount	0.175258
0	Age	0.164948
5	Debt_Ratio	0.154639
2	Credit_Score	0.134021

Feature Extraction using PCA

```
import pandas as pd

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA


df = pd.read_csv("Loan_application.csv")


X = df[["Age", "Income", "Credit_Score", "Loan_Amount",
        "Employment_Years", "Debt_Ratio"]]


scaler = StandardScaler()


X_scaled = scaler.fit_transform(X)


pca = PCA(n_components=2)


X_pca = pca.fit_transform(X_scaled)


pca_df = pd.DataFrame(
    X_pca,
    columns=["Principal_Component_1", "Principal_Component_2"]
)


print("Feature Extraction using PCA:")
print(pca_df)


print("\nExplained Variance Ratio:")
print(pca.explained_variance_ratio_)


print("\nTotal Variance Explained:")
print(pca.explained_variance_ratio_.sum())
```

Output

Feature Extraction using PCA:

	Principal_Component_1	Principal_Component_2
0	-3.303471	-0.065789
1	-0.389469	-0.181688
2	1.986589	-0.379555
3	-2.254782	-0.055789
4	3.608833	0.131424
5	-1.074904	-0.367187
6	1.577569	0.203999
7	-3.474851	0.500779
8	0.183924	-0.073729
9	3.140560	0.287536

Explained Variance Ratio:

```
[0.97890887 0.011952]
```

Total Variance Explained:

```
0.9908608638587001
```

OR

Q2. Using the built-in Iris dataset, apply K-Means clustering with K = 3. Plot the datapoints, visualize the clusters with different colors, and display the cluster centroids.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans

iris = load_iris()

X = iris.data

df = pd.DataFrame(X, columns=iris.feature_names)
```

```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
```

```
df["Cluster"] = kmeans.fit_predict(X)
```

```
centroids = kmeans.cluster_centers_
```

```
print("Clustered Iris Dataset:")
```

```
print(df)
```

```
print("\nCluster Centroids:")
```

```
print(centroids)
```

```
plt.figure(figsize=(8, 6))
```

```
plt.scatter(  
    X[:, 0],  
    X[:, 1],  
    c=df["Cluster"],  
    cmap="viridis",  
    s=50  
)
```

```
plt.scatter(  
    centroids[:, 0],  
    centroids[:, 1],  
    c="red",  
    marker="X",  
    s=200,  
    label="Centroids"  
)
```

```
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Sepal Width (cm)")
plt.title("K-Means Clustering of Iris Dataset (K = 3)")
plt.legend()
plt.show()
```

Output

Clustered Iris Dataset:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0 0.2	5.1	3.5	1.4	
1 0.2	4.9	3.0	1.4	
2 0.2	4.7	3.2	1.3	
3 0.2	4.6	3.1	1.5	
4 0.2	5.0	3.6	1.4	
.. ...	...	...	...	
145 2.3	6.7	3.0	5.2	
146 1.9	6.3	2.5	5.0	
147 2.0	6.5	3.0	5.2	
148 2.3	6.2	3.4	5.4	
149 1.8	5.9	3.0	5.1	

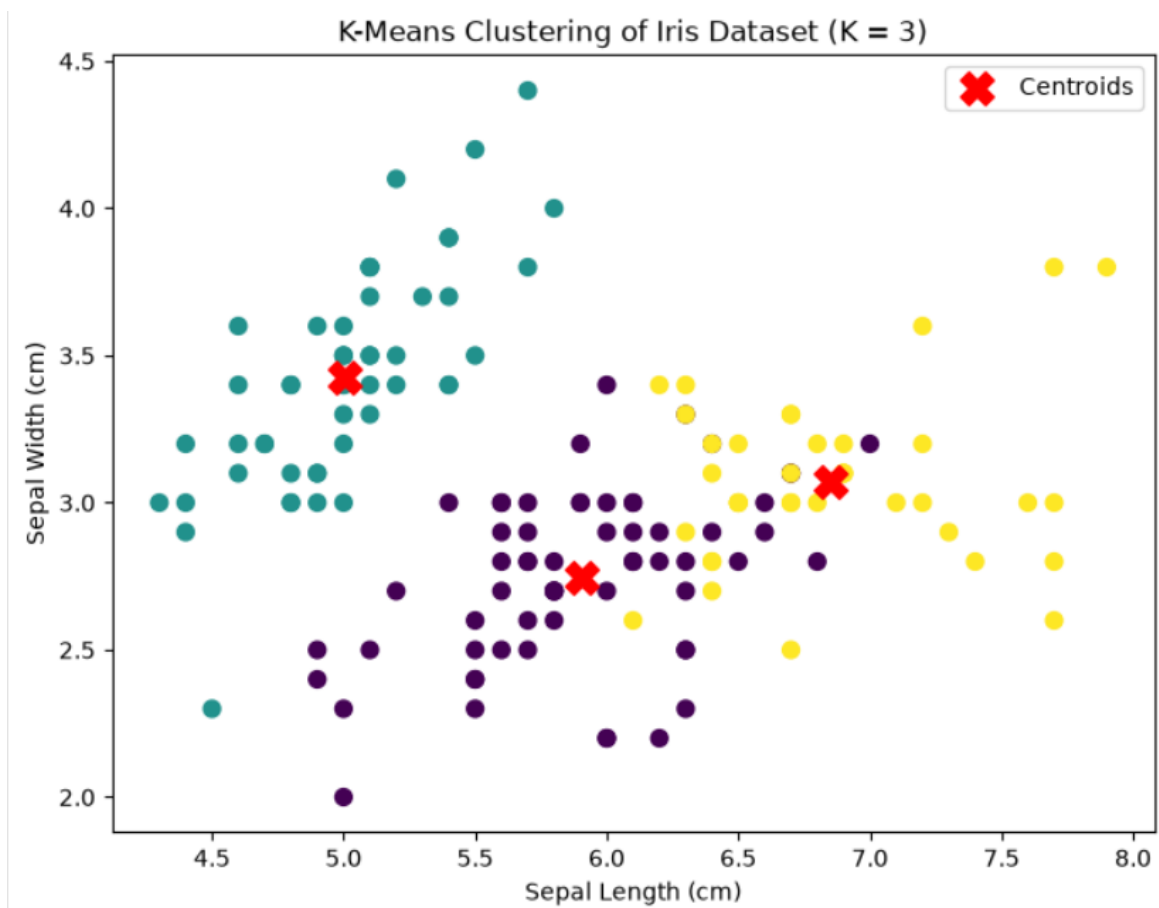
	Cluster
0	1
1	1
2	1
3	1
4	1
..	...


```
145      2
146      0
147      2
148      2
149      0
```

```
[150 rows x 5 columns]
```

Cluster Centroids:

```
[[5.9016129  2.7483871  4.39354839  1.43387097]
 [5.006      3.428      1.462      0.246      ]
 [6.85      3.07368421  5.74210526  2.07105263]]
```



Slip 2

Q1. Write a Python program to create an age_records DataFrame containing the attributes Person_ID and Age for suitable sample records. Perform Data Discretization on the Age attribute using Equal Width Binning. Display the original and discretized data.

```
import pandas as pd

age_records = pd.DataFrame({
    "Person_ID": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "Age": [18, 22, 27, 31, 36, 42, 48, 55, 63, 70]
})

age_records["Age_Group"] = pd.cut(
    age_records["Age"],
    bins=4,
    labels=["Young", "Adult", "Middle Age", "Senior"]
)

print("Original and Discretized Data:")
print(age_records)
```

Output

Original and Discretized Data:

	Person_ID	Age	Age_Group
0	1	18	Young
1	2	22	Young
2	3	27	Young
3	4	31	Young
4	5	36	Adult
5	6	42	Adult
6	7	48	Middle Age
7	8	55	Middle Age
8	9	63	Senior
9	10	70	Senior

Q2. Apply Linear Regression on a YouTube Video dataset to predict video views using relevant video attributes. Visualize the relationship between the predictor and target variable using a scatter plot and regression line.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

data = {
    "Likes": [500, 800, 1200, 1500, 2000, 2500, 3000, 3500, 4000,
4500],
    "Views": [5000, 8000, 12000, 15000, 21000, 25000, 31000, 36000,
42000, 48000]
}

df = pd.DataFrame(data)

X = df[["Likes"]]
y = df["Views"]

model = LinearRegression()
model.fit(X, y)

df["Predicted_Views"] = model.predict(X)

print("YouTube Video Dataset:")
print(df)

print("\nCoefficient:", model.coef_[0])
print("Intercept:", model.intercept_)

plt.figure(figsize=(8, 5))

plt.scatter(df["Likes"], df["Views"], label="Actual Data")
```

```
plt.plot(
    df["Likes"],
    df["Predicted_Views"],
    label="Regression Line"
)

plt.xlabel("Number of Likes")
plt.ylabel("Number of Views")
plt.title("Linear Regression: Likes vs Views")
plt.legend()
plt.show()
```

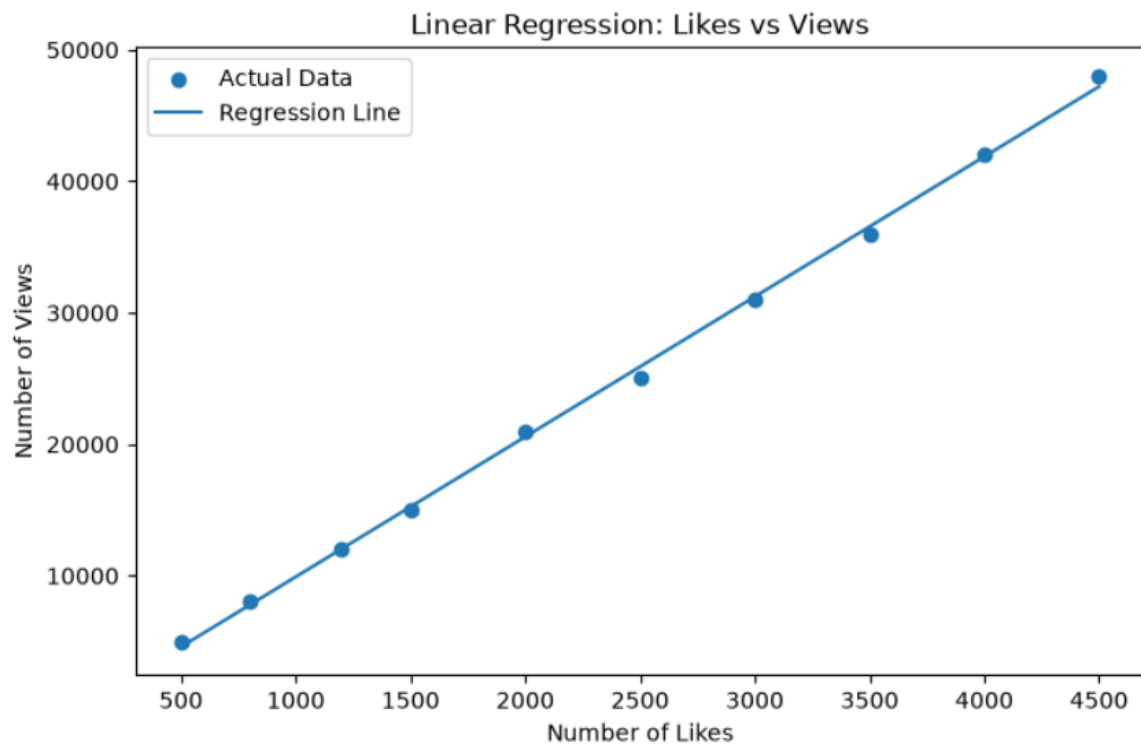
Output

YouTube Video Dataset:

	Likes	Views	Predicted_Views
0	500	5000	4588.658287
1	800	8000	7785.092078
2	1200	12000	12047.003800
3	1500	15000	15243.437591
4	2000	21000	20570.827243
5	2500	25000	25898.216896
6	3000	31000	31225.606548
7	3500	36000	36552.996200
8	4000	42000	41880.385852
9	4500	48000	47207.775504

Coefficient: 10.654779304296989

Intercept: -738.7313650979231



OR

Q2. Create the following dataset in python. Apply the Apriori algorithm to generate frequent itemsets and association rules. Repeat the process with different minimum support values 0.2, and 0.3.

TID	Items
1	Video Lecture, Quiz
2	Video Lecture, Assignment
3	Quiz, Discussion Forum
4	Video Lecture, Quiz, Assignment
5	Assignment, Discussion Forum
6	Video Lecture, Certificate
7	Quiz, Certificate
8	Video Lecture, Quiz
9	Assignment, Certificate
10	Video Lecture, Quiz, Certificate

```
import pandas as pd

from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

transactions=[
```

```

["Video Lecture","Quiz"],
["Video Lecture","Assignment"],
["Quiz","Discussion Forum"],
["Video Lecture","Quiz","Assignment"],
["Assignment","Discussion Forum"],
["Video Lecture","Certificate"],
["Quiz","Certificate"],
["Video Lecture","Quiz"],
[],
["Assignment","Certificate"],
["Video Lecture","Quiz","Certificate"]
]

```

```

transactions=[t for t in transactions if t]

```

```

te=TransactionEncoder()

```

```

te_array=te.fit(transactions).transform(transactions)

```

```

df=pd.DataFrame(te_array,columns=te.columns_)

```

```

print("Dataset:")

```

```

print(df)

```

```

for support in [0.2,0.3]:

```

```

    print("\nMinimum Support:",support)

```

```

    freq_items=apriori(df,min_support=support,use_colnames=True)

```

```

    print("\nFrequent Itemsets:")

```

```

    print(freq_items)

```

```

rules=association_rules(freq_items,metric="confidence",min_threshold
=0.5)

```

```

rules=rules.sort_values(["support","confidence"],ascending=[False,Fa
lse])

```

```

print("\nAssociation Rules:")

if len(rules)>0:

print(rules[["antecedents","consequents","support","confidence","lif
t"]])

else:

    print("No association rules found")

```

Output

Dataset:

	Assignment	Certificate	Discussion Forum	Quiz	Video Lecture
0	False	False	False	True	True
1	True	False	False	False	True
2	False	False	True	True	False
3	True	False	False	True	True
4	True	False	True	False	False
5	False	True	False	False	True
6	False	True	False	True	False
7	False	False	False	True	True
8	True	True	False	False	False
9	False	True	False	True	True

Minimum Support: 0.2

Frequent Itemsets:

	support	itemsets
0	0.4	frozenset({Assignment})
1	0.4	frozenset({Certificate})
2	0.2	frozenset({Discussion Forum})
3	0.6	frozenset({Quiz})
4	0.6	frozenset({Video Lecture})
5	0.2	frozenset({Assignment, Video Lecture})
6	0.2	frozenset({Quiz, Certificate})
7	0.2	frozenset({Certificate, Video Lecture})
8	0.4	frozenset({Quiz, Video Lecture})

Association Rules:

	antecedents	consequents	support \
3	frozenset({Quiz})	frozenset({Video Lecture})	0.4
4	frozenset({Video Lecture})	frozenset({Quiz})	0.4
0	frozenset({Assignment})	frozenset({Video Lecture})	0.2
1	frozenset({Certificate})	frozenset({Quiz})	0.2
2	frozenset({Certificate})	frozenset({Video Lecture})	0.2

	confidence	lift
3	0.666667	1.111111
4	0.666667	1.111111
0	0.500000	0.833333
1	0.500000	0.833333
2	0.500000	0.833333

Minimum Support: 0.3

Frequent Itemsets:

	support	itemsets
0	0.4	frozenset({Assignment})
1	0.4	frozenset({Certificate})
2	0.6	frozenset({Quiz})
3	0.6	frozenset({Video Lecture})
4	0.4	frozenset({Quiz, Video Lecture})

Association Rules:

	antecedents	consequents	support \
0	frozenset({Quiz})	frozenset({Video Lecture})	0.4
1	frozenset({Video Lecture})	frozenset({Quiz})	0.4

	confidence	lift
0	0.666667	1.111111
1	0.666667	1.111111

Slip 3

Q1. Create a dataframe containing information about different vehicles such as Fuel_Efficiency, Engine_Power, Vehicle_Weight, Engine_Size, and Acceleration. Add 5 records in dataframe, including outlier. Create Box Plots for Fuel Efficiency and Engine Power and identify outliers.

```
import pandas as pd
import matplotlib.pyplot as plt

data={
    "Fuel_Efficiency": [18,22,25,20,60] ,
    "Engine_Power": [100,120,150,130,400] ,
    "Vehicle_Weight": [1200,1400,1600,1350,1800] ,
    "Engine_Size": [1.2,1.5,1.8,1.6,3.5] ,
    "Acceleration": [12,10,9,11,6]
}

df=pd.DataFrame(data)

print("Vehicle DataFrame:")
print(df)

plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.boxplot(df["Fuel_Efficiency"])
plt.title("Fuel Efficiency")
plt.ylabel("Fuel Efficiency")

plt.subplot(1,2,2)
plt.boxplot(df["Engine_Power"])
plt.title("Engine Power")
```

```

plt.ylabel("Engine Power")

plt.tight_layout()

plt.show()

def find_outliers(column):
    Q1=df[column].quantile(0.25)
    Q3=df[column].quantile(0.75)
    IQR=Q3-Q1
    lower=Q1-1.5*IQR
    upper=Q3+1.5*IQR
    return df[(df[column]<lower)|(df[column]>upper)][column]

print("Fuel Efficiency Outliers:")
print(find_outliers("Fuel_Efficiency"))

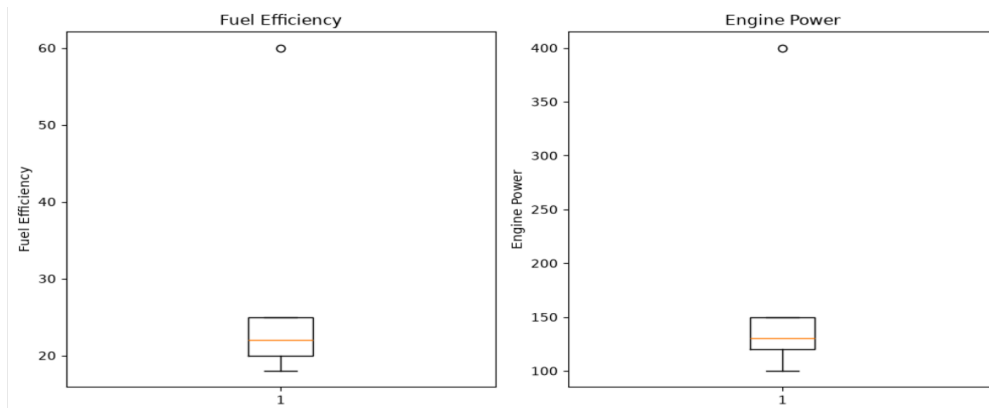
print("Engine Power Outliers:")
print(find_outliers("Engine_Power"))

```

Output

Vehicle DataFrame:

	Fuel_Efficiency	Engine_Power	Vehicle_Weight	Engine_Size	Acceleration
0	18	100	1200	1.2	12
1	22	120	1400	1.5	10
2	25	150	1600	1.8	9
3	20	130	1350	1.6	11
4	60	400	1800	3.5	6



Fuel Efficiency Outliers:

4 60

Name: Fuel_Efficiency, dtype: int64

Engine Power Outliers:

4 400

Name: Engine_Power, dtype: int64

Q2. Consider the Salary Dataset. Write a Python program to read the dataset and display its information. Preprocess the data if required and split it into training and testing sets. Apply Simple Linear Regression to predict salary based on years of experience. Plot the regression line and evaluate the model using appropriate metrics such as Mean Squared Error.

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error

data={

    "YearsExperience": [1.1,1.3,1.5,2.0,2.2,2.9,3.0,3.2,3.7,3.9,4.0,4.5,4.9,5.1,5.3,5.9,6.0,6.8,7.1,7.9,8.2,8.7,9.0,9.5,9.6,10.3] ,

    "Salary": [39343,46205,37731,43525,39891,56642,60150,54445,57189,63218,55794,56957,57081,66029,83088,81363,93940,91738,98273,101302,113812,109431,105582,116969,112635,122391]

}
```

```
df=pd.DataFrame(data)

print("Dataset Information:")
df.info()

print("\nDataset:")
print(df.head())

df=df.dropna()

X=df[["YearsExperience"]]
y=df["Salary"]

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)

model=LinearRegression()
model.fit(X_train,y_train)

y_pred=model.predict(X_test)

print("\nMean Squared Error:",mean_squared_error(y_test,y_pred))

X_sorted=X_test.sort_values(by="YearsExperience")
y_sorted=model.predict(X_sorted)

plt.scatter(X_test,y_test,label="Actual Data")
plt.plot(X_sorted,y_sorted,label="Regression Line")
plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.title("Simple Linear Regression")
plt.legend()
plt.show()
```

Output

Dataset Information:

```
<class 'pandas.DataFrame'>
```

RangeIndex: 26 entries, 0 to 25

Data columns (total 2 columns):

#	Column	Non-Null Count	Dtype
0	YearsExperience	26 non-null	float64
1	Salary	26 non-null	int64

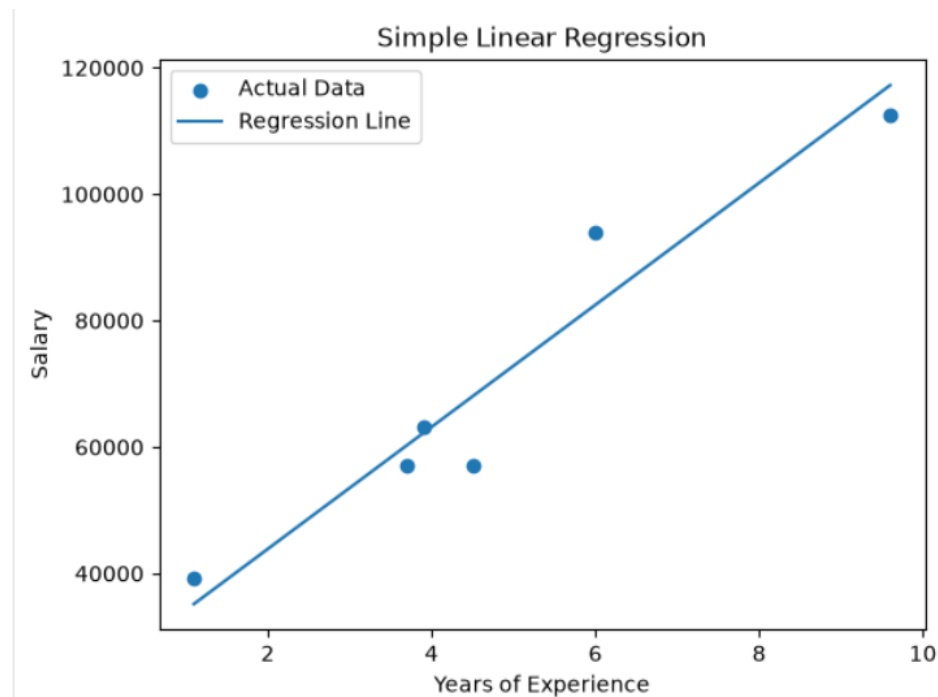
dtypes: float64(1), int64(1)

memory usage: 544.0 bytes

Dataset:

	YearsExperience	Salary
0	1.1	39343
1	1.3	46205
2	1.5	37731
3	2.0	43525
4	2.2	39891

Mean Squared Error: 50401346.87518399



OR

Q2. Apply the Random Forest algorithm on the Heart Disease dataset to predict the presence of heart disease. Preprocess the data, evaluate the model using accuracy and confusion matrix.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import
accuracy_score, confusion_matrix, ConfusionMatrixDisplay

data={

"Age": [63,37,41,56,57,57,56,44,52,57,54,48,49,64,58,50,58,66,43,69] ,
    "Sex": [1,1,0,1,1,1,0,1,1,1,1,0,1,1,1,1,0,1,1,1] ,
    "ChestPain": [3,2,1,1,0,0,1,2,3,0,2,1,1,3,3,2,2,0,2,1] ,

"RestBP": [145,130,130,120,120,140,140,120,172,150,140,130,130,110,15
0,120,132,160,150,140] ,

"Cholesterol": [233,250,204,236,354,192,294,263,199,168,239,275,266,2
11,283,219,224,246,247,239] ,

"MaxHR": [150,187,172,178,163,148,153,173,162,174,160,139,171,144,162
,158,173,150,171,151] ,

"Oldpeak": [2.3,3.5,1.4,0.8,0.6,0.4,1.3,0,0.5,1.6,1.2,0.2,0.6,1.8,1,1
.6,3.2,0,1.5,1.8] ,
    "Target": [1,1,1,1,1,1,1,1,1,1,1,0,0,1,0,0,1,1,0,1]
}

df=pd.DataFrame(data)

print("Dataset Information:")

df.info()
```

```
print("\nDataset:")
print(df.head())

df=df.dropna()

X=df.drop("Target",axis=1)
y=df["Target"]

X_train,X_test,y_train,y_test=train_test_split(
    X,y,test_size=0.2,random_state=42,stratify=y
)

model=RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

model.fit(X_train,y_train)

y_pred=model.predict(X_test)

accuracy=accuracy_score(y_test,y_pred)
cm=confusion_matrix(y_test,y_pred)

print("\nAccuracy:",accuracy)
print("\nConfusion Matrix:")
print(cm)

disp=ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["No Disease","Disease"]
```

```
)
```

```
disp.plot()
```

```
plt.title("Random Forest - Heart Disease")
```

```
plt.show()
```

Output

Dataset Information:

```
<class 'pandas.DataFrame'>
```

RangeIndex: 20 entries, 0 to 19

Data columns (total 8 columns):

#	Column	Non-Null Count	Dtype
0	Age	20 non-null	int64
1	Sex	20 non-null	int64
2	ChestPain	20 non-null	int64
3	RestBP	20 non-null	int64
4	Cholesterol	20 non-null	int64
5	MaxHR	20 non-null	int64
6	Oldpeak	20 non-null	float64
7	Target	20 non-null	int64

dtypes: float64(1), int64(7)

memory usage: 1.4 KB

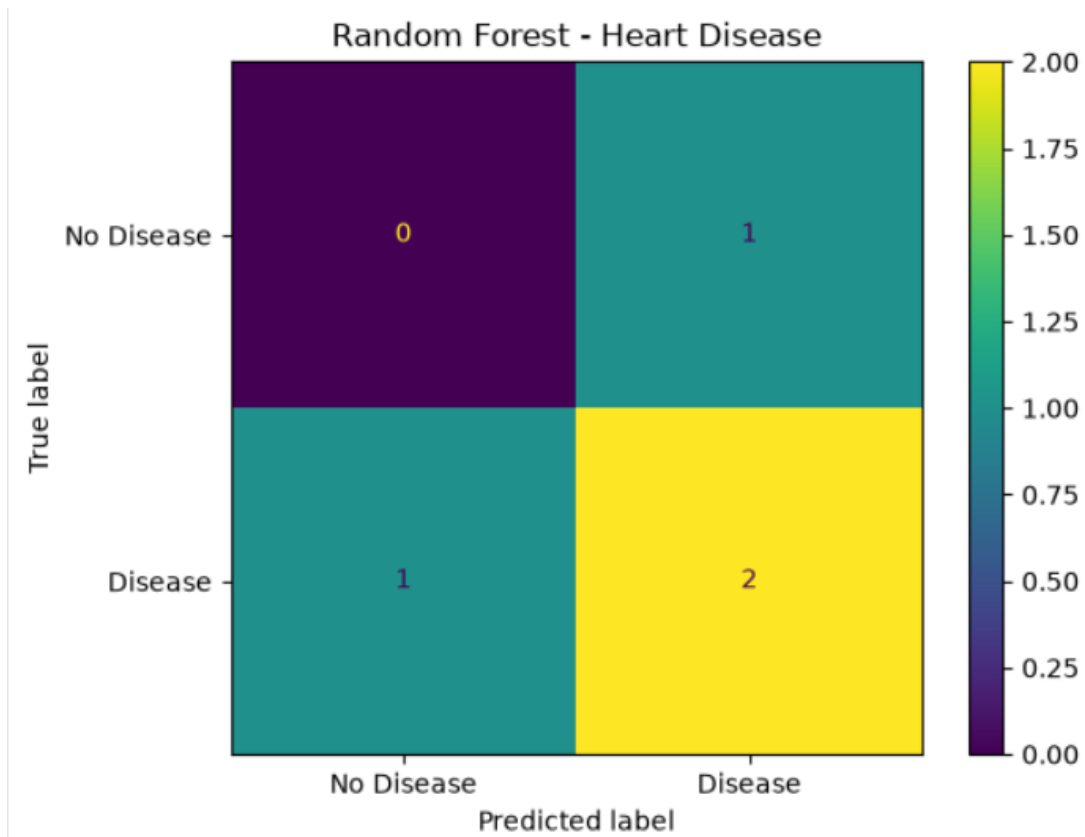
Dataset:

	Age	Sex	ChestPain	RestBP	Cholesterol	MaxHR	Oldpeak	Target
0	63	1	3	145	233	150	2.3	1
1	37	1	2	130	250	187	3.5	1
2	41	0	1	130	204	172	1.4	1
3	56	1	1	120	236	178	0.8	1
4	57	1	0	120	354	163	0.6	1

Accuracy: 0.5

Confusion Matrix:


```
[[0 1]  
 [1 2]]
```



Slip 4

Q1. A survey was conducted to determine the preferred payment method among customers as shown below: payment_methods = ["UPI", "Credit Card", "Debit Card", "Cash", "Net Banking"] customers = [150, 80, 60, 40, 30] Create a Pie Chart to represent the percentage of customers using each payment method. Display the percentage contribution of each category and add an appropriate title.

```
import matplotlib.pyplot as plt

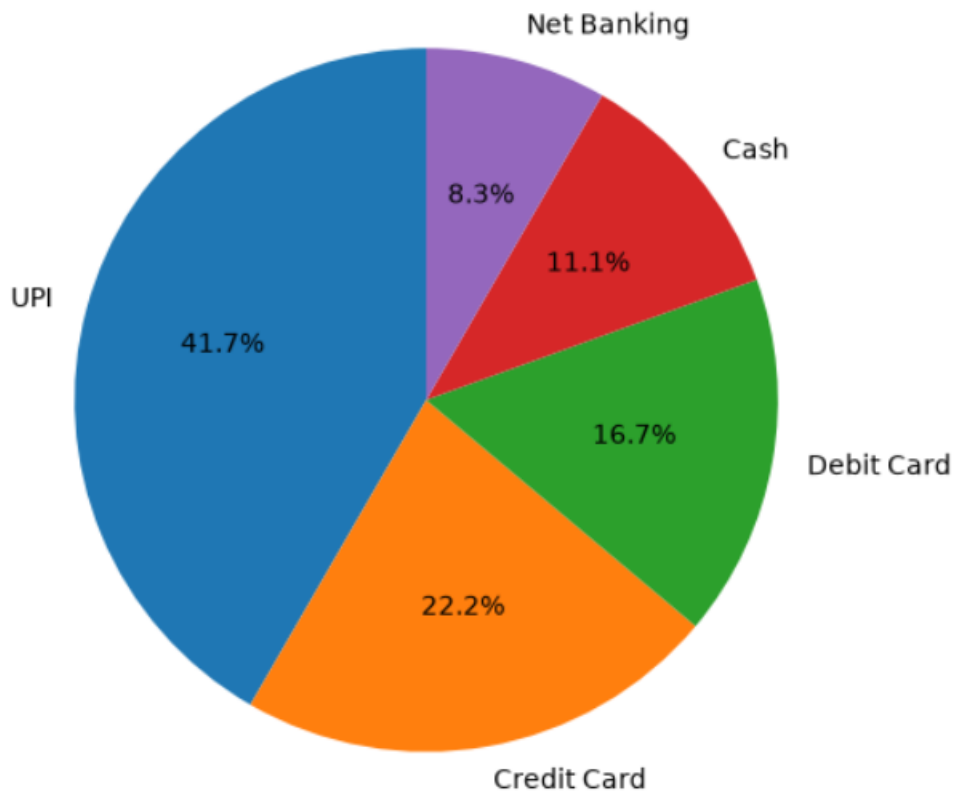
payment_methods=["UPI","Credit Card","Debit Card","Cash","Net Banking"]

customers=[150,80,60,40,30]

plt.figure(figsize=(8,6))
plt.pie(
    customers,
    labels=payment_methods,
    autopct="%1.1f%%",
    startangle=90
)
plt.title("Customer Preferred Payment Methods")
plt.show()
```

Output

Customer Preferred Payment Methods



Q2. Apply Logistic Regression on an Online Gaming dataset to classify players as Casual or Professional gamers based on features such as daily play time, achievements unlocked, in-game purchases, and gaming level. Evaluate the model using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import
accuracy_score, precision_score, recall_score, f1_score, confusion_matri
x, ConfusionMatrixDisplay

import matplotlib.pyplot as plt

data={

    "Daily_Play_Time": [1,2,3,4,5,6,7,8,2,3,4,5,6,7,8,9,1,2,7,9] ,

    "Achievements": [2,4,5,8,10,12,15,18,3,6,7,9,13,16,20,22,1,3,14,25] ,
```

```

    "In_Game_Purchases": [0,1,1,2,3,4,5,6,0,1,2,2,4,5,7,8,0,1,6,9],
    "Gaming_Level": [1,2,3,4,5,6,7,8,2,3,4,5,6,7,8,9,1,2,7,10],
    "Player_Type": [0,0,0,0,0,1,1,1,0,0,0,0,1,1,1,1,0,0,1,1]
}

df=pd.DataFrame(data)

print("Dataset:")
print(df)

print("\nDataset Information:")
df.info()

df=df.dropna()

X=df[["Daily_Play_Time","Achievements","In_Game_Purchases","Gaming_Level"]]
y=df["Player_Type"]

X_train,X_test,y_train,y_test=train_test_split(
    X,y,test_size=0.2,random_state=42,stratify=y
)

model=LogisticRegression()
model.fit(X_train,y_train)

y_pred=model.predict(X_test)

accuracy=accuracy_score(y_test,y_pred)
precision=precision_score(y_test,y_pred)
recall=recall_score(y_test,y_pred)
f1=f1_score(y_test,y_pred)
cm=confusion_matrix(y_test,y_pred)

```

```

print("\nAccuracy:",accuracy)
print("Precision:",precision)
print("Recall:",recall)
print("F1-Score:",f1)

print("\nConfusion Matrix:")
print(cm)

disp=ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Casual","Professional"]
)

disp.plot()
plt.title("Logistic Regression - Online Gaming")
plt.show()

```

Output

Dataset:

	Daily_Play_Time	Achievements	In_Game_Purchases	Gaming_Level	\
0	1	2	0	1	
1	2	4	1	2	
2	3	5	1	3	
3	4	8	2	4	
4	5	10	3	5	
5	6	12	4	6	
6	7	15	5	7	
7	8	18	6	8	
8	2	3	0	2	
9	3	6	1	3	
10	4	7	2	4	
11	5	9	2	5	
12	6	13	4	6	

13	7	16	5	7
14	8	20	7	8
15	9	22	8	9
16	1	1	0	1
17	2	3	1	2
18	7	14	6	7
19	9	25	9	10

	Player_Type
0	0
1	0
2	0
3	0
4	0
5	1
6	1
7	1
8	0
9	0
10	0
11	0
12	1
13	1
14	1
15	1
16	0
17	0
18	1
19	1

Dataset Information:

<class 'pandas.DataFrame'>

RangeIndex: 20 entries, 0 to 19

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	Daily_Play_Time	20 non-null	int64
1	Achievements	20 non-null	int64
2	In_Game_Purchases	20 non-null	int64
3	Gaming_Level	20 non-null	int64
4	Player_Type	20 non-null	int64

dtypes: int64(5)

memory usage: 928.0 bytes

Accuracy: 1.0

Precision: 1.0

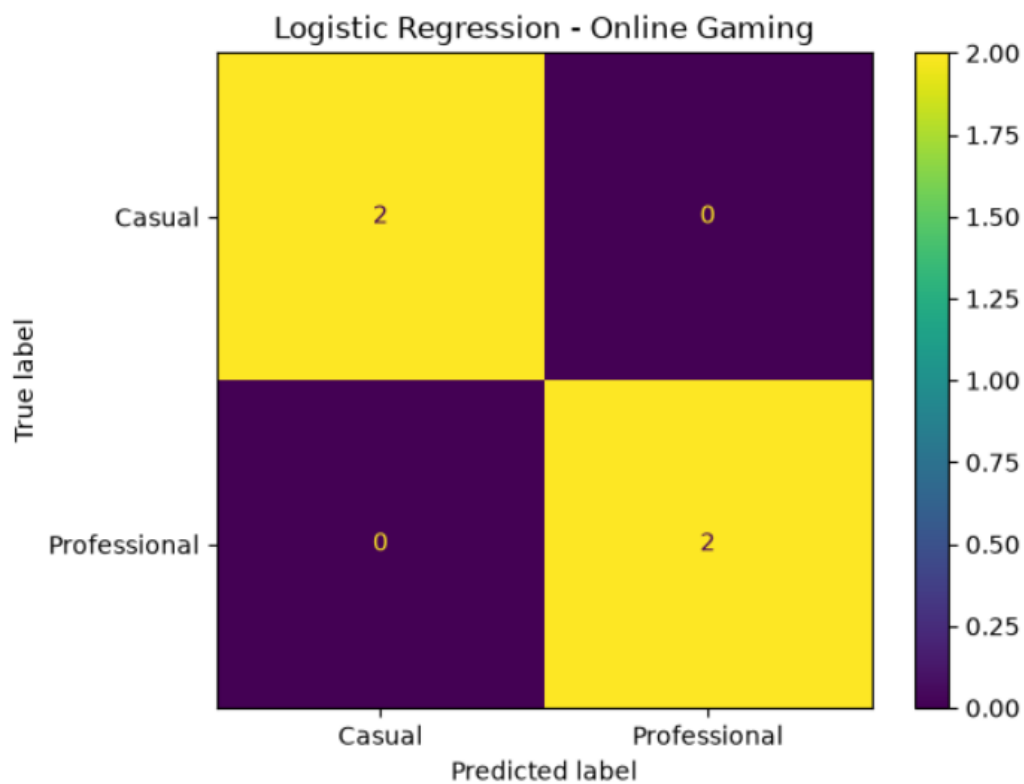
Recall: 1.0

F1-Score: 1.0

Confusion Matrix:

[[2 0]

[0 2]]



OR

Q2. Apply the K-Means Clustering algorithm on the given E-commerce Customer dataset. Read the dataset using Python, inspect and preprocess the data by handling missing values and selecting suitable numerical features. Apply K-Means clustering to segment the customers into different categories and interpret the resulting clusters.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

data={
    "CustomerID": [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15] ,
    "Annual_Income": [15000,18000,22000,25000,30000,35000,40000,45000,50000,60000,70000,80000,90000,100000,120000] ,
    "Spending_Score": [20,25,30,35,40,45,50,55,60,65,70,75,80,85,90] ,
    "Purchase_Frequency": [2,3,3,4,5,5,6,6,7,7,8,8,9,10,10]
}

df=pd.DataFrame(data)

print("Dataset:")
print(df)

print("\nDataset Information:")
df.info()

df=df.dropna()

X=df[["Annual_Income","Spending_Score","Purchase_Frequency"]]

scaler=StandardScaler()
```



```

X_scaled=scaler.fit_transform(X)

kmeans=KMeans(n_clusters=3,random_state=42,n_init=10)
df["Cluster"]=kmeans.fit_predict(X_scaled)

print("\nClustered Customer Dataset:")
print(df)

print("\nCluster Centers:")
print(kmeans.cluster_centers_)

print("\nCluster Interpretation:")
for cluster in sorted(df["Cluster"].unique()):
    cluster_data=df[df["Cluster"]==cluster]
    print("\nCluster",cluster)
    print("Number of Customers:",len(cluster_data))
    print("Average Annual
Income:",cluster_data["Annual_Income"].mean())
    print("Average Spending
Score:",cluster_data["Spending_Score"].mean())
    print("Average Purchase
Frequency:",cluster_data["Purchase_Frequency"].mean())

plt.figure(figsize=(8,6))
plt.scatter(
    df["Annual_Income"],
    df["Spending_Score"],
    c=df["Cluster"],
    cmap="viridis",
    s=100
)
plt.xlabel("Annual Income")
plt.ylabel("Spending Score")
plt.title("K-Means Clustering of E-commerce Customers")

```

```
plt.show()
```

Output

Dataset:

	CustomerID	Annual_Income	Spending_Score	Purchase_Frequency
0	1	15000	20	2
1	2	18000	25	3
2	3	22000	30	3
3	4	25000	35	4
4	5	30000	40	5
5	6	35000	45	5
6	7	40000	50	6
7	8	45000	55	6
8	9	50000	60	7
9	10	60000	65	7
10	11	70000	70	8
11	12	80000	75	8
12	13	90000	80	9
13	14	100000	85	10
14	15	120000	90	10

Dataset Information:

```
<class 'pandas.DataFrame'>
```

RangeIndex: 15 entries, 0 to 14

Data columns (total 4 columns):

#	Column	Non-Null Count	Dtype
0	CustomerID	15 non-null	int64
1	Annual_Income	15 non-null	int64
2	Spending_Score	15 non-null	int64
3	Purchase_Frequency	15 non-null	int64

dtypes: int64(4)

memory usage: 608.0 bytes

Clustered Customer Dataset:

	CustomerID	Annual_Income	Spending_Score	Purchase_Frequency	Cluster
0	1	15000	20	2	2
1	2	18000	25	3	2
2	3	22000	30	3	2
3	4	25000	35	4	2
4	5	30000	40	5	0
5	6	35000	45	5	0
6	7	40000	50	6	0
7	8	45000	55	6	0
8	9	50000	60	7	0
9	10	60000	65	7	0
10	11	70000	70	8	1
11	12	80000	75	8	1
12	13	90000	80	9	1
13	14	100000	85	10	1
14	15	120000	90	10	1

Cluster Centers:

```

[[-0.32007762 -0.11572751 -0.08146882]
 [ 1.23763348  1.15727512  1.14056344]
 [-1.06692542 -1.27300264 -1.30350107]]

```

Cluster Interpretation:

Cluster 0

Number of Customers: 6

Average Annual Income: 43333.333333333336

Average Spending Score: 52.5

Average Purchase Frequency: 6.0

Cluster 1

Number of Customers: 5

Average Annual Income: 92000.0

Average Spending Score: 80.0

Average Purchase Frequency: 9.0

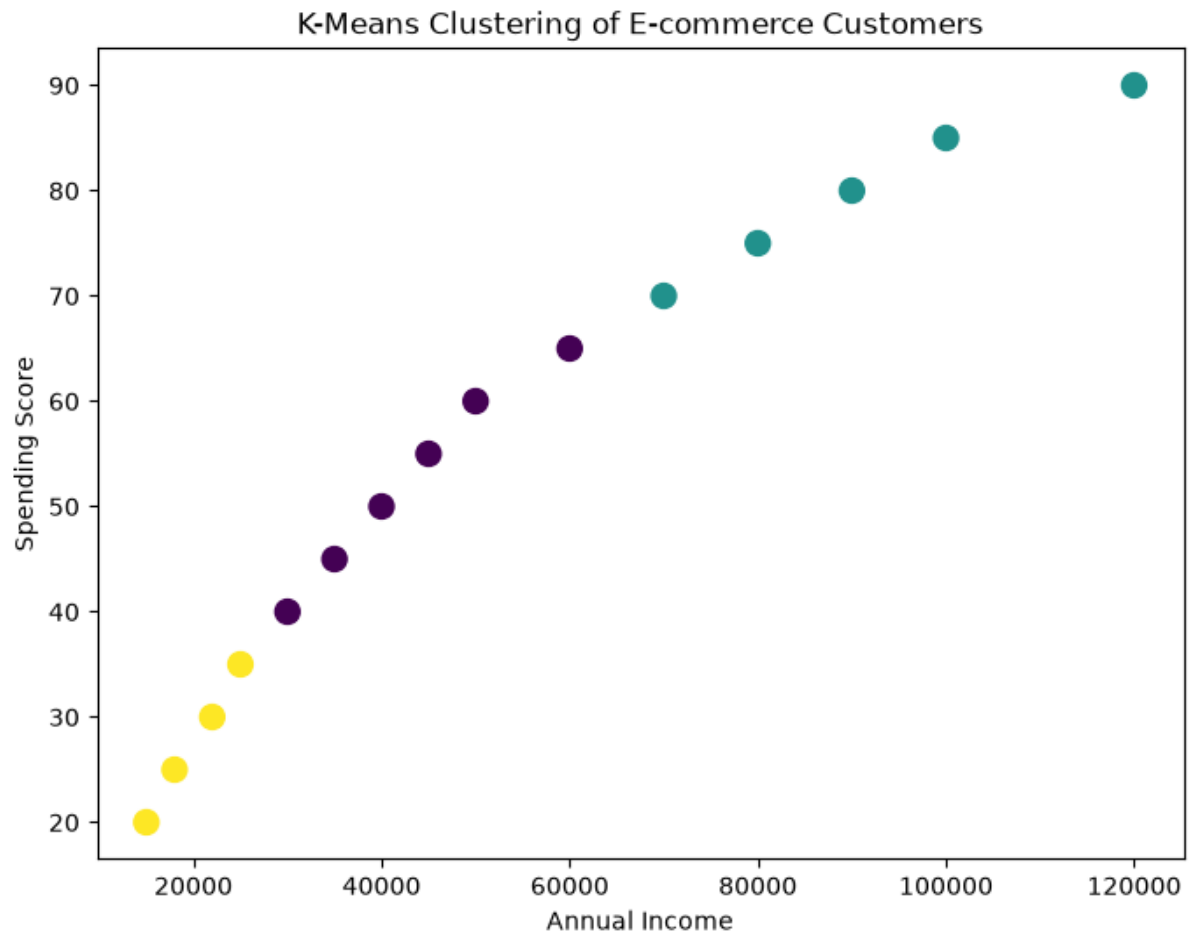
Cluster 2

Number of Customers: 4

Average Annual Income: 20000.0

Average Spending Score: 27.5

Average Purchase Frequency: 3.0



Slip 5

Q1. Write a Python program to create a customer_DataFrame with suitable attributes such as Customer_ID, Gender, City, and Membership_Type. Perform label encoding technique on appropriate categorical attributes. Display the DataFrame before and after encoding.

```
import pandas as pd

from sklearn.preprocessing import LabelEncoder

data = {
    "Customer_ID": [101, 102, 103, 104, 105],
    "Gender": ["Male", "Female", "Female", "Male", "Female"],
    "City": ["Pune", "Mumbai", "Pune", "Nashik", "Mumbai"],
    "Membership_Type": ["Gold", "Silver", "Gold", "Bronze",
                        "Silver"]
}

customer_DataFrame = pd.DataFrame(data)

print("DataFrame Before Encoding:")
print(customer_DataFrame)

le = LabelEncoder()

for column in ["Gender", "City", "Membership_Type"]:
    customer_DataFrame[column] =
le.fit_transform(customer_DataFrame[column])

print("\nDataFrame After Label Encoding:")
print(customer_DataFrame)
```

Output

DataFrame Before Encoding:

	Customer_ID	Gender	City	Membership_Type
0	101	Male	Pune	Gold

1	102	Female	Mumbai	Silver
2	103	Female	Pune	Gold
3	104	Male	Nashik	Bronze
4	105	Female	Mumbai	Silver

DataFrame After Label Encoding:

	Customer_ID	Gender	City	Membership_Type
0	101	1	2	1
1	102	0	0	2
2	103	0	2	1
3	104	1	1	0
4	105	0	0	2

Q2. Generate a DataFrame containing customer information for Netflix and Amazon Prime users. Create a Venn diagram and a Pie chart to show the customers using only Netflix, only Amazon Prime, and both services. Add appropriate labels, colors, and titles to both visualizations.

```
import pandas as pd

import matplotlib.pyplot as plt

from matplotlib_venn import venn2

data = {
    "Customer_ID": [101, 102, 103, 104, 105, 106, 107, 108],
    "Netflix": ["Yes", "Yes", "No", "Yes", "No", "Yes", "No",
"Yes"],
    "Amazon_Prime": ["No", "Yes", "Yes", "Yes", "Yes", "No", "Yes",
"Yes"]
}

df = pd.DataFrame(data)

print("Customer DataFrame:")

print(df)

netflix = set(df[df["Netflix"] == "Yes"]["Customer_ID"])
```

```

amazon = set(df[df["Amazon_Prime"] == "Yes"]["Customer_ID"])

only_netflix = netflix - amazon
only_amazon = amazon - netflix
both = netflix & amazon

print("\nOnly Netflix:", len(only_netflix))
print("Only Amazon Prime:", len(only_amazon))
print("Both Services:", len(both))

plt.figure(figsize=(6, 5))
venn2([netflix, amazon], set_labels=("Netflix", "Amazon Prime"))
plt.title("Netflix and Amazon Prime Users")
plt.show()

labels = ["Only Netflix", "Only Amazon Prime", "Both Services"]
values = [len(only_netflix), len(only_amazon), len(both)]

plt.figure(figsize=(6, 5))
plt.pie(values, labels=labels, autopct="%1.1f%%", startangle=90)
plt.title("Customer Usage of Netflix and Amazon Prime")
plt.show()

```

Output

Customer DataFrame:

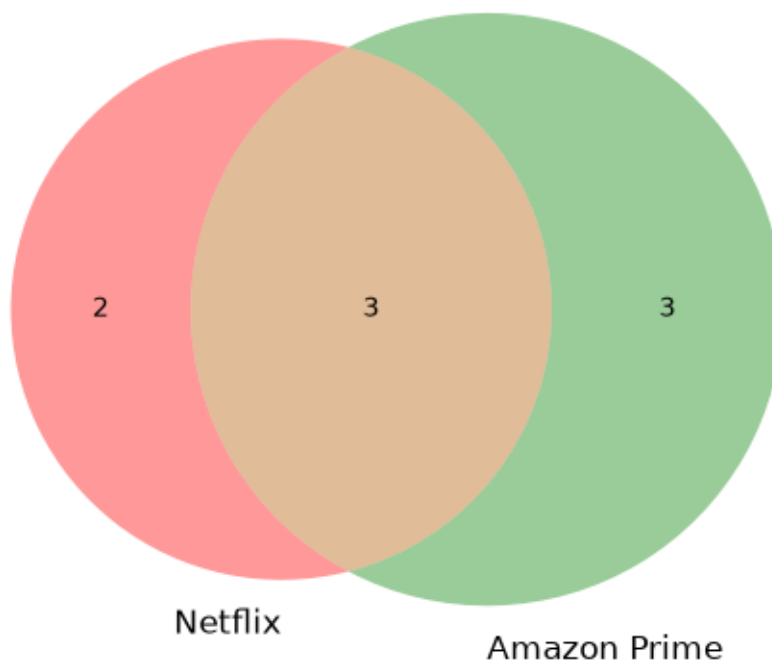
	Customer_ID	Netflix	Amazon_Prime
0	101	Yes	No
1	102	Yes	Yes
2	103	No	Yes
3	104	Yes	Yes
4	105	No	Yes
5	106	Yes	No
6	107	No	Yes
7	108	Yes	Yes

Only Netflix: 2

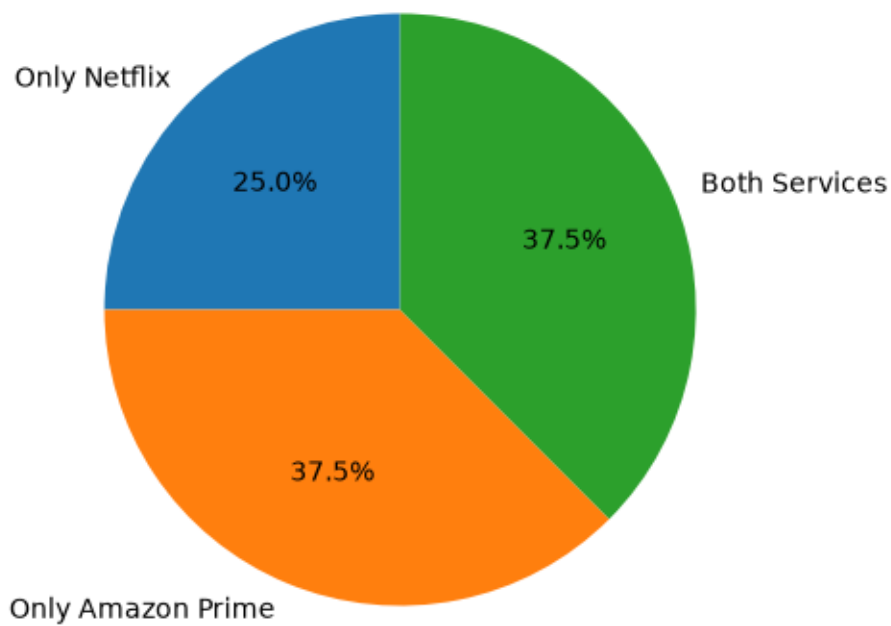
Only Amazon Prime: 3

Both Services: 3

Netflix and Amazon Prime Users



Customer Usage of Netflix and Amazon Prime



OR

Q2. Use the K-Nearest Neighbors (KNN) algorithm to classify emails as spam or non-spam using the Spambase dataset. Preprocess the data, train the KNN classifier, and evaluate its performance using accuracy, confusion matrix, precision, recall, and F1-score.

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, confusion_matrix,
precision_score, recall_score, f1_score

data = {
    "word_freq_make": [0.21, 0.06, 0.00, 0.12, 0.15, 0.01, 0.30, 0.02,
0.10, 0.25, 0.03, 0.18],
    "word_freq_address": [0.28, 0.00, 0.20, 0.05, 0.00, 0.15, 0.30, 0.01,
0.00, 0.25, 0.02, 0.18],
    "word_freq_money": [0.65, 0.10, 0.00, 0.25, 0.50, 0.05, 0.80, 0.02,
0.15, 0.70, 0.08, 0.40],
    "capital_run_length_total": [120, 35, 20, 70, 150, 25, 200, 30, 45,
180, 40, 100],
    "capital_run_length_longest": [15, 5, 3, 10, 20, 4, 25, 5, 6, 22, 5,
12],
    "spam": [1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1]
}

df = pd.DataFrame(data)

print("Spambase DataFrame:")
print(df)

X = df.drop("spam", axis=1)
y = df["spam"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
```

```

)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)

print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nPrecision:", precision_score(y_test, y_pred, zero_division=0))
print("Recall:", recall_score(y_test, y_pred, zero_division=0))
print("F1-Score:", f1_score(y_test, y_pred, zero_division=0))

```

Output

Spambase DataFrame:

	word_freq_make	word_freq_address	word_freq_money	\
0	0.21	0.28	0.65	
1	0.06	0.00	0.10	
2	0.00	0.20	0.00	
3	0.12	0.05	0.25	
4	0.15	0.00	0.50	
5	0.01	0.15	0.05	
6	0.30	0.30	0.80	
7	0.02	0.01	0.02	
8	0.10	0.00	0.15	
9	0.25	0.25	0.70	
10	0.03	0.02	0.08	
11	0.18	0.18	0.40	

capital_run_length_total capital_run_length_longest spam

0	120	15	1
1	35	5	0
2	20	3	0
3	70	10	1
4	150	20	1
5	25	4	0
6	200	25	1
7	30	5	0
8	45	6	0
9	180	22	1
10	40	5	0
11	100	12	1

Accuracy: 1.0

Confusion Matrix:

```
[[2 0]
 [0 1]]
```

Precision: 1.0

Recall: 1.0

F1-Score: 1.0

Slip 6

Q1. Create a DataFrame containing the car model, engine size (in litres), and fuel efficiency (in km/L) for the given cars. Create a scatter plot to visualize the relationship between engine size and fuel efficiency. Add an appropriate title, label both axes, and use different colors for the data points.

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Car Model": ["Swift", "Honda City", "Creta", "Baleno",
                  "Fortuner", "Nexon", "i20", "Thar"],
    "Engine Size (L)": [1.2, 1.5, 1.5, 1.2, 2.8, 1.2, 1.2, 2.0],
    "Fuel Efficiency (km/L)": [22, 17, 18, 21, 10, 19, 20, 15]
}

df = pd.DataFrame(data)
print("Car DataFrame:")
print(df)

plt.figure(figsize=(8, 5))

plt.scatter(df["Engine Size (L)"], df["Fuel Efficiency (km/L)"],
            c=range(len(df)), cmap="viridis", s=100)

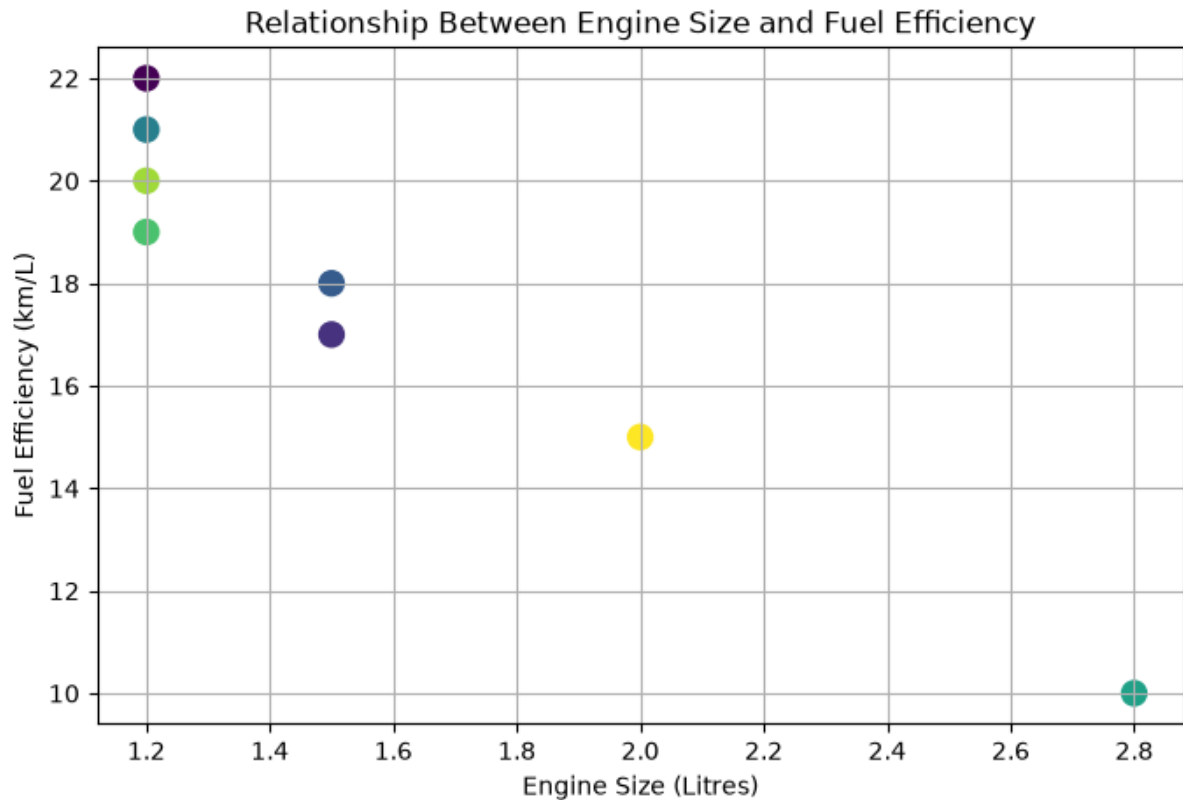
plt.title("Relationship Between Engine Size and Fuel Efficiency")
plt.xlabel("Engine Size (Litres)")
plt.ylabel("Fuel Efficiency (km/L)")
plt.grid(True)
plt.show()
```

Output

Car DataFrame:

	Car Model	Engine Size (L)	Fuel Efficiency (km/L)
0	Swift	1.2	22
1	Honda City	1.5	17
2	Creta	1.5	18
3	Baleno	1.2	21

4	Fortuner	2.8	10
5	Nexon	1.2	19
6	i20	1.2	20
7	Thar	2.0	15



Q2. Use the Sports Performance dataset to develop a Simple Linear Regression model to predict Performance Score based on Training Hours Per Week. Evaluate the model using the R^2 score and plot a scatter graph with the regression line to visualize the relationship between training hours and performance score.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

data = {
    "Training Hours Per Week": [5, 8, 10, 12, 15, 18, 20, 22, 25,
28],
```

```

        "Performance Score": [45, 52, 58, 62, 68, 73, 78, 82, 88, 94]
    }

df = pd.DataFrame(data)

print("Sports Performance DataFrame:")
print(df)

X = df[["Training Hours Per Week"]]
y = df["Performance Score"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

r2 = r2_score(y_test, y_pred)
print("\nR2 Score:", r2)

plt.figure(figsize=(8, 5))
plt.scatter(X, y, color="blue", label="Actual Data")
plt.plot(X, model.predict(X), color="red", label="Regression Line")
plt.title("Training Hours vs Performance Score")
plt.xlabel("Training Hours Per Week")
plt.ylabel("Performance Score")
plt.legend()
plt.grid(True)
plt.show()

```

Output

Sports Performance DataFrame:

	Training Hours Per Week	Performance Score
0	5	45
1	8	52
2	10	58
3	12	62
4	15	68
5	18	73
6	20	78
7	22	82
8	25	88
9	28	94

R^2 Score: 0.998932938744804



OR

Q2. Implement a KNN classifier on the given Customer dataset. Preprocess the data and classify customers into different segments. Evaluate the model using Accuracy and Confusion Matrix .

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

data = {
    "Age": [22, 25, 28, 35, 40, 45, 23, 30, 38, 50, 27, 42],
    "Annual_Income": [25000, 30000, 35000, 50000, 60000, 70000,
28000, 45000, 55000, 80000, 32000, 65000],
    "Spending_Score": [80, 75, 70, 60, 40, 30, 85, 65, 50, 25, 72,
35],
    "Membership_Type": ["Silver", "Silver", "Gold", "Gold",
"Platinum", "Platinum",
                        "Silver", "Gold", "Gold", "Platinum",
"Silver", "Platinum"],
    "Segment": ["High", "High", "High", "Medium", "Low", "Low",
                "High", "Medium", "Medium", "Low", "High", "Low"]
}

df = pd.DataFrame(data)

print("Customer DataFrame:")
print(df)

le = LabelEncoder()
df["Membership_Type"] = le.fit_transform(df["Membership_Type"])
df["Segment"] = le.fit_transform(df["Segment"])
X = df.drop("Segment", axis=1)
y = df["Segment"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
```



```

)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

```

Output

Customer DataFrame:

	Age	Annual_Income	Spending_Score	Membership_Type	Segment
0	22	25000	80	Silver	High
1	25	30000	75	Silver	High
2	28	35000	70	Gold	High
3	35	50000	60	Gold	Medium
4	40	60000	40	Platinum	Low
5	45	70000	30	Platinum	Low
6	23	28000	85	Silver	High
7	30	45000	65	Gold	Medium
8	38	55000	50	Gold	Medium
9	50	80000	25	Platinum	Low
10	27	32000	72	Silver	High
11	42	65000	35	Platinum	Low

Accuracy: 1.0

Confusion Matrix:

```

[[1 0 0]
 [0 1 0]
 [0 0 1]]

```

Slip 7

Q1. Consider the following dataset representing the number of students enrolled in different courses: courses = ["BCA", "BBA", "B.Com", "B.Sc", "BA"] students = [60, 45, 75, 50, 35] Create a Bar Chart to visualize the number of students enrolled in each course. Use different bar colors, add an appropriate title, and label both axes.

```
import matplotlib.pyplot as plt

courses = ["BCA", "BBA", "B.Com", "B.Sc", "BA"]
students = [60, 45, 75, 50, 35]

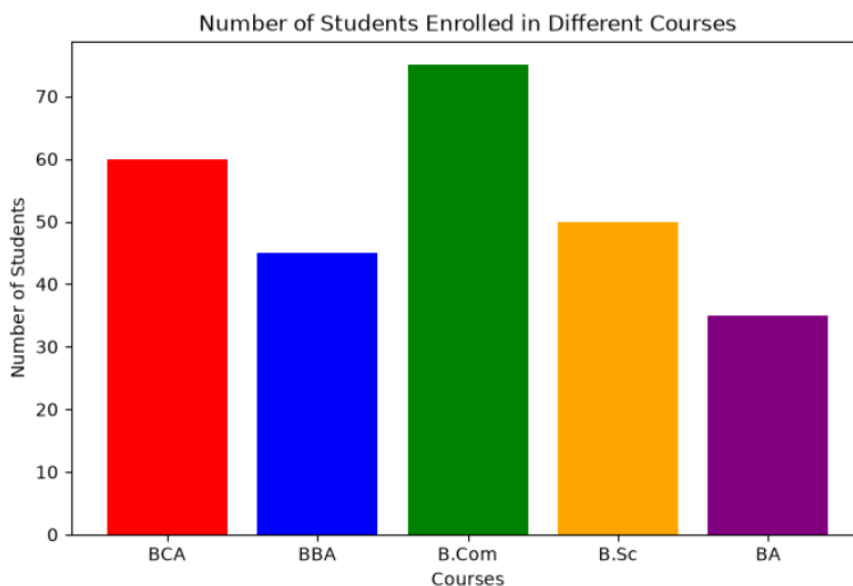
plt.figure(figsize=(8, 5))

plt.bar(courses, students, color=["red", "blue", "green", "orange",
"purple"])

plt.title("Number of Students Enrolled in Different Courses")
plt.xlabel("Courses")
plt.ylabel("Number of Students")

plt.show()
```

Output



Q2. Implement a KNN classifier on the given Student dataset. Preprocess the data and classify students into different performance categories. Evaluate the model using Accuracy and Confusion Matrix.

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

data = {
    "Study_Hours": [2, 3, 4, 5, 6, 7, 8, 9, 3, 6, 8, 10],
    "Attendance": [60, 65, 70, 75, 80, 85, 90, 95, 68, 82, 88, 98],
    "Assignment_Score": [50, 55, 60, 65, 70, 75, 80, 90, 58, 72, 85, 95],
    "Performance": ["Low", "Low", "Low", "Medium", "Medium",
"Medium",
                    "High", "High", "Low", "Medium", "High",
"High"]
}

df = pd.DataFrame(data)

print("Student DataFrame:")
print(df)

le = LabelEncoder()
df["Performance"] = le.fit_transform(df["Performance"])

X = df.drop("Performance", axis=1)
y = df["Performance"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
```

```

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)

print("\nAccuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

```

Output

Student DataFrame:

	Study_Hours	Attendance	Assignment_Score	Performance
0	2	60	50	Low
1	3	65	55	Low
2	4	70	60	Low
3	5	75	65	Medium
4	6	80	70	Medium
5	7	85	75	Medium
6	8	90	80	High
7	9	95	90	High
8	3	68	58	Low
9	6	82	72	Medium
10	8	88	85	High
11	10	98	95	High

Accuracy: 1.0

Confusion Matrix:

```
[[1 0 0]
 [0 1 0]
 [0 0 1]]
```

OR

Q2. Write a Python program to perform market basket analysis on the Groceries dataset using the Apriori algorithm. Read and explore the dataset, preprocess the data by handling missing values, convert categorical transaction data into a suitable numeric format, and generate frequent itemsets and association rules. Repeat the analysis using minimum support values 0.3 and 0.4.

S.No.	Items Purchased
1	Basmati Rice, Toor Dal, Turmeric Powder, Cooking Oil
2	Atta, Sugar, Tea, Milk
3	Rice, Moong Dal, Salt, Cooking Oil
4	Wheat Flour, Chickpeas, Turmeric Powder, Cumin Seeds
5	Poha, Peanuts, Onions, Cooking Oil
6	Basmati Rice, Paneer, Garam Masala, Tomatoes
7	Atta, Potatoes, Onions, Cooking Oil

```
import pandas as pd

from mlxtend.frequent_patterns import apriori, association_rules

data = {
    "S.No.": [1, 2, 3, 4, 5, 6, 7],
    "Items Purchased": [
        "Basmati Rice, Toor Dal, Turmeric Powder, Cooking Oil",
        "Atta, Sugar, Tea, Milk",
        "Rice, Moong Dal, Salt, Cooking Oil",
        "Wheat Flour, Chickpeas, Turmeric Powder, Cumin Seeds",
        "Poha, Peanuts, Onions, Cooking Oil",
        "Basmati Rice, Paneer, Garam Masala, Tomatoes",
```

```

        "Atta, Potatoes, Onions, Cooking Oil"
    ]
}

df = pd.DataFrame(data)

print("Original Dataset:")
print(df)

df["Items Purchased"] = df["Items Purchased"].fillna("")

transactions = df["Items Purchased"].apply(
    lambda x: [item.strip() for item in x.split(",")]
)

items = sorted(set(item for transaction in transactions for item in
transaction))

basket = pd.DataFrame(
    False,
    index=range(len(transactions)),
    columns=items
)

for i, transaction in enumerate(transactions):
    for item in transaction:
        basket.loc[i, item] = True

print("\nTransaction Data:")
print(basket)

for support in [0.3, 0.4]:

```

```

print("\nMinimum Support:", support)

frequent_itemsets = apriori(
    basket,
    min_support=support,
    use_colnames=True
)

print("\nFrequent Itemsets:")
print(frequent_itemsets)

if len(frequent_itemsets) > 0:

    rules = association_rules(
        frequent_itemsets,
        metric="confidence",
        min_threshold=0.5
    )

    print("\nAssociation Rules:")

    if len(rules) > 0:
        print(rules[
            ["antecedents", "consequents",
            "support", "confidence", "lift"]
        ])
    else:
        print("No association rules generated.")

```

Output

Original Dataset:

S.No.	Items Purchased
0	1 Basmati Rice, Toor Dal, Turmeric Powder, Cooki...

1	2	Atta, Sugar, Tea, Milk
2	3	Rice, Moong Dal, Salt, Cooking Oil
3	4	Wheat Flour, Chickpeas, Turmeric Powder, Cumin...
4	5	Poha, Peanuts, Onions, Cooking Oil
5	6	Basmati Rice, Paneer, Garam Masala, Tomatoes
6	7	Atta, Potatoes, Onions, Cooking Oil

Transaction Data:

	Atta Masala	Basmati Rice	Chickpeas	Cooking Oil	Cumin Seeds	Garam Masala
--	----------------	-----------------	-----------	-------------	-------------	-----------------

0	False	True	False	True	False	False
1	True	False	False	False	False	False
2	False	False	False	True	False	False
3	False	False	True	False	True	False
4	False	False	False	True	False	False
5	False	True	False	False	False	False
6	True	False	False	True	False	False

	Milk Salt	Moong Dal	Onions	Paneer	...	Poha	Potatoes	Rice
--	--------------	-----------	--------	--------	-----	------	----------	------

0	False	False	False	False	...	False	False	False
1	True	False	False	False	...	False	False	False
2	False	True	False	False	...	False	False	True
3	False	False	False	False	...	False	False	False
4	False	False	True	False	...	True	False	False

5	False	False	False	True	...	False	False	False
---	-------	-------	-------	------	-----	-------	-------	-------

6	False	False	True	False	...	False	True	False
---	-------	-------	------	-------	-----	-------	------	-------

	Sugar	Tea	Tomatoes	Toor Dal	Turmeric Powder	Wheat	Flour
0	False	False	False	True	True	False	False
1	True	True	False	False	False	False	False
2	False	False	False	False	False	False	False
3	False	False	False	False	True	True	True
4	False	False	False	False	False	False	False
5	False	False	True	False	False	False	False
6	False	False	False	False	False	False	False

[7 rows x 21 columns]

Minimum Support: 0.3

Frequent Itemsets:

	support	itemsets
0	0.571429	frozenset({Cooking Oil})

Association Rules:

No association rules generated.

Minimum Support: 0.4

Frequent Itemsets:

	support	itemsets
0	0.571429	frozenset({Cooking Oil})

Association Rules:

No association rules generated.

Slip 8

Q1. Create a suitable dataset containing information about earthquakes using Latitude, Longitude, Depth, and Magnitude. Perform the following tasks: a. Create a Geospatial Visualization to display the locations of earthquakes on a map. b. Use color to represent the magnitude of each earthquake.

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Latitude": [19.07, 28.61, 23.02, 34.05, 35.68, 13.08, 22.57,
26.85],
    "Longitude": [72.87, 77.21, 72.57, 118.24, 139.65, 80.27, 88.36,
80.95],
    "Depth": [10, 15, 25, 30, 40, 12, 18, 22],
    "Magnitude": [4.2, 5.1, 3.8, 6.2, 5.7, 4.5, 3.9, 6.0]
}

df = pd.DataFrame(data)

print("Earthquake DataFrame:")
print(df)

plt.figure(figsize=(10, 6))

scatter = plt.scatter(
    df["Longitude"],
    df["Latitude"],
    c=df["Magnitude"],
    cmap="hot",
    s=df["Magnitude"] * 50,
    edgecolors="black"
)
```

```
plt.colorbar(scatter, label="Magnitude")
```

```
plt.title("Geospatial Visualization of Earthquakes")
```

```
plt.xlabel("Longitude")
```

```
plt.ylabel("Latitude")
```

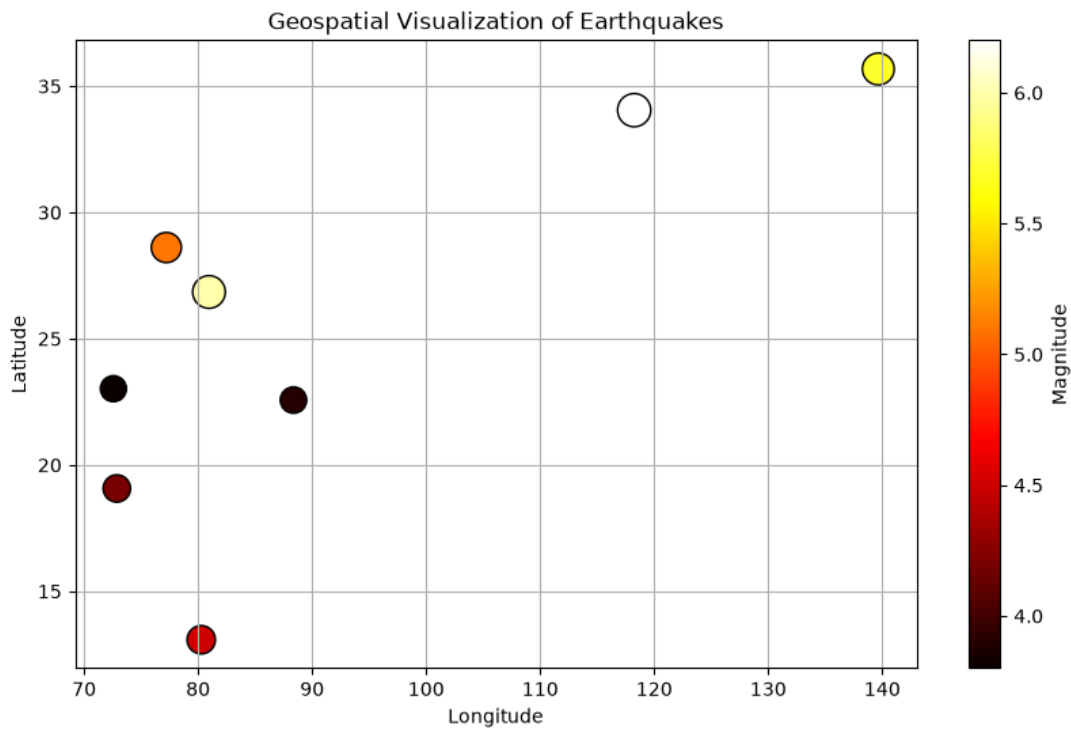
```
plt.grid(True)
```

```
plt.show()
```

Output

Earthquake DataFrame:

	Latitude	Longitude	Depth	Magnitude
0	19.07	72.87	10	4.2
1	28.61	77.21	15	5.1
2	23.02	72.57	25	3.8
3	34.05	118.24	30	6.2
4	35.68	139.65	40	5.7
5	13.08	80.27	12	4.5
6	22.57	88.36	18	3.9
7	26.85	80.95	22	6.0



Q2. Create a suitable dataset named Loan_application.csv containing at least 10 records with the attributes Application_ID, Age, Income, Credit_Score, Loan_Amount, Employment_Years, Debt_Ratio, and Loan_Status, where Loan_Status represents whether the loan application is Approved or Rejected. Write a Python program to perform the following data reduction techniques: a) Feature Selection: Identify the important features for predicting Loan_Status. b) Feature Extraction: Reduce the number of numerical attributes using a suitable technique such as Principal Component Analysis (PCA).

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.ensemble import RandomForestClassifier

from sklearn.decomposition import PCA


data = {
    "Application_ID": [101, 102, 103, 104, 105, 106, 107, 108, 109,
110],
    "Age": [25, 32, 45, 29, 51, 38, 27, 42, 35, 48],
    "Income": [30000, 45000, 70000, 35000, 85000, 60000, 32000,
75000, 50000, 90000],
    "Credit_Score": [650, 720, 780, 680, 800, 740, 620, 770, 700,
810],
    "Loan_Amount": [200000, 300000, 500000, 250000, 400000, 350000,
180000, 450000, 280000, 500000],
    "Employment_Years": [2, 5, 15, 3, 20, 10, 1, 12, 6, 18],
    "Debt_Ratio": [0.45, 0.35, 0.25, 0.50, 0.20, 0.30, 0.55, 0.22,
0.40, 0.18],
    "Loan_Status": ["Rejected", "Approved", "Approved", "Rejected",
"Approved",
                    "Approved", "Rejected", "Approved", "Rejected",
"Approved"]
}

df = pd.DataFrame(data)
```

```

df.to_csv("Loan_application.csv", index=False)

print("Loan Application Dataset:")
print(df)

le = LabelEncoder()
df["Loan_Status"] = le.fit_transform(df["Loan_Status"])

X = df.drop(["Application_ID", "Loan_Status"], axis=1)
y = df["Loan_Status"]

# a) Feature Selection
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X, y)

importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
})

importance = importance.sort_values(
    by="Importance", ascending=False
)

print("\nFeature Importance:")
print(importance)

# b) Feature Extraction using PCA
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca = PCA(n_components=2)

```

```

X_pca = pca.fit_transform(X_scaled)

pca_df = pd.DataFrame(
    X_pca,
    columns=["Principal_Component_1", "Principal_Component_2"]
)

print("\nPCA Reduced Data:")
print(pca_df)

print("\nExplained Variance Ratio:")
print(pca.explained_variance_ratio_)

plt.figure(figsize=(8, 5))
plt.scatter(
    pca_df["Principal_Component_1"],
    pca_df["Principal_Component_2"],
    c=y,
    s=100
)

plt.title("PCA Feature Extraction")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.grid(True)
plt.show()

```

Output

Loan Application Dataset:

	Application_ID	Age	Income	Credit_Score	Loan_Amount	Employment_Years
0	101	25	30000	650	200000	2
1	102	32	45000	720	300000	5
2	103	45	70000	780	500000	15

3	104	29	35000	680	250000	3
4	105	51	85000	800	400000	20
5	106	38	60000	740	350000	10
6	107	27	32000	620	180000	1
7	108	42	75000	770	450000	12
8	109	35	50000	700	280000	6
9	110	48	90000	810	500000	18

	Debt_Ratio	Loan_Status
0	0.45	Rejected
1	0.35	Approved
2	0.25	Approved
3	0.50	Rejected
4	0.20	Approved
5	0.30	Approved
6	0.55	Rejected
7	0.22	Approved
8	0.40	Rejected
9	0.18	Approved

Feature Importance:

	Feature	Importance
3	Loan_Amount	0.219697
5	Debt_Ratio	0.207872
2	Credit_Score	0.191639
4	Employment_Years	0.148429
1	Income	0.135802
0	Age	0.096561

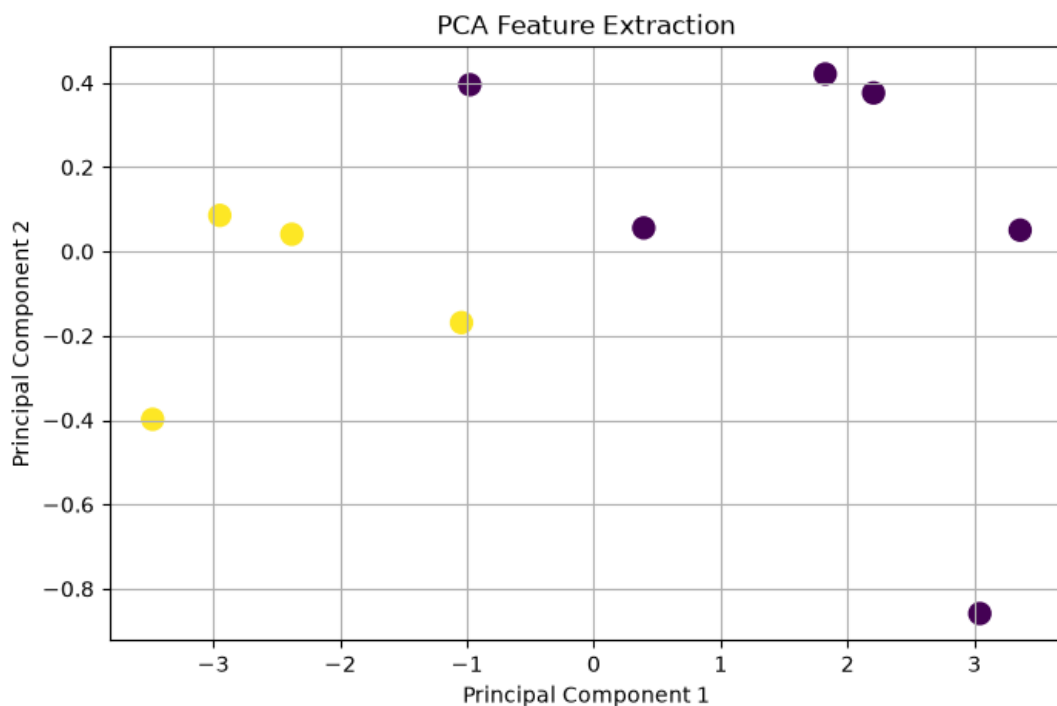
PCA Reduced Data:

	Principal_Component_1	Principal_Component_2
0	-2.948658	0.085608
1	-0.977457	0.395651
2	2.205213	0.376066

3	-2.380545	0.040875
4	3.042863	-0.858867
5	0.393903	0.055884
6	-3.477663	-0.398089
7	1.824400	0.421051
8	-1.043008	-0.168752
9	3.360952	0.050572

Explained Variance Ratio:

[0.95667644 0.02357557]



OR

Q2. Apply Simple Linear Regression on the Salary dataset to predict Salary using Years of Experience. Split the data into training and testing sets, plot the regression line, and evaluate the model using MSE.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
```



```

data = {
    "YearsExperience": [1.1, 1.3, 1.5, 2.0, 2.2, 2.9, 3.0, 3.2, 3.7,
3.9,
                        4.0, 4.5, 4.9, 5.1, 5.3, 5.9, 6.0, 6.8, 7.1,
7.9],
    "Salary": [39343, 46205, 37731, 43525, 39891, 56642, 60150,
54445, 57189, 63218,
                56957, 61111, 67938, 66029, 83088, 81363, 93940,
91738, 98273, 101302]
}

df = pd.DataFrame(data)

print("Salary Dataset:")
print(df)

X = df[["YearsExperience"]]
y = df["Salary"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)

print("\nMean Squared Error:", mse)

plt.figure(figsize=(8, 5))

```

```
plt.scatter(X, y, label="Actual Data")

plt.plot(X, model.predict(X), label="Regression Line")


plt.title("Years of Experience vs Salary")
plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.legend()
plt.grid(True)


plt.show()
```

Output

Salary Dataset:

	YearsExperience	Salary
0	1.1	39343
1	1.3	46205
2	1.5	37731
3	2.0	43525
4	2.2	39891
5	2.9	56642
6	3.0	60150
7	3.2	54445
8	3.7	57189
9	3.9	63218
10	4.0	56957
11	4.5	61111
12	4.9	67938
13	5.1	66029
14	5.3	83088
15	5.9	81363
16	6.0	93940
17	6.8	91738
18	7.1	98273
19	7.9	101302

Mean Squared Error: 45077965.88870029



Slip 9

Q1. Write a Python program to create a product DataFrame with the attributes Product_ID, Price, Quantity_Sold, Discount, and Revenue. Apply Min-Max Scaling data transformation techniques on the DataFrame and display the transformed data.

```
import pandas as pd

from sklearn.preprocessing import MinMaxScaler

data = {
    "Product_ID": [101, 102, 103, 104, 105],
    "Price": [500, 800, 1200, 1500, 2000],
    "Quantity_Sold": [20, 35, 50, 65, 80],
    "Discount": [5, 10, 15, 20, 25],
    "Revenue": [9500, 25200, 51000, 78000, 120000]
}

df = pd.DataFrame(data)

print("Original Product DataFrame:")
print(df)

scaler = MinMaxScaler()

df[["Price", "Quantity_Sold", "Discount", "Revenue"]] =
    scaler.fit_transform(
        df[["Price", "Quantity_Sold", "Discount", "Revenue"]]
    )

print("\nDataFrame After Min-Max Scaling:")
print(df)
```

Output

Original Product DataFrame:

Product_ID	Price	Quantity_Sold	Discount	Revenue
------------	-------	---------------	----------	---------

0	101	500	20	5	9500
1	102	800	35	10	25200
2	103	1200	50	15	51000
3	104	1500	65	20	78000
4	105	2000	80	25	120000

DataFrame After Min-Max Scaling:

	Product_ID	Price	Quantity_Sold	Discount	Revenue
0	101	0.000000	0.00	0.00	0.000000
1	102	0.200000	0.25	0.25	0.142081
2	103	0.466667	0.50	0.50	0.375566
3	104	0.666667	0.75	0.75	0.619910
4	105	1.000000	1.00	1.00	1.000000

Q2. Apply Logistic Regression to classify smartphones into different price categories using specifications such as RAM, storage, battery capacity, and camera resolution. Assess the model using Accuracy, F1-Score, and Confusion Matrix.

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, f1_score,
confusion_matrix

from sklearn.preprocessing import StandardScaler

data = {
    "RAM": [2, 3, 4, 4, 6, 6, 8, 8, 12, 12, 16, 16],
    "Storage": [32, 32, 64, 128, 64, 128, 128, 256, 256, 512, 512,
1024],
    "Battery": [3000, 3500, 4000, 4500, 4000, 5000, 4500, 5000,
5000, 5500, 5000, 6000],
    "Camera": [12, 13, 16, 20, 24, 32, 48, 50, 64, 108, 108, 200],
    "Price_Category": [
        "Budget", "Budget", "Budget", "Budget",
        "Mid-Range", "Mid-Range", "Mid-Range", "Mid-Range",
```

```

        "Premium", "Premium", "Premium", "Premium"
    ]
}

df = pd.DataFrame(data)

print("Smartphone Dataset:")
print(df)

X = df[["RAM", "Storage", "Battery", "Camera"]]
y = df["Price_Category"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred, average="weighted")
cm = confusion_matrix(y_test, y_pred)

print("\nAccuracy:", accuracy)
print("F1-Score:", f1)

```

```
print("\nConfusion Matrix:")
print(cm)
```

Output

Smartphone Dataset:

	RAM	Storage	Battery	Camera	Price_Category
0	2	32	3000	12	Budget
1	3	32	3500	13	Budget
2	4	64	4000	16	Budget
3	4	128	4500	20	Budget
4	6	64	4000	24	Mid-Range
5	6	128	5000	32	Mid-Range
6	8	128	4500	48	Mid-Range
7	8	256	5000	50	Mid-Range
8	12	256	5000	64	Premium
9	12	512	5500	108	Premium
10	16	512	5000	108	Premium
11	16	1024	6000	200	Premium

Accuracy: 1.0

F1-Score: 1.0

Confusion Matrix:

```
[[1 0 0]
 [0 1 0]
 [0 0 1]]
```

OR

Q2. Create a dataset containing information about different books with the attributes Book Title, Author, Publication Year, Average Rating, and Number of Ratings. Perform the following tasks on created dataset: a. Create a Venn Diagram comparing the authors of books published in two different time periods. b. Generate a Word Cloud using the book titles.

```
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib_venn import venn2
```

```
from wordcloud import WordCloud

data = {
    "Book Title": [
        "The Alchemist",
        "Harry Potter",
        "The Kite Runner",
        "The Book Thief",
        "The Fault in Our Stars",
        "The Midnight Library",
        "The Silent Patient",
        "The Seven Husbands of Evelyn Hugo",
        "Atomic Habits",
        "The Psychology of Money"
    ],
    "Author": [
        "Paulo Coelho",
        "J.K. Rowling",
        "Khaled Hosseini",
        "Markus Zusak",
        "John Green",
        "Matt Haig",
        "Alex Michaelides",
        "Taylor Jenkins Reid",
        "James Clear",
        "Morgan Housel"
    ],
    "Publication Year": [
        1988, 1997, 2003, 2005, 2012,
        2020, 2019, 2017, 2018, 2020
    ],
    "Average Rating": [
```



```

        4.4, 4.6, 4.5, 4.4, 4.1,
        4.3, 4.2, 4.5, 4.3, 4.4
    ],
    "Number of Ratings": [
        2500000, 9500000, 3000000, 1800000, 5000000,
        1500000, 2200000, 1200000, 1000000, 900000
    ]
}

df = pd.DataFrame(data)

print("Book Dataset:")
print(df)

period1 = set(df[df["Publication Year"] < 2010]["Author"])
period2 = set(df[df["Publication Year"] >= 2010]["Author"])

plt.figure(figsize=(8, 6))

venn2(
    [period1, period2],
    set_labels=("Before 2010", "2010 and Later")
)

plt.title("Comparison of Authors by Publication Period")
plt.show()

titles = " ".join(df["Book Title"])

wordcloud = WordCloud(
    width=1000,
    height=500,

```

```

        background_color="white"
    ).generate(titles)

plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation="bilinear")
plt.axis("off")
plt.title("Word Cloud of Book Titles")
plt.show()

```

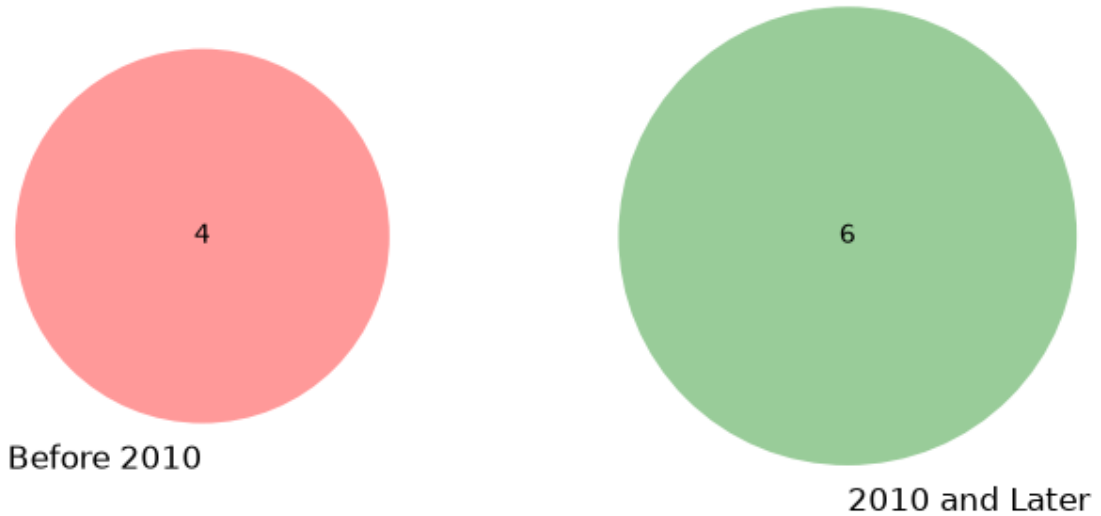
Output

Book Dataset:

	Book Title	Author	Publication Year
0	The Alchemist	Paulo Coelho	1988
1	Harry Potter	J.K. Rowling	1997
2	The Kite Runner	Khaled Hosseini	2003
3	The Book Thief	Markus Zusak	2005
4	The Fault in Our Stars	John Green	2012
5	The Midnight Library	Matt Haig	2020
6	The Silent Patient	Alex Michaelides	2019
7	The Seven Husbands of Evelyn Hugo	Taylor Jenkins Reid	2017
8	Atomic Habits	James Clear	2018
9	The Psychology of Money	Morgan Housel	2020

	Average Rating	Number of Ratings
0	4.4	2500000
1	4.6	9500000
2	4.5	3000000
3	4.4	1800000
4	4.1	5000000
5	4.3	1500000
6	4.2	2200000
7	4.5	1200000
8	4.3	1000000
9	4.4	900000

Comparison of Authors by Publication Period



Slip 10

Q1. A company conducted a survey to determine the preferred mode of transportation among employees as shown below: transport = ["Train", "Bicycle", "Bike", "Car", "Bus"], employees = [220, 60, 70, 80, 40] Create a Pie Chart to represent the percentage of employees using each mode of transportation. Display the percentage contribution of each category. Give appropriate labels and colors.

```
import matplotlib.pyplot as plt

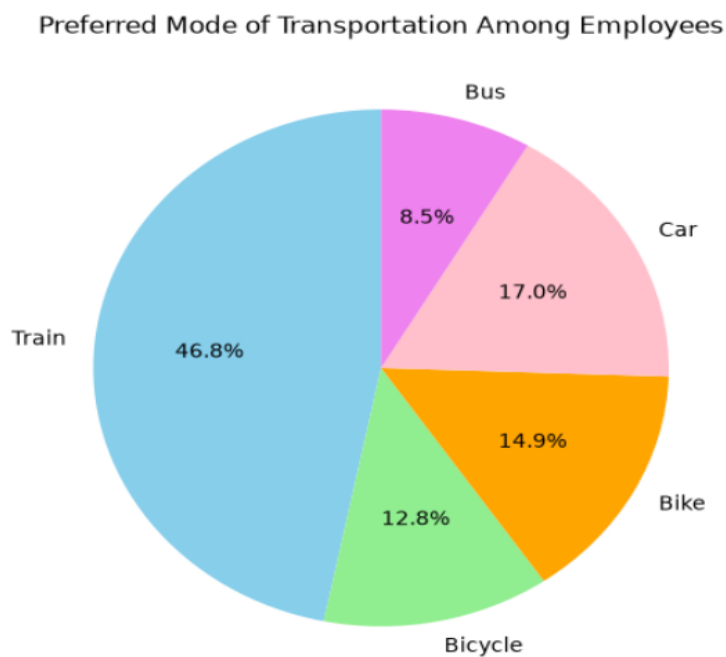
transport = ["Train", "Bicycle", "Bike", "Car", "Bus"]
employees = [220, 60, 70, 80, 40]

plt.figure(figsize=(8, 6))

plt.pie(
    employees,
    labels=transport,
    autopct="%1.1f%%",
    colors=["skyblue", "lightgreen", "orange", "pink", "violet"],
    startangle=90
)

plt.title("Preferred Mode of Transportation Among Employees")
plt.show()
```

Output



Q2. Using the given housing dataset, develop a Random Forest Regression model to predict house prices based on the available property attributes. Perform the necessary data preprocessing, train and test the model, and evaluate its predictive performance using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R² Score.

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestRegressor

from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score

import numpy as np

data = {

    "Area": [1200, 1500, 1800, 1000, 2200, 1600, 2000, 1300, 2500,
1100, 1900, 1700, 1400, 2300, 1250],

    "Bedrooms": [2, 3, 3, 2, 4, 3, 4, 2, 5, 2, 4, 3, 3, 4, 2],

    "Bathrooms": [1, 2, 2, 1, 3, 2, 3, 1, 4, 1, 3, 2, 2, 3, 1],

    "Age": [10, 5, 3, 15, 2, 7, 4, 12, 1, 18, 6, 8, 9, 3, 14],

    "Price": [3500000, 5000000, 6200000, 3000000, 8500000, 5500000,
7800000, 3800000, 10500000, 2800000, 7000000, 5800000, 4500000,
9000000, 3200000]

}

df = pd.DataFrame(data)

print("Housing Dataset:")

print(df)

X = df.drop("Price", axis=1)

y = df["Price"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```

model = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("\nPredicted Prices:")
print(y_pred)

print("\nModel Evaluation:")
print("Mean Absolute Error (MAE):", mae)
print("Root Mean Squared Error (RMSE):", rmse)
print("R2 Score:", r2)

```

Output

Housing Dataset:

	Area	Bedrooms	Bathrooms	Age	Price
0	1200	2	1	10	3500000
1	1500	3	2	5	5000000
2	1800	3	2	3	6200000
3	1000	2	1	15	3000000
4	2200	4	3	2	8500000
5	1600	3	2	7	5500000
6	2000	4	3	4	7800000
7	1300	2	1	12	3800000
8	2500	5	4	1	10500000

9	1100	2	1	18	2800000
10	1900	4	3	6	7000000
11	1700	3	2	8	5800000
12	1400	3	2	9	4500000
13	2300	4	3	3	9000000
14	1250	2	1	14	3200000

Predicted Prices:

[3234000. 5514000. 3717000.]

Model Evaluation:

Mean Absolute Error (MAE): 312333.3333333333

Root Mean Squared Error (RMSE): 325187.63814142754

R² Score: 0.9356073748308524

OR

Q2. Create the following dataset in python. Apply the Apriori algorithm to discover frequently used learning resources and generate association rules among them. Repeat the process with different minimum support values 0.3, and 0.4.

TID	Items
1	Video Lecture, Quiz
2	Video Lecture, Assignment
3	Quiz, Discussion Forum
4	Video Lecture, Quiz, Assignment
5	Assignment, Discussion Forum
6	Video Lecture, Certificate
7	Quiz, Certificate
8	Video Lecture, Quiz
9	Assignment, Certificate
10	Video Lecture, Quiz, Certificate

```

import pandas as pd

from mlxtend.frequent_patterns import apriori, association_rules

data = {
    "TID": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "Items": [
        "Video Lecture, Quiz",
        "Video Lecture, Assignment",
        "Quiz, Discussion Forum",
        "Video Lecture, Quiz, Assignment",
        "Assignment, Discussion Forum",
        "Video Lecture, Certificate",
        "Quiz, Certificate",
        "Video Lecture, Quiz",
        "Assignment",
        "Video Lecture, Quiz, Certificate"
    ]
}

df = pd.DataFrame(data)

print("Original Dataset:")
print(df)

transactions = df["Items"].apply(
    lambda x: [item.strip() for item in x.split(",")]
)

items = sorted(set(item for transaction in transactions for item in
transaction))

basket = pd.DataFrame(
    False,
    index=range(len(transactions)),

```



```

        columns=items
    )

for i, transaction in enumerate(transactions):
    for item in transaction:
        basket.loc[i, item] = True
print("\nTransaction Data:")
print(basket)
for support in [0.3, 0.4]:

    print("\nMinimum Support:", support)

    frequent_itemsets = apriori(
        basket,
        min_support=support,
        use_colnames=True
    )
    print("\nFrequent Itemsets:")
    print(frequent_itemsets)

    if len(frequent_itemsets) > 0:

        rules = association_rules(
            frequent_itemsets,
            metric="confidence",
            min_threshold=0.5
        )

        print("\nAssociation Rules:")

        if len(rules) > 0:
            print(rules[

```

```

        ["antecedents", "consequents",
        "support", "confidence", "lift"]
    })
else:
    print("No association rules generated.")

```

Output

Original Dataset:

	TID	Items
0	1	Video Lecture, Quiz
1	2	Video Lecture, Assignment
2	3	Quiz, Discussion Forum
3	4	Video Lecture, Quiz, Assignment
4	5	Assignment, Discussion Forum
5	6	Video Lecture, Certificate
6	7	Quiz, Certificate
7	8	Video Lecture, Quiz
8	9	Assignment
9	10	Video Lecture, Quiz, Certificate

Transaction Data:

	Assignment	Certificate	Discussion Forum	Quiz	Video Lecture
0	False	False	False	True	True
1	True	False	False	False	True
2	False	False	True	True	False
3	True	False	False	True	True
4	True	False	True	False	False
5	False	True	False	False	True
6	False	True	False	True	False
7	False	False	False	True	True
8	True	False	False	False	False
9	False	True	False	True	True

Minimum Support: 0.3

Frequent Itemsets:

	support	itemsets
0	0.4	frozenset({Assignment})
1	0.3	frozenset({Certificate})
2	0.6	frozenset({Quiz})
3	0.6	frozenset({Video Lecture})
4	0.4	frozenset({Quiz, Video Lecture})

Association Rules:

	antecedents	consequents	support \
0	frozenset({Quiz})	frozenset({Video Lecture})	0.4
1	frozenset({Video Lecture})	frozenset({Quiz})	0.4

	confidence	lift
0	0.666667	1.111111
1	0.666667	1.111111

Minimum Support: 0.4

Frequent Itemsets:

	support	itemsets
0	0.4	frozenset({Assignment})
1	0.6	frozenset({Quiz})
2	0.6	frozenset({Video Lecture})
3	0.4	frozenset({Quiz, Video Lecture})

Association Rules:

	antecedents	consequents	support \
0	frozenset({Quiz})	frozenset({Video Lecture})	0.4
1	frozenset({Video Lecture})	frozenset({Quiz})	0.4

	confidence	lift
0	0.666667	1.111111
1	0.666667	1.111111

Slip 11

Q1. An organization has recorded the number of employees working in different departments such as HR, Finance, IT, Sales, and Marketing. Write a Python program to create a bar chart showing the distribution of employees among these departments. Display suitable labels and a title.

```
import matplotlib.pyplot as plt

departments = ["HR", "Finance", "IT", "Sales", "Marketing"]
employees = [25, 40, 60, 35, 30]

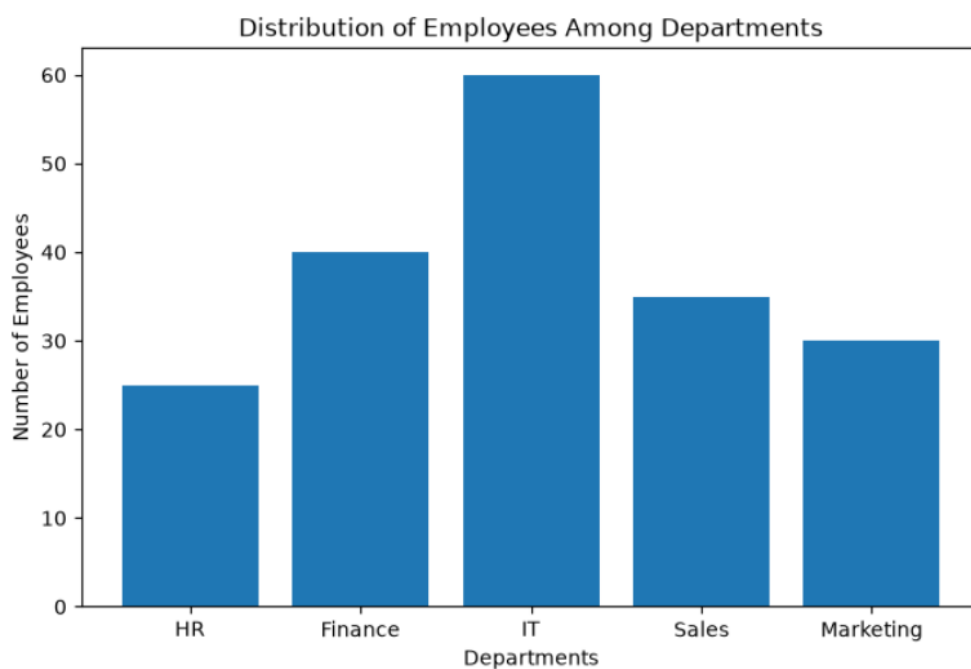
plt.figure(figsize=(8, 5))

plt.bar(departments, employees)

plt.title("Distribution of Employees Among Departments")
plt.xlabel("Departments")
plt.ylabel("Number of Employees")

plt.show()
```

Output



Q2. Apply Logistic Regression to classify smartphones into different price categories using the given smartphone dataset. Preprocess the data appropriately and assess the model using Accuracy, F1Score, and Confusion Matrix.

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score,
confusion_matrix

data = {
    "RAM": [2, 3, 4, 4, 6, 6, 8, 8, 12, 12, 16, 16],
    "Storage": [32, 32, 64, 128, 64, 128, 128, 256, 256, 512, 512,
1024],
    "Battery": [3000, 3500, 4000, 4500, 4000, 5000, 4500, 5000,
5000, 5500, 5000, 6000],
    "Camera": [12, 13, 16, 20, 24, 32, 48, 50, 64, 108, 108, 200],
    "Price_Category": [
        "Budget", "Budget", "Budget", "Budget",
        "Mid-Range", "Mid-Range", "Mid-Range", "Mid-Range",
        "Premium", "Premium", "Premium", "Premium"
    ]
}

df = pd.DataFrame(data)

print("Smartphone Dataset:")
print(df)

X = df[["RAM", "Storage", "Battery", "Camera"]]
y = df["Price_Category"]

X_train, X_test, y_train, y_test = train_test_split(
```

```

X, y, test_size=0.25, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred, average="weighted")
cm = confusion_matrix(y_test, y_pred)

print("\nAccuracy:", accuracy)
print("F1-Score:", f1)

print("\nConfusion Matrix:")
print(cm)

```

Output

Smartphone Dataset:

	RAM	Storage	Battery	Camera	Price_Category
0	2	32	3000	12	Budget
1	3	32	3500	13	Budget
2	4	64	4000	16	Budget
3	4	128	4500	20	Budget
4	6	64	4000	24	Mid-Range
5	6	128	5000	32	Mid-Range
6	8	128	4500	48	Mid-Range
7	8	256	5000	50	Mid-Range
8	12	256	5000	64	Premium

9	12	512	5500	108	Premium
10	16	512	5000	108	Premium
11	16	1024	6000	200	Premium

Accuracy: 1.0

F1-Score: 1.0

Confusion Matrix:

```
[[1 0 0]
 [0 1 0]
 [0 0 1]]
```

OR

Q2. Apply the K-Means Clustering algorithm on the given E-commerce Customer dataset. Read the dataset using Python, inspect and preprocess the data by handling missing values and selecting suitable numerical features. Apply K-Means clustering to segment the customers into different categories and display the clusters.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

data = {
    "CustomerID":
    [101,102,103,104,105,106,107,108,109,110,111,112,113,114,115] ,
    "Avg. Session Length":
    [34.5,31.2,38.5,35.2,40.1,28.5,33.8,42.0,30.5,37.2,39.5,29.8,35.8,41.2,32.5] ,
    "Time on App":
    [12.5,11.2,14.5,13.2,15.5,9.5,12.0,16.2,10.5,14.0,15.0,10.2,13.5,16.0,11.8] ,
    "Time on Website":
    [39.6,37.2,40.5,38.8,42.1,35.5,36.8,43.0,34.5,41.2,42.5,36.0,39.5,44.0,37.5] ,
```

```
    "Length of Membership":  
    [4.5,3.2,5.1,4.0,6.2,2.1,3.8,6.8,2.8,5.0,5.8,2.5,4.2,7.0,3.5]  
}
```

```
df = pd.DataFrame(data)
```

```
df.loc[3, "Time on App"] = None  
df.loc[7, "Time on Website"] = None
```

```
print("Original Dataset:")  
print(df)
```

```
print("\nMissing Values:")  
print(df.isnull().sum())
```

```
features = [  
    "Avg. Session Length",  
    "Time on App",  
    "Time on Website",  
    "Length of Membership"  
]
```

```
df[features] = df[features].fillna(df[features].median())
```

```
print("\nDataset After Handling Missing Values:")  
print(df)
```

```
X = df[features]
```

```
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X)
```

```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
```



```

df["Cluster"] = kmeans.fit_predict(X_scaled)

print("\nCustomer Clusters:")
print(df[["CustomerID"] + features + ["Cluster"]])

plt.figure(figsize=(8, 5))
plt.scatter(
    df["Time on App"],
    df["Length of Membership"],
    c=df["Cluster"],
    s=80
)

plt.title("E-commerce Customer Segmentation using K-Means")
plt.xlabel("Time on App")
plt.ylabel("Length of Membership")
plt.grid(True)
plt.show()

```

Output

Original Dataset:

	CustomerID	Avg. Session Length	Time on App	Time on Website	\
0	101	34.5	12.5	39.6	
1	102	31.2	11.2	37.2	
2	103	38.5	14.5	40.5	
3	104	35.2	NaN	38.8	
4	105	40.1	15.5	42.1	
5	106	28.5	9.5	35.5	
6	107	33.8	12.0	36.8	
7	108	42.0	16.2	NaN	
8	109	30.5	10.5	34.5	
9	110	37.2	14.0	41.2	
10	111	39.5	15.0	42.5	
11	112	29.8	10.2	36.0	

12	113	35.8	13.5	39.5
13	114	41.2	16.0	44.0
14	115	32.5	11.8	37.5

Length of Membership

0	4.5
1	3.2
2	5.1
3	4.0
4	6.2
5	2.1
6	3.8
7	6.8
8	2.8
9	5.0
10	5.8
11	2.5
12	4.2
13	7.0
14	3.5

Missing Values:

CustomerID	0
Avg. Session Length	0
Time on App	1
Time on Website	1
Length of Membership	0

dtype: int64

Dataset After Handling Missing Values:

	CustomerID	Avg. Session Length	Time on App	Time on Website	\
0	101	34.5	12.5	39.60	
1	102	31.2	11.2	37.20	
2	103	38.5	14.5	40.50	

3	104	35.2	13.0	38.80
4	105	40.1	15.5	42.10
5	106	28.5	9.5	35.50
6	107	33.8	12.0	36.80
7	108	42.0	16.2	39.15
8	109	30.5	10.5	34.50
9	110	37.2	14.0	41.20
10	111	39.5	15.0	42.50
11	112	29.8	10.2	36.00
12	113	35.8	13.5	39.50
13	114	41.2	16.0	44.00
14	115	32.5	11.8	37.50

Length of Membership

0	4.5
1	3.2
2	5.1
3	4.0
4	6.2
5	2.1
6	3.8
7	6.8
8	2.8
9	5.0
10	5.8
11	2.5
12	4.2
13	7.0
14	3.5

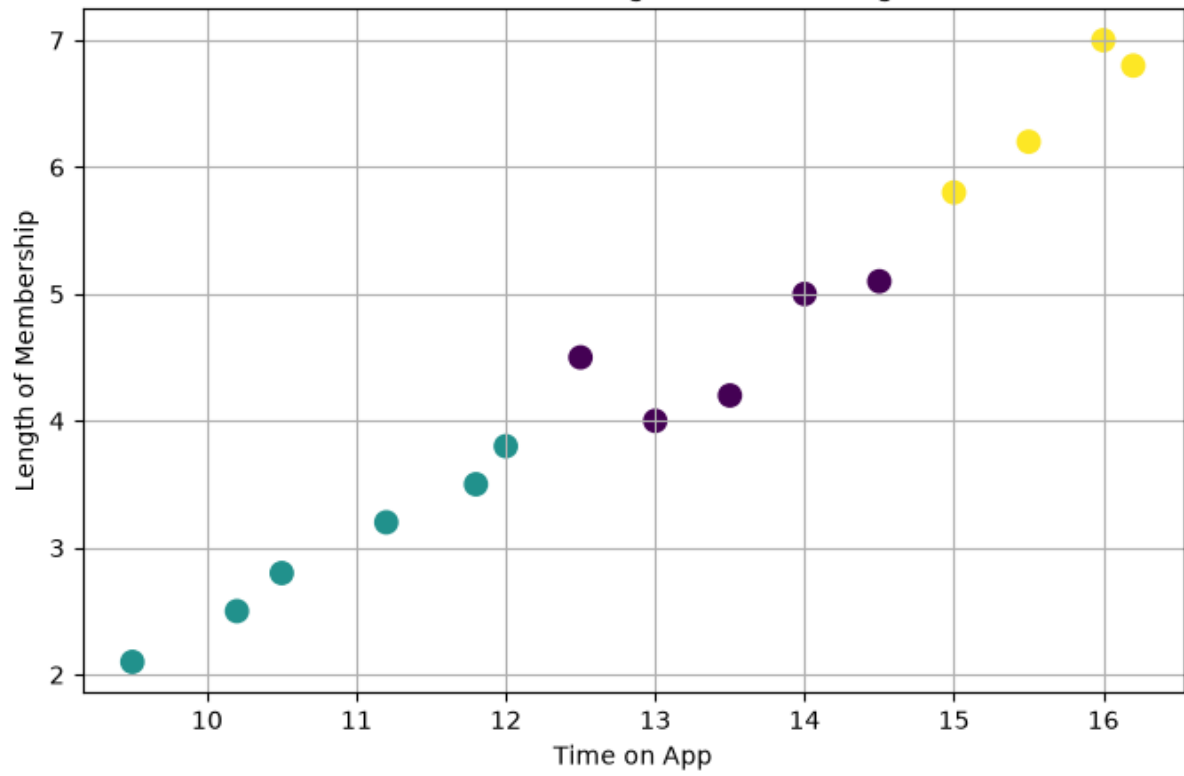
Customer Clusters:

	CustomerID	Avg. Session Length	Time on App	Time on Website \
0	101	34.5	12.5	39.60
1	102	31.2	11.2	37.20

2	103	38.5	14.5	40.50
3	104	35.2	13.0	38.80
4	105	40.1	15.5	42.10
5	106	28.5	9.5	35.50
6	107	33.8	12.0	36.80
7	108	42.0	16.2	39.15
8	109	30.5	10.5	34.50
9	110	37.2	14.0	41.20
10	111	39.5	15.0	42.50
11	112	29.8	10.2	36.00
12	113	35.8	13.5	39.50
13	114	41.2	16.0	44.00
14	115	32.5	11.8	37.50

	Length of Membership	Cluster
0	4.5	0
1	3.2	1
2	5.1	0
3	4.0	0
4	6.2	2
5	2.1	1
6	3.8	1
7	6.8	2
8	2.8	1
9	5.0	0
10	5.8	2
11	2.5	1
12	4.2	0
13	7.0	2
14	3.5	1

E-commerce Customer Segmentation using K-Means



Slip 12

Q1. Load the Titanic dataset (titanic.csv) Create a histogram showing the number of passengers in each passenger class. Create a Pie Chart showing the survival distribution.

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "PassengerClass": [1, 1, 1, 2, 2, 2, 3, 3, 3, 3, 3, 3],
    "Survived": [1, 1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0]
}

df = pd.DataFrame(data)

df.to_csv("titanic.csv", index=False)

df = pd.read_csv("titanic.csv")

print("Titanic Dataset:")
print(df)

plt.hist(df["PassengerClass"], bins=[0.5, 1.5, 2.5, 3.5],
         rwidth=0.8)

plt.xlabel("Passenger Class")
plt.ylabel("Number of Passengers")
plt.title("Number of Passengers in Each Passenger Class")
plt.xticks([1, 2, 3])
plt.show()

survival = df["Survived"].value_counts()

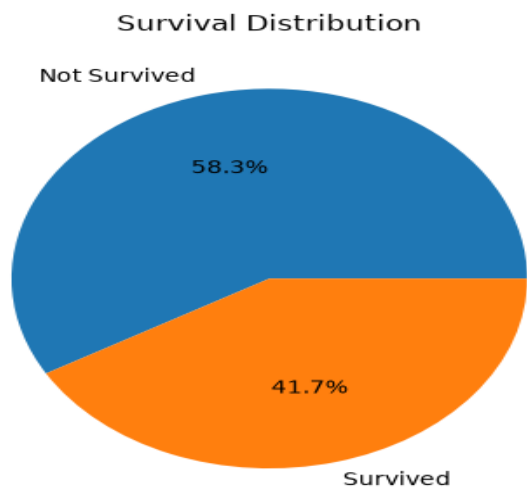
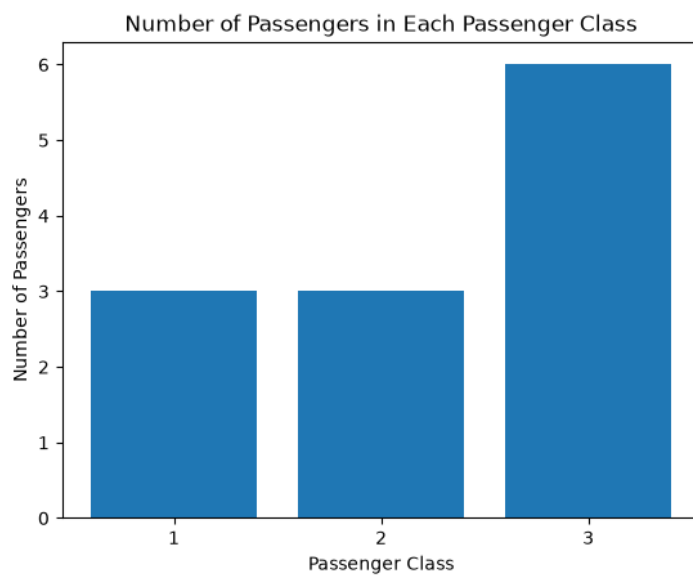
plt.pie(survival, labels=["Not Survived", "Survived"],
        autopct="%1.1f%%")
```

```
plt.title("Survival Distribution")  
plt.show()
```

Output

Titanic Dataset:

	PassengerClass	Survived
0	1	1
1	1	1
2	1	0
3	2	1
4	2	0
5	2	0
6	3	1
7	3	0
8	3	0
9	3	1
10	3	0
11	3	0



Q2. Apply Polynomial Regression to predict song popularity using audio features such as danceability, energy, and tempo. Evaluate the model using MAE, RMSE, and R² Score, and visualize the fitted curve.

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score

import numpy as np

data = {
    "danceability": [0.45, 0.50, 0.55, 0.60, 0.65, 0.70, 0.75, 0.80,
0.85, 0.90],
    "energy": [0.40, 0.45, 0.50, 0.55, 0.60, 0.65, 0.70, 0.75, 0.80,
0.85],
    "tempo": [90, 95, 100, 105, 110, 115, 120, 125, 130, 135],
    "popularity": [35, 40, 45, 52, 60, 68, 75, 82, 88, 92]
}

df = pd.DataFrame(data)
df.to_csv("songs.csv", index=False)

df = pd.read_csv("songs.csv")

print("Song Dataset:")
print(df)

X = df[["danceability", "energy", "tempo"]]
y = df["popularity"]

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```



```

poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

model = LinearRegression()
model.fit(X_train_poly, y_train)

y_pred = model.predict(X_test_poly)

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("\nMAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)

plt.scatter(df["danceability"], df["popularity"], label="Actual
Data")

X_curve = np.linspace(df["danceability"].min(),
df["danceability"].max(), 100)
curve_data = pd.DataFrame({
    "danceability": X_curve,
    "energy": df["energy"].mean(),
    "tempo": df["tempo"].mean()
})

curve_poly = poly.transform(curve_data)
y_curve = model.predict(curve_poly)

plt.plot(X_curve, y_curve, label="Polynomial Fitted Curve")

```

```
plt.xlabel("Danceability")
plt.ylabel("Popularity")
plt.title("Polynomial Regression: Song Popularity")
plt.legend()
plt.show()
```

Output

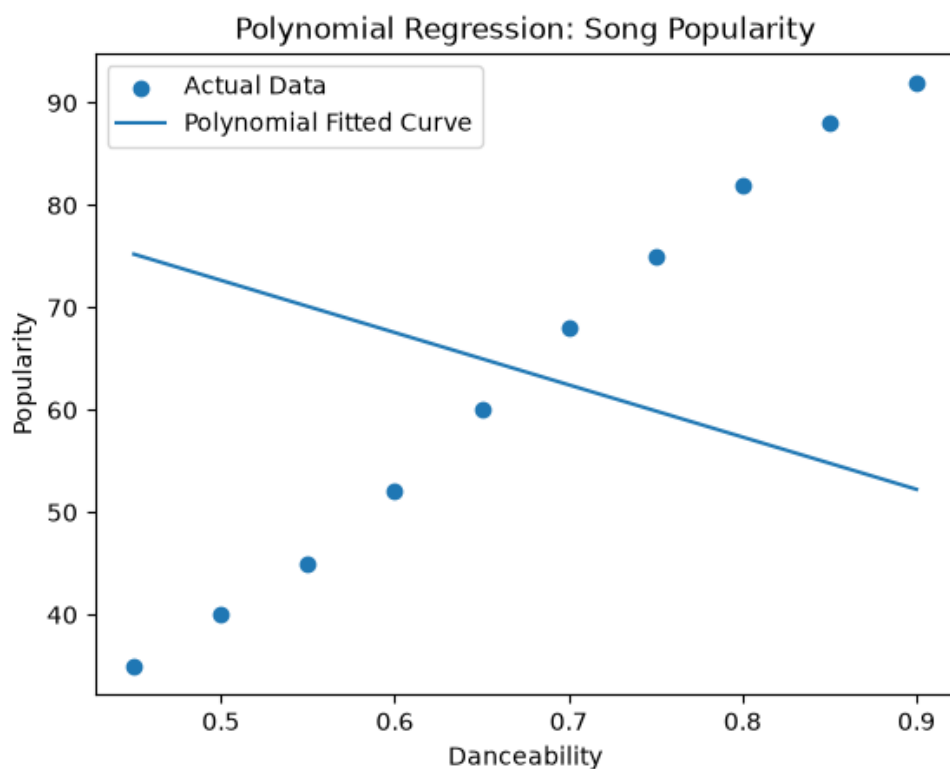
Song Dataset:

	danceability	energy	tempo	popularity
0	0.45	0.40	90	35
1	0.50	0.45	95	40
2	0.55	0.50	100	45
3	0.60	0.55	105	52
4	0.65	0.60	110	60
5	0.70	0.65	115	68
6	0.75	0.70	120	75
7	0.80	0.75	125	82
8	0.85	0.80	130	88
9	0.90	0.85	135	92

MAE: 0.6163793103448114

RMSE: 0.7612380602331128

R² Score: 0.99899395245773



OR

Q2. Use the K-Nearest Neighbors (KNN) algorithm to classify emails as spam or non-spam using the Spambase dataset. Preprocess the data, train the KNN classifier, and evaluate its performance using accuracy, confusion matrix, precision, recall, and F1-score.

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, confusion_matrix,
precision_score, recall_score, f1_score, ConfusionMatrixDisplay

data = {

    "word_freq_make": [0.21,0.06,0.00,0.00,0.10,0.02,0.00,0.00,0.05,0.15,
0.01,0.00,0.20,0.04,0.00,0.30,0.02,0.01,0.00,0.25],

    "word_freq_address": [0.28,0.00,0.00,0.00,0.15,0.00,0.10,0.00,0.02,0.
18,0.00,0.05,0.12,0.00,0.03,0.20,0.00,0.02,0.01,0.16],

    "word_freq_free": [0.25,0.00,0.10,0.00,0.30,0.05,0.20,0.00,0.15,0.35,
0.02,0.00,0.28,0.04,0.00,0.32,0.01,0.03,0.00,0.27],

    "word_freq_money": [0.00,0.00,0.15,0.00,0.20,0.00,0.18,0.00,0.10,0.25
,0.00,0.02,0.15,0.00,0.05,0.30,0.00,0.01,0.00,0.22],

    "char_freq_bang": [0.50,0.05,0.10,0.02,0.70,0.08,0.40,0.03,0.60,0.80,
0.05,0.02,0.65,0.06,0.04,0.75,0.03,0.07,0.02,0.68],

    "capital_run_length_average": [5,3,4,2,8,3,7,2,9,10,3,2,6,3,2,9,3,4,2
,8],

    "capital_run_length_longest": [20,10,15,8,30,12,25,7,35,40,9,6,28,11,
8,38,10,14,7,32],

    "capital_run_length_total": [100,60,80,40,150,70,120,35,160,200,55,30
,130,65,45,180,50,75,35,145],

    "spam": [1,0,1,0,1,0,1,0,1,1,0,0,1,0,0,1,0,0,0,1]

}
```

```

df = pd.DataFrame(data)
df.to_csv("spambase.csv", index=False)
df = pd.read_csv("spambase.csv")
print("Spambase Dataset:")
print(df.head())

X = df.drop("spam", axis=1)
y = df["spam"]

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, zero_division=0)
recall = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)
cm = confusion_matrix(y_test, y_pred)

print("\nAccuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1-Score:", f1)
print("\nConfusion Matrix:")
print(cm)

disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=["Non-Spam", "Spam"])

disp.plot()

plt.title("KNN Confusion Matrix - Spam Classification")
plt.show()

```

Output

Spambase Dataset:

	word_freq_make	word_freq_address	word_freq_free	word_freq_money	\
0	0.21	0.28	0.25	0.00	
1	0.06	0.00	0.00	0.00	
2	0.00	0.00	0.10	0.15	
3	0.00	0.00	0.00	0.00	
4	0.10	0.15	0.30	0.20	

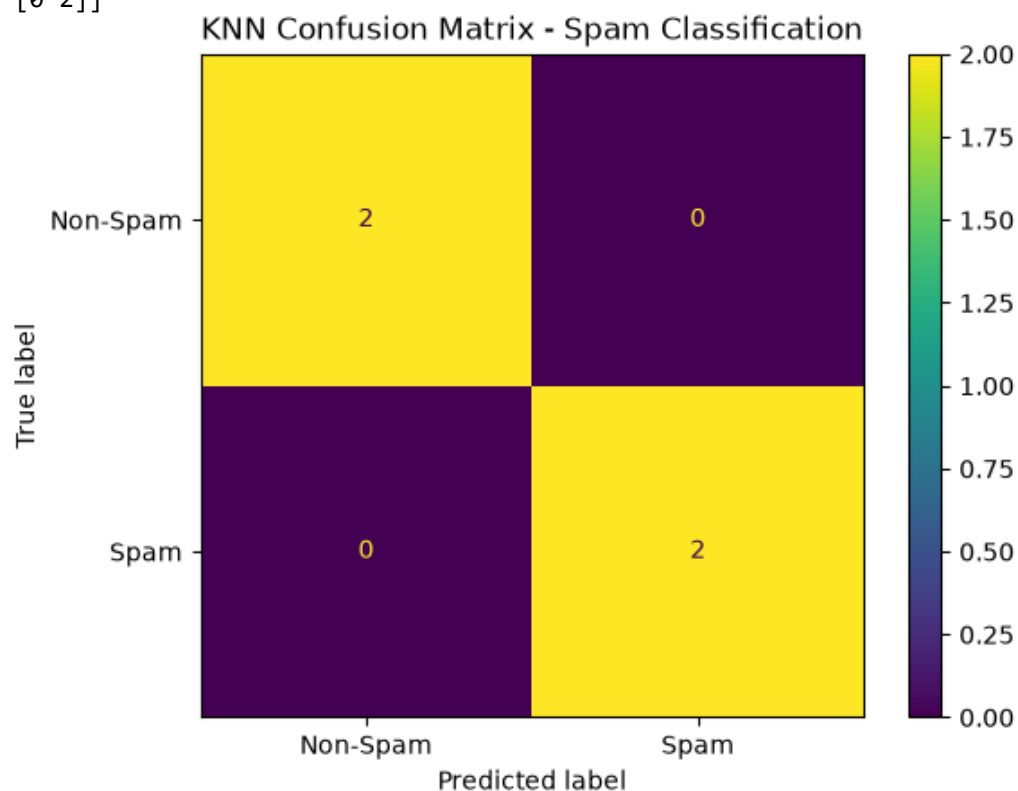
	char_freq_bang	capital_run_length_average	capital_run_length_longest	\
0	0.50	5	20	
1	0.05	3	10	
2	0.10	4	15	
3	0.02	2	8	
4	0.70	8	30	

	capital_run_length_total	spam
0	100	1
1	60	0
2	80	1
3	40	0
4	150	1

Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1-Score: 1.0

Confusion Matrix:

```
[[2 0]
 [0 2]]
```



Slip 13

Q1. A retail store has recorded the daily sales for two months. Create a dataframe using sales.csv file and generate a Box Plot to identify any outliers. Customize the graph with title and labels.

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Day": list(range(1, 61)),
    "Sales": [1200, 1350, 1100, 1450, 1500, 1250, 1300, 1400, 1550,
1600, 1180, 1320, 1270, 1480, 1520, 1380, 1420, 1290, 1350, 1500,
1450, 1550, 1620, 1280, 1340, 1410, 1370, 1490, 1530, 1300, 1250,
1400, 1580, 1650, 1350, 1420, 1500, 1550, 1700, 1320, 1380, 1450,
1510, 1600, 1750, 1300, 1350, 1420, 1480, 1550, 1800, 1250, 1380,
1450, 1520, 1600, 1900, 1300, 1400, 5000]
}

df = pd.DataFrame(data)
df.to_csv("sales.csv", index=False)
df = pd.read_csv("sales.csv")

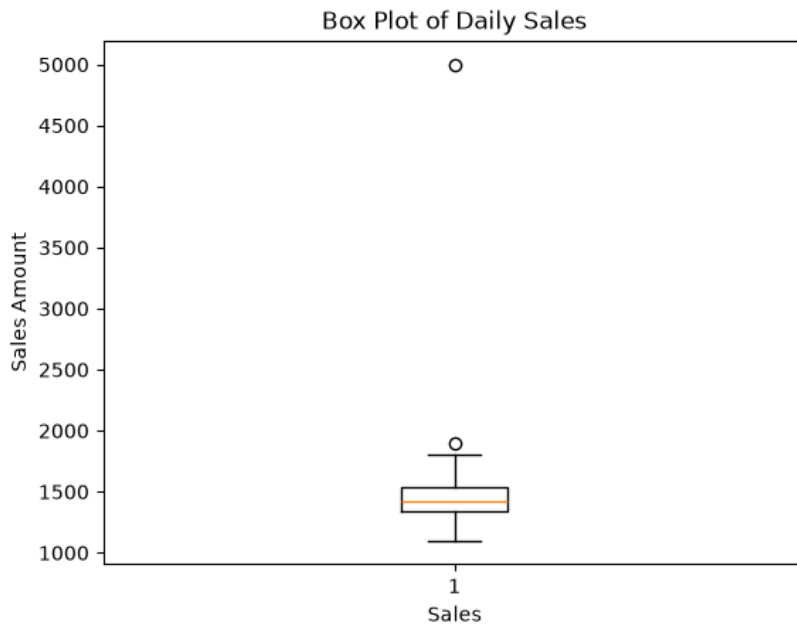
print("Sales Dataset:")
print(df)

plt.boxplot(df["Sales"])
plt.title("Box Plot of Daily Sales")
plt.xlabel("Sales")
plt.ylabel("Sales Amount")
plt.show()
```

Output

```
Sales Dataset:
   Day  Sales
0     1   1200
1     2   1350
2     3   1100
3     4   1450
4     5   1500
```

5	6	1250
6	7	1300
7	8	1400
8	9	1550
9	10	1600
10	11	1180
11	12	1320
12	13	1270
13	14	1480
14	15	1520
15	16	1380
16	17	1420
17	18	1290
18	19	1350
19	20	1500
20	21	1450
21	22	1550
22	23	1620
23	24	1280
24	25	1340
25	26	1410
26	27	1370
27	28	1490
28	29	1530
29	30	1300
30	31	1250
31	32	1400
32	33	1580
33	34	1650
34	35	1350
35	36	1420
36	37	1500
37	38	1550
38	39	1700
39	40	1320
40	41	1380
41	42	1450
42	43	1510
43	44	1600
44	45	1750
45	46	1300
46	47	1350
47	48	1420
48	49	1480
49	50	1550
50	51	1800
51	52	1250
52	53	1380
53	54	1450
54	55	1520
55	56	1600
56	57	1900
57	58	1300
58	59	1400
59	60	5000



Q2. Implement the K-Nearest Neighbors (KNN) algorithm to classify the Wine dataset. Perform the required data preprocessing, model training, and testing, and evaluate the model performance using Accuracy, Precision, Recall, and F1-Score.

```
import pandas as pd

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score

wine = load_wine()

X = pd.DataFrame(wine.data, columns=wine.feature_names)
y = wine.target

print("Wine Dataset:")
print(X.head())

X_train, X_test, y_train, y_test = train_test_split(
```



```

X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average="weighted")
recall = recall_score(y_test, y_pred, average="weighted")
f1 = f1_score(y_test, y_pred, average="weighted")

print("\nModel Performance:")
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1-Score:", f1)

```

Output

Wine Dataset:

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	\
0	14.23	1.71	2.43	15.6	127.0	2.80	
1	13.20	1.78	2.14	11.2	100.0	2.65	
2	13.16	2.36	2.67	18.6	101.0	2.80	
3	14.37	1.95	2.50	16.8	113.0	3.85	
4	13.24	2.59	2.87	21.0	118.0	2.80	

	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hue	\
0	3.06		0.28	2.29	5.64	1.04
1	2.76		0.26	1.28	4.38	1.05
2	3.24		0.30	2.81	5.68	1.03
3	3.49		0.24	2.18	7.80	0.86
4	2.69		0.39	1.82	4.32	1.04

	od280/od315_of_diluted_wines	proline
0	3.92	1065.0

1	3.40	1050.0
2	3.17	1185.0
3	3.45	1480.0
4	2.93	735.0

Model Performance:

Accuracy: 0.9722222222222222

Precision: 0.9747474747474748

Recall: 0.9722222222222222

F1-Score: 0.9723691945914168

OR

Q2. Apply LogisticRegression on an Online Gaming dataset to classify players as Casual or Professional gamers based on features such as daily play time, achievements unlocked, in-game purchases, and gaming level. Evaluate the model using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, confusion_matrix, ConfusionMatrixDisplay

data = {

    "Daily_Play_Time": [1,2,1.5,3,2.5,4,3.5,5,4.5,6,5.5,7,6.5,8,7.5,9,8.5,
,10,9.5,11],

    "Achievements_Unlocked": [5,8,6,15,12,25,20,35,30,45,40,55,50,65,60,7
5,70,85,80,95],

    "In_Game_Purchases": [1,0,1,2,1,3,2,4,3,5,4,6,5,7,6,8,7,9,8,10],

    "Gaming_Level": [2,3,2,4,3,5,4,6,5,7,6,8,7,9,8,10,9,10,10,10],

    "Gamer_Type": [0,0,0,0,0,0,0,0,0,0,0,1,1,1,1,1,1,1,1,1]

}

df = pd.DataFrame(data)

df.to_csv("online_gaming.csv", index=False)

df = pd.read_csv("online_gaming.csv")
```

```

print("Online Gaming Dataset:")

print(df)

X =
df[["Daily_Play_Time", "Achievements_Unlocked", "In_Game_Purchases", "G
aming_Level"]]

y = df["Gamer_Type"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, zero_division=0)
recall = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)
cm = confusion_matrix(y_test, y_pred)

print("\nModel Performance:")
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1-Score:", f1)
print("\nConfusion Matrix:")

```

```

print(cm)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Casual", "Professional"]
)

disp.plot()

plt.title("Logistic Regression - Gamer Classification")

plt.show()

```

Output

Online Gaming Dataset:

	Daily_Play_Time	Achievements_Unlocked	In_Game_Purchases	Gaming_Level	\
0	1.0	5	1	2	
1	2.0	8	0	3	
2	1.5	6	1	2	
3	3.0	15	2	4	
4	2.5	12	1	3	
5	4.0	25	3	5	
6	3.5	20	2	4	
7	5.0	35	4	6	
8	4.5	30	3	5	
9	6.0	45	5	7	
10	5.5	40	4	6	
11	7.0	55	6	8	
12	6.5	50	5	7	
13	8.0	65	7	9	
14	7.5	60	6	8	
15	9.0	75	8	10	
16	8.5	70	7	9	
17	10.0	85	9	10	
18	9.5	80	8	10	
19	11.0	95	10	10	

	Gamer_Type
0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	0
8	0
9	0
10	1
11	1
12	1
13	1
14	1
15	1

16 1
17 1
18 1
19 1

Model Performance:

Accuracy: 1.0

Precision: 1.0

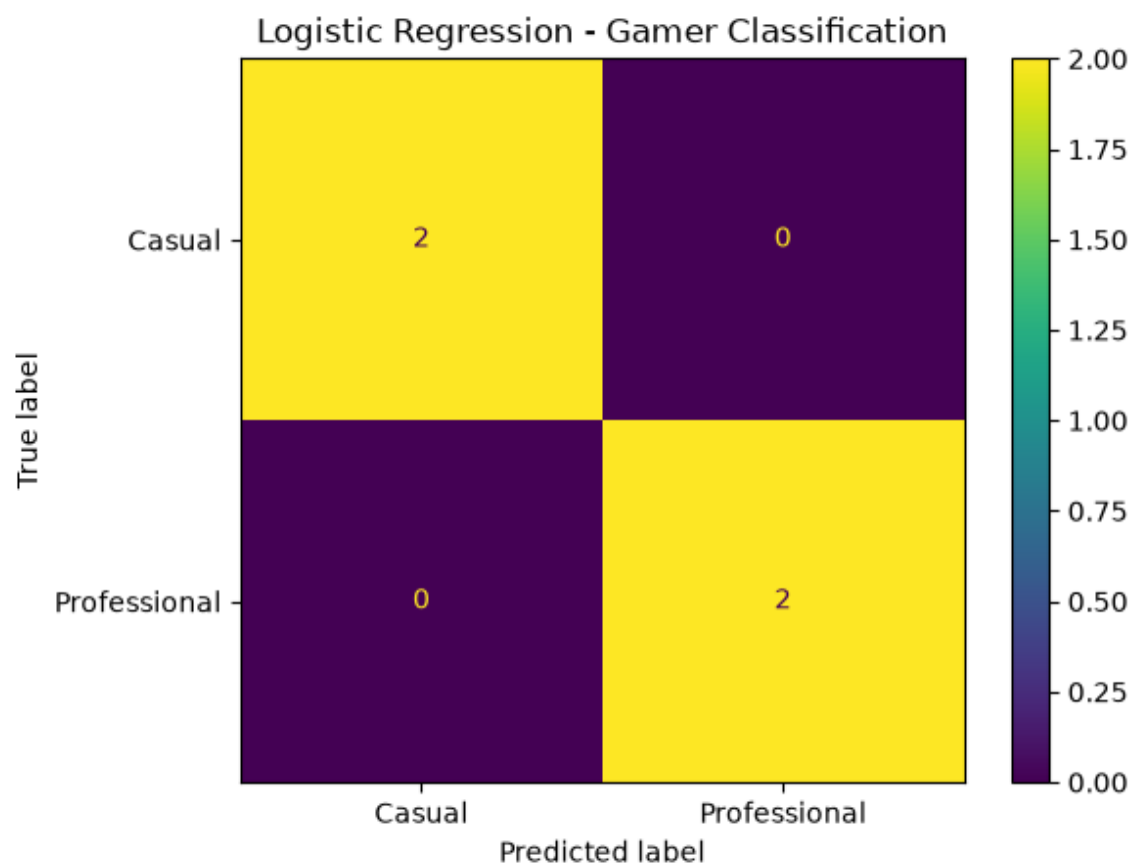
Recall: 1.0

F1-Score: 1.0

Confusion Matrix:

[[2 0]

[0 2]]



Slip 14

Q1. Create a dataframe containing information about different products such as Price, Rating, and Number of Sales. Generate a Dendrogram to group similar products based on their characteristics.

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage

data = {
    "Product": ["Laptop", "Mobile", "Tablet", "Headphones",
                "Smartwatch", "Camera", "TV", "Speaker"],
    "Price": [60000, 25000, 30000, 3000, 8000, 45000, 55000, 7000],
    "Rating": [4.5, 4.3, 4.4, 4.1, 4.2, 4.6, 4.5, 4.0],
    "Sales": [120, 300, 200, 500, 350, 100, 150, 400]
}

df = pd.DataFrame(data)

print("Product Dataset:")
print(df)

X = df[["Price", "Rating", "Sales"]]

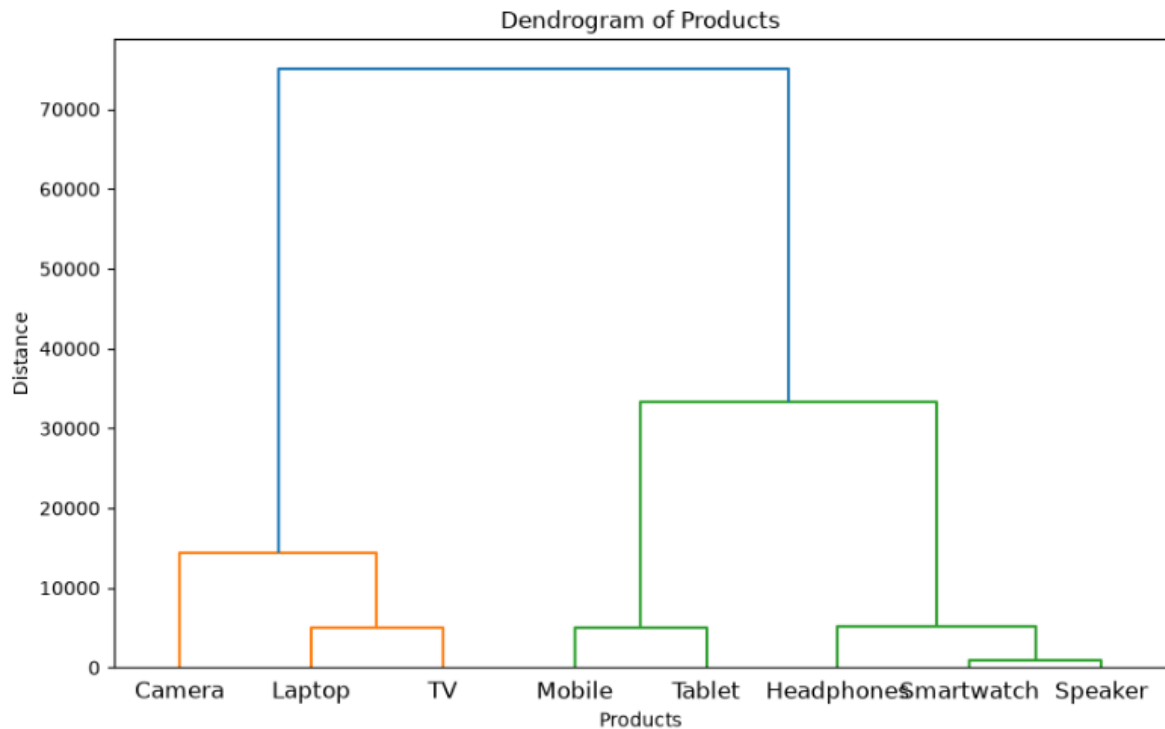
Z = linkage(X, method="ward")

plt.figure(figsize=(10, 6))
dendrogram(Z, labels=df["Product"].values)
plt.title("Dendrogram of Products")
plt.xlabel("Products")
plt.ylabel("Distance")
plt.show()
```

Output

Product Dataset:

	Product	Price	Rating	Sales
0	Laptop	60000	4.5	120
1	Mobile	25000	4.3	300
2	Tablet	30000	4.4	200
3	Headphones	3000	4.1	500
4	Smartwatch	8000	4.2	350
5	Camera	45000	4.6	100
6	TV	55000	4.5	150
7	Speaker	7000	4.0	400



Q2. Using the Car Price Prediction dataset, apply Polynomial Regression to predict the Selling Price based on Vehicle Age, Present Price, and Kilometers Driven. Evaluate the model using the R^2 Score and visualize the actual data points along with the fitted polynomial curve.

```
import pandas as pd

import matplotlib.pyplot as plt

import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

data = {
```

```

    "Vehicle_Age": [2, 3, 4, 5, 6, 7, 8, 9, 10, 3, 4, 5, 6, 7, 8, 9,
2, 5, 7, 10],
    "Present_Price": [5, 7, 8, 10, 12, 15, 18, 20, 22, 6, 9, 11, 13,
16, 19, 21, 4, 10, 17, 23],
    "Kilometers_Driven": [15000, 20000, 25000, 30000, 35000, 40000,
45000, 50000, 55000, 18000, 27000, 32000, 38000, 42000, 48000,
52000, 12000, 33000, 44000, 60000],
    "Selling_Price": [4.5, 5.8, 6.5, 7.8, 8.5, 10.2, 11.5, 12.5,
13.2, 5.2, 7.0, 8.2, 9.0, 10.8, 11.8, 12.8, 3.8, 7.5, 10.5, 13.5]
}

```

```

df = pd.DataFrame(data)
df.to_csv("car_price.csv", index=False)
df = pd.read_csv("car_price.csv")

```

```

print("Car Price Dataset:")
print(df)

```

```

X = df[["Vehicle_Age", "Present_Price", "Kilometers_Driven"]]
y = df["Selling_Price"]

```

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

```

```

poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

```

```

model = LinearRegression()
model.fit(X_train_poly, y_train)

```

```

y_pred = model.predict(X_test_poly)

```



```
r2 = r2_score(y_test, y_pred)
```

```
print("\nR2 Score:", r2)
```

```
X_curve = np.linspace(  
    df["Vehicle_Age"].min(),  
    df["Vehicle_Age"].max(),  
    100  
)
```

```
curve_data = pd.DataFrame({  
    "Vehicle_Age": X_curve,  
    "Present_Price": df["Present_Price"].mean(),  
    "Kilometers_Driven": df["Kilometers_Driven"].mean()  
})
```

```
curve_poly = poly.transform(curve_data)  
y_curve = model.predict(curve_poly)
```

```
plt.scatter(  
    df["Vehicle_Age"],  
    df["Selling_Price"],  
    label="Actual Data"  
)
```

```
plt.plot(  
    X_curve,  
    y_curve,  
    label="Fitted Polynomial Curve"  
)
```

```
plt.xlabel("Vehicle Age")
```

```
plt.ylabel("Selling Price")

plt.title("Polynomial Regression - Car Price Prediction")

plt.legend()

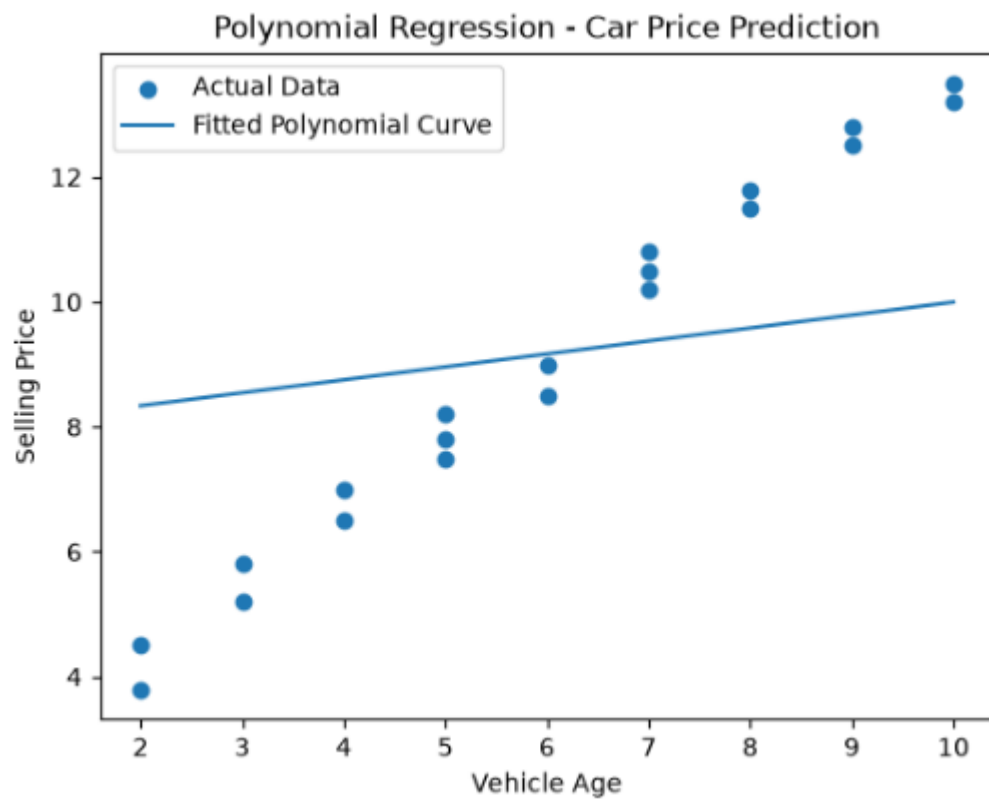
plt.show()
```

Output

Car Price Dataset:

	Vehicle_Age	Present_Price	Kilometers_Driven	Selling_Price
0	2	5	15000	4.5
1	3	7	20000	5.8
2	4	8	25000	6.5
3	5	10	30000	7.8
4	6	12	35000	8.5
5	7	15	40000	10.2
6	8	18	45000	11.5
7	9	20	50000	12.5
8	10	22	55000	13.2
9	3	6	18000	5.2
10	4	9	27000	7.0
11	5	11	32000	8.2
12	6	13	38000	9.0
13	7	16	42000	10.8
14	8	19	48000	11.8
15	9	21	52000	12.8
16	2	4	12000	3.8
17	5	10	33000	7.5
18	7	17	44000	10.5
19	10	23	60000	13.5

R² Score: 0.9969118405158383



OR

Q2. Create two DataFrames containing information about different mobile phones. DataFrame 1: Brand, Model, Price, RAM DataFrame 2: Model, Storage, Battery_Capacity, Operating_System Perform the following data transformation techniques: a) Data Integration by combining the two DataFrames using Model b) Label Encoding on the Brand attribute of the merged DataFrame c) One-Hot Encoding on the Operating_System attribute of the merged DataFrame Display the original DataFrames, merged DataFrame, and transformed DataFrame.

```
import pandas as pd

from sklearn.preprocessing import LabelEncoder

df1 = pd.DataFrame({
    "Brand": ["Samsung", "Apple", "OnePlus", "Xiaomi"],
    "Model": ["S23", "iPhone 15", "12R", "Redmi Note 13"],
    "Price": [70000, 80000, 45000, 20000],
    "RAM": [8, 8, 8, 6]
})

df2 = pd.DataFrame({
    "Model": ["S23", "iPhone 15", "12R", "Redmi Note 13"],
    "Storage": [128, 128, 256, 128],
    "Battery_Capacity": [3900, 3349, 5500, 5000],
    "Operating_System": ["Android", "iOS", "Android", "Android"]
})

print("DataFrame 1:")
print(df1)

print("\nDataFrame 2:")
print(df2)

merged_df = pd.merge(df1, df2, on="Model")

print("\nMerged DataFrame:")
print(merged_df)
```

```

le = LabelEncoder()

merged_df["Brand"] = le.fit_transform(merged_df["Brand"])

merged_df = pd.get_dummies(
    merged_df,
    columns=["Operating_System"],
    dtype=int
)

print("\nTransformed DataFrame:")

print(merged_df)

```

Output

DataFrame 1:

	Brand	Model	Price	RAM
0	Samsung	S23	70000	8
1	Apple	iPhone 15	80000	8
2	OnePlus	12R	45000	8
3	Xiaomi	Redmi Note 13	20000	6

DataFrame 2:

	Model	Storage	Battery_Capacity	Operating_System
0	S23	128	3900	Android
1	iPhone 15	128	3349	iOS
2	12R	256	5500	Android
3	Redmi Note 13	128	5000	Android

Merged DataFrame:

	Brand	Model	Price	RAM	Storage	Battery_Capacity	\
0	Samsung	S23	70000	8	128	3900	
1	Apple	iPhone 15	80000	8	128	3349	
2	OnePlus	12R	45000	8	256	5500	
3	Xiaomi	Redmi Note 13	20000	6	128	5000	

	Operating_System
0	Android
1	iOS
2	Android
3	Android

Transformed DataFrame:

	Brand	Model	Price	RAM	Storage	Battery_Capacity	\
0	2	S23	70000	8	128	3900	
1	0	iPhone 15	80000	8	128	3349	
2	1	12R	45000	8	256	5500	
3	3	Redmi Note 13	20000	6	128	5000	

	Operating_System_Android	Operating_System_iOS
0	1	0
1	0	1
2	1	0
3	1	0

Slip 15

Q1. Write a Python program to create an age_records DataFrame containing the attributes Person_ID and Age for suitable sample records. Perform Data Discretization on the Age attribute using Equal Frequency Binning. Display the original and discretized data.

```
import pandas as pd

age_records = pd.DataFrame({
    "Person_ID": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "Age": [18, 22, 25, 28, 32, 35, 40, 45, 50, 60]
})

print("Original Data:")
print(age_records)

age_records["Age_Group"] = pd.qcut(
    age_records["Age"],
    q=3,
    labels=["Young", "Middle", "Old"]
)

print("\nDiscretized Data:")
print(age_records)
```

Output

Original Data:

	Person_ID	Age
0	1	18
1	2	22
2	3	25
3	4	28
4	5	32
5	6	35
6	7	40
7	8	45
8	9	50
9	10	60

Discretized Data:

	Person_ID	Age	Age_Group
--	-----------	-----	-----------

0	1	18	Young
1	2	22	Young
2	3	25	Young
3	4	28	Middle
4	5	32	Middle
5	6	35	Middle
6	7	40	Old
7	8	45	Old
8	9	50	Old
9	10	60	Old

Q2. Create a DataFrame containing information about different cities with the attributes City, Area (sq. km), Average Temperature (°C), Rainfall (mm), and Population Density. Generate a Treemap to visualize the population density of each city and create a 3D Scatter Plot using Area, Average Temperature, and Rainfall. Add suitable labels, colors, and titles to both visualizations.

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "City": ["Mumbai", "Pune", "Delhi", "Bengaluru", "Chennai",
            "Hyderabad"],
    "Area (sq. km)": [603, 516, 1484, 741, 426, 650],
    "Average Temperature (°C)": [27, 25, 25, 23, 28, 27],
    "Rainfall (mm)": [2200, 722, 800, 1400, 1400, 800],
    "Population Density": [31700, 16000, 11300, 12000, 26000, 18000]
}

df = pd.DataFrame(data)

print("City Data:")
print(df)

plt.figure(figsize=(10, 6))
plt.bar(df["City"], df["Population Density"])
plt.xlabel("City")
plt.ylabel("Population Density")
plt.title("Population Density of Different Cities")
```

```

plt.xticks(rotation=45)

plt.show()

fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection="3d")

scatter = ax.scatter(
    df["Area (sq. km)"],
    df["Average Temperature (°C)"],
    df["Rainfall (mm)"],
    c=df["Population Density"],
    cmap="viridis",
    s=100
)

ax.set_xlabel("Area (sq. km)")
ax.set_ylabel("Average Temperature (°C)")
ax.set_zlabel("Rainfall (mm)")
ax.set_title("3D Scatter Plot of City Data")
plt.colorbar(scatter, ax=ax, label="Population Density")
plt.show()

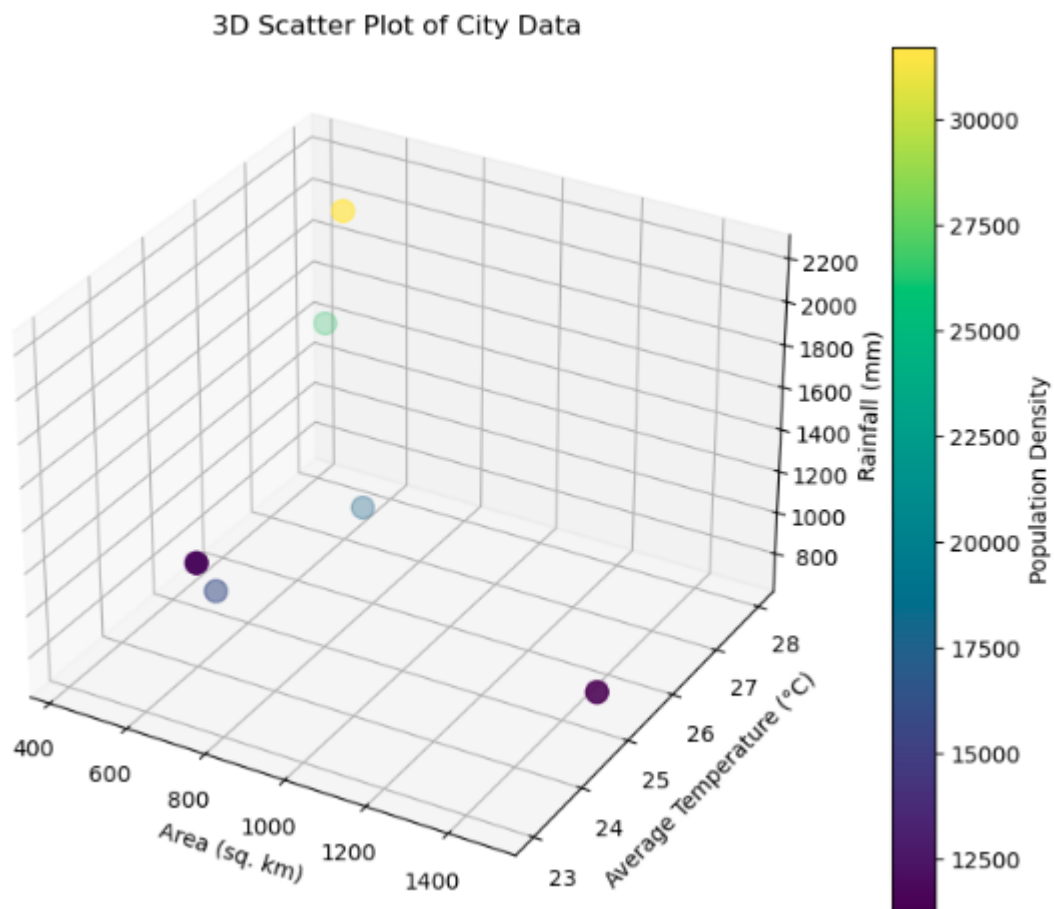
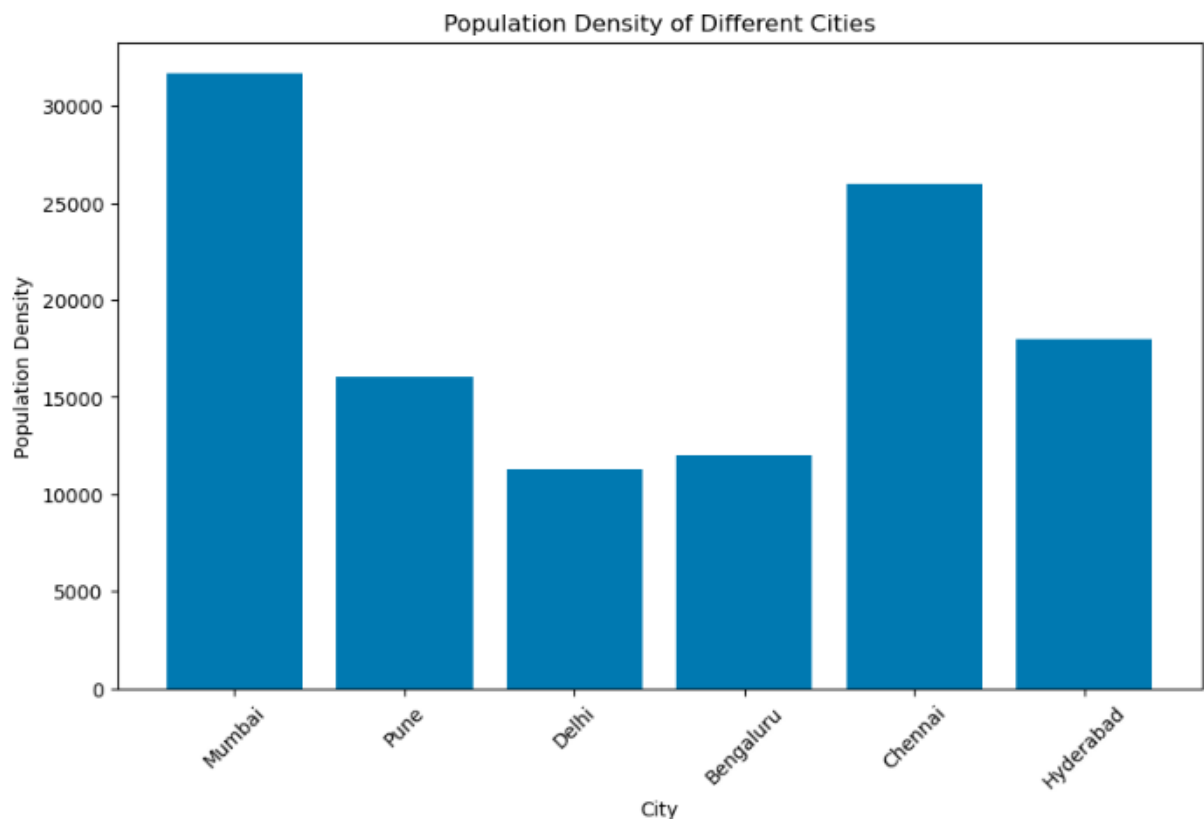
```

Output

City Data:

	City	Area (sq. km)	Average Temperature (°C)	Rainfall (mm)	\
0	Mumbai	603	27	2200	
1	Pune	516	25	722	
2	Delhi	1484	25	800	
3	Bengaluru	741	23	1400	
4	Chennai	426	28	1400	
5	Hyderabad	650	27	800	

	Population Density
0	31700
1	16000
2	11300
3	12000
4	26000
5	18000



OR

Q2. Write a Python program to perform market basket analysis on the Groceries dataset using the Apriori algorithm. Read and explore the dataset, preprocess the data by handling missing values, convert categorical transaction data into a suitable numeric format, and generate frequent itemsets and association rules. Repeat the analysis using minimum support values 0.3 and 0.4.

S.No.	Items Purchased
1	Basmati Rice, Toor Dal, Turmeric Powder, Cooking Oil
2	Atta, Sugar, Tea, Milk
3	Rice, Moong Dal, Salt, Cooking Oil
4	Wheat Flour, Chickpeas, Turmeric Powder, Cumin Seeds
5	Poha, Peanuts, Onions, Cooking Oil
6	Basmati Rice, Paneer, Garam Masala, Tomatoes
7	Atta, Potatoes, Onions, Cooking Oil

```
import pandas as pd

from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

data = {
    "S.No.": [1, 2, 3, 4, 5, 6, 7],
    "Items Purchased": [
        "Basmati Rice, Toor Dal, Turmeric Powder, Cooking Oil",
        "Atta, Sugar, Tea, Milk",
        "Rice, Moong Dal, Salt, Cooking Oil",
        "Wheat Flour, Chickpeas, Turmeric Powder, Cumin Seeds",
        "Poha, Peanuts, Onions, Cooking Oil",
        "Basmati Rice, Paneer, Garam Masala, Tomatoes",
        "Atta, Potatoes, Onions, Cooking Oil"
    ]
}

df = pd.DataFrame(data)
```

```

print("Original Dataset:")
print(df)

print("\nMissing Values:")
print(df.isnull().sum())

df["Items Purchased"] = df["Items Purchased"].fillna("")
df = df[df["Items Purchased"].str.strip() != ""]

transactions = df["Items Purchased"].apply(
    lambda x: [item.strip() for item in x.split(",")]
)

te = TransactionEncoder()
encoded_data = te.fit(transactions).transform(transactions)

basket = pd.DataFrame(encoded_data, columns=te.columns_).astype(int)
print("\nPreprocessed Transaction Data:")
print(basket)

for support in [0.3, 0.4]:
    print("\n" + "=" * 50)
    print("Minimum Support:", support)

    frequent_itemsets = apriori(
        basket,
        min_support=support,
        use_colnames=True
    )

    print("\nFrequent Itemsets:")

```

```

if frequent_itemsets.empty:
    print("No frequent itemsets found.")
    continue

display_itemsets = frequent_itemsets.copy()

display_itemsets["itemsets"] =
display_itemsets["itemsets"].apply(
    lambda x: ", ".join(x)
)

print(display_itemsets)

if frequent_itemsets["itemsets"].apply(len).max() >= 2:
    rules = association_rules(
        frequent_itemsets,
        metric="confidence",
        min_threshold=0.5
    )

    print("\nAssociation Rules:")

    if rules.empty:
        print("No association rules found.")
    else:
        print(rules[
            ["antecedents", "consequents",
            "support", "confidence", "lift"]
        ])
else:
    print("\nAssociation Rules:")
    print("No association rules found.")

```

Output

Original Dataset:

	S.No.	Items Purchased
0	1	Basmati Rice, Toor Dal, Turmeric Powder, Cooki...
1	2	Atta, Sugar, Tea, Milk
2	3	Rice, Moong Dal, Salt, Cooking Oil

3	4	Wheat Flour, Chickpeas, Turmeric Powder, Cumin...
4	5	Poha, Peanuts, Onions, Cooking Oil
5	6	Basmati Rice, Paneer, Garam Masala, Tomatoes
6	7	Atta, Potatoes, Onions, Cooking Oil

Missing Values:

S.No. 0

Items Purchased 0

dtype: int64

Preprocessed Transaction Data:

	Atta	Basmati Rice	Chickpeas	Cooking Oil	Cumin Seeds	Garam Masala	\
0	0	1	0	1	0	0	
1	1	0	0	0	0	0	
2	0	0	0	1	0	0	
3	0	0	1	0	1	0	
4	0	0	0	1	0	0	
5	0	1	0	0	0	1	
6	1	0	0	1	0	0	

	Milk	Moong Dal	Onions	Paneer	...	Poha	Potatoes	Rice	Salt	Sugar	\
0	0	0	0	0	...	0	0	0	0	0	
1	1	0	0	0	...	0	0	0	0	1	
2	0	1	0	0	...	0	0	1	1	0	
3	0	0	0	0	...	0	0	0	0	0	
4	0	0	1	0	...	1	0	0	0	0	
5	0	0	0	1	...	0	0	0	0	0	
6	0	0	1	0	...	0	1	0	0	0	

	Tea	Tomatoes	Toor Dal	Turmeric Powder	Wheat Flour
0	0	0	1		0
1	1	0	0		0
2	0	0	0		0
3	0	0	0		1
4	0	0	0		0
5	0	1	0		0
6	0	0	0		0

[7 rows x 21 columns]

=====

Minimum Support: 0.3

Frequent Itemsets:

support	itemsets
0 0.571429	Cooking Oil

Association Rules:

No association rules found.

=====

Minimum Support: 0.4

Frequent Itemsets:

support	itemsets
0 0.571429	Cooking Oil

Association Rules:

No association rules found.

Slip 16

Q1. Create a DataFrame containing student performance data with attributes such as Study Hours, Attendance, Assignment Score, and Exam Score. Calculate the correlation matrix for these attributes and generate a Heat Map to visualize the correlation between them. Display the correlation values, suitable labels, and an appropriate title.

```
import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

data = {
    "Study Hours": [2, 3, 4, 5, 6, 7, 8, 9],
    "Attendance": [60, 65, 70, 75, 80, 85, 90, 95],
    "Assignment Score": [50, 55, 60, 65, 72, 78, 85, 92],
    "Exam Score": [45, 50, 58, 63, 70, 76, 84, 90]
}

df = pd.DataFrame(data)

print("Student Performance Data:")
print(df)

correlation_matrix = df.corr()

print("\nCorrelation Matrix:")
print(correlation_matrix)

plt.figure(figsize=(8, 6))
sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    linewidths=0.5
```

)

```
plt.title("Correlation Heat Map of Student Performance")
plt.xlabel("Performance Attributes")
plt.ylabel("Performance Attributes")
plt.show()
```

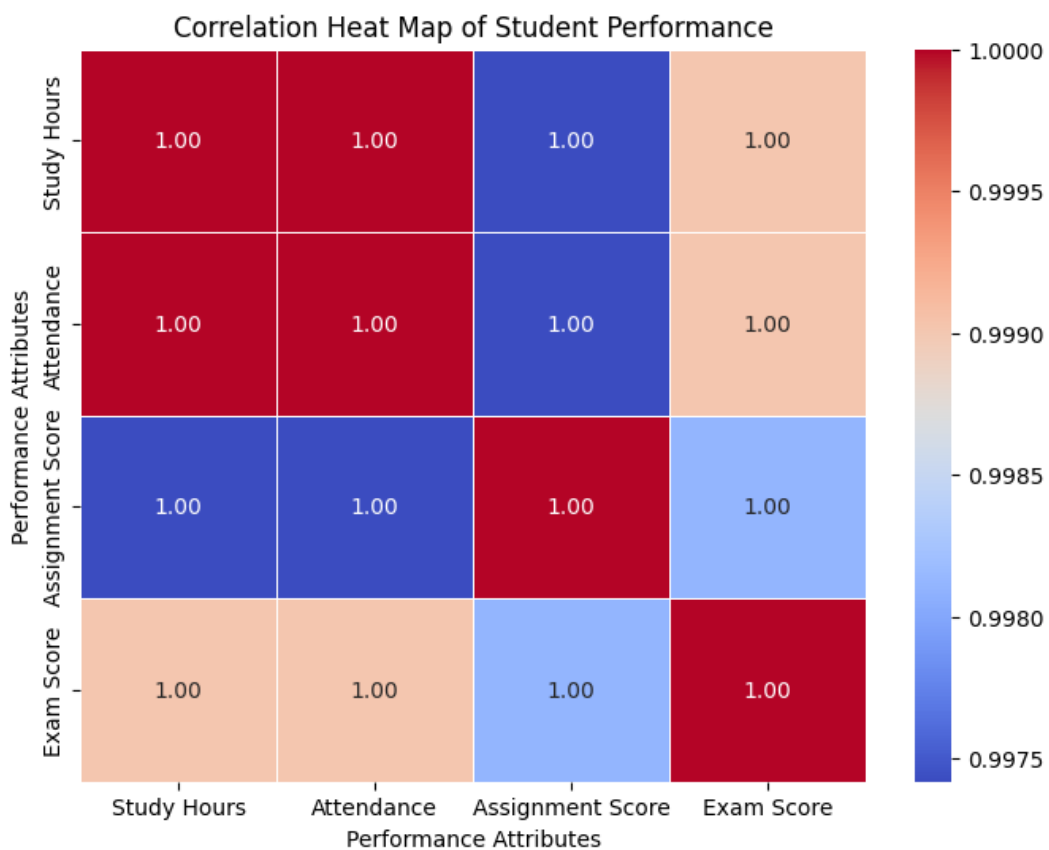
Output

Student Performance Data:

	Study Hours	Attendance	Assignment Score	Exam Score
0	2	60	50	45
1	3	65	55	50
2	4	70	60	58
3	5	75	65	63
4	6	80	72	70
5	7	85	78	76
6	8	90	85	84
7	9	95	92	90

Correlation Matrix:

	Study Hours	Attendance	Assignment Score	Exam Score
Study Hours	1.000000	1.000000	0.997418	0.999015
Attendance	1.000000	1.000000	0.997418	0.999015
Assignment Score	0.997418	0.997418	1.000000	0.998106
Exam Score	0.999015	0.999015	0.998106	1.000000



Q2. Write a Python program to create a DataFrame with the attributes Temperature, Humidity, Wind-Speed, Pressure, and Rainfall. Apply the following data transformation techniques and display the transformed data:

a)Min-MaxScaling b)Standardization

c) Normalization

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler,
Normalizer
```

```
data = {
    "Temperature": [20, 25, 30, 35, 40],
    "Humidity": [40, 50, 60, 70, 80],
    "Wind-Speed": [5, 10, 15, 20, 25],
    "Pressure": [1000, 1005, 1010, 1015, 1020],
    "Rainfall": [10, 20, 30, 40, 50]
}
```

```
df = pd.DataFrame(data)
```

```
print("Original Data:")
print(df)
```

```
minmax = MinMaxScaler()
minmax_data = pd.DataFrame(minmax.fit_transform(df),
columns=df.columns)
```

```
print("\nA) Min-Max Scaling:")
print(minmax_data)
```

```
standard = StandardScaler()
standard_data = pd.DataFrame(standard.fit_transform(df),
columns=df.columns)
```

```
print("\nB) Standardization:")
print(standard_data)
```

```
normalizer = Normalizer()
normalized_data = pd.DataFrame(normalizer.fit_transform(df),
columns=df.columns)
```

```
print("\nC) Normalization:")
print(normalized_data)
```

Output

Original Data:

	Temperature	Humidity	Wind-Speed	Pressure	Rainfall
0	20	40	5	1000	10
1	25	50	10	1005	20
2	30	60	15	1010	30
3	35	70	20	1015	40
4	40	80	25	1020	50

A) Min-Max Scaling:

	Temperature	Humidity	Wind-Speed	Pressure	Rainfall
0	0.00	0.00	0.00	0.00	0.00
1	0.25	0.25	0.25	0.25	0.25
2	0.50	0.50	0.50	0.50	0.50
3	0.75	0.75	0.75	0.75	0.75
4	1.00	1.00	1.00	1.00	1.00

B) Standardization:

	Temperature	Humidity	Wind-Speed	Pressure	Rainfall
0	-1.414214	-1.414214	-1.414214	-1.414214	-1.414214
1	-0.707107	-0.707107	-0.707107	-0.707107	-0.707107
2	0.000000	0.000000	0.000000	0.000000	0.000000
3	0.707107	0.707107	0.707107	0.707107	0.707107
4	1.414214	1.414214	1.414214	1.414214	1.414214

C) Normalization:

	Temperature	Humidity	Wind-Speed	Pressure	Rainfall
0	0.019979	0.039958	0.004995	0.998939	0.009989
1	0.024831	0.049662	0.009932	0.998210	0.019865
2	0.029621	0.059243	0.014811	0.997254	0.029621
3	0.034348	0.068695	0.019627	0.996080	0.039254
4	0.039008	0.078015	0.024380	0.994696	0.048760

OR

Q2. Implement the Apriori algorithm on the given student activity dataset to identify frequent itemsets and generate association rules among the activities. Repeat the analysis using minimum support values 0.2 and 0.3.

TID	Items
1.	Library, Seminar
2.	Workshop, Project
3.	Library, Assignment
4.	Seminar, Workshop
5.	Assignment, Seminar
6.	Workshop, Assignment
7.	Project, Library
8.	Library, Seminar, Project
9.	Library, Seminar, Workshop

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules
```

```
data = {
    "TID": [1, 2, 3, 4, 5, 6, 7, 8, 9],
    "Items": [
        "Library, Seminar",
        "Workshop, Project",
        "Library, Assignment",
```



```

        "Seminar, Workshop",
        "Assignment, Seminar",
        "Workshop, Assignment",
        "Project, Library",
        "Library, Seminar, Project",
        "Library, Seminar, Workshop"
    ]
}

df = pd.DataFrame(data)

print("Original Dataset:")
print(df)

transactions = df["Items"].apply(
    lambda x: [item.strip() for item in x.split(",")]
)

te = TransactionEncoder()
encoded = te.fit(transactions).transform(transactions)

basket = pd.DataFrame(encoded, columns=te.columns_).astype(int)

print("\nPreprocessed Transaction Data:")
print(basket)

for support in [0.2, 0.3]:

    print("\n" + "=" * 60)
    print("Minimum Support:", support)

    frequent_itemsets = apriori(
        basket,
        min_support=support,
        use_colnames=True
    )

    print("\nFrequent Itemsets:")

    if frequent_itemsets.empty:
        print("No frequent itemsets found.")
        continue

    display_itemsets = frequent_itemsets.copy()
    display_itemsets["itemsets"] =
display_itemsets["itemsets"].apply(
    lambda x: ", ".join(x)
)

    print(display_itemsets)

    rules = association_rules(
        frequent_itemsets,
        metric="confidence",
        min_threshold=0.5
    )

```

```

print("\nAssociation Rules:")

if rules.empty:
    print("No association rules found.")
else:
    rules["antecedents"] = rules["antecedents"].apply(
        lambda x: ", ".join(x)
    )
    rules["consequents"] = rules["consequents"].apply(
        lambda x: ", ".join(x)
    )

    print(
        rules[
            ["antecedents", "consequents",
             "support", "confidence", "lift"]
        ]
    )

```

Output

Original Dataset:

	TID	Items
0	1	Library, Seminar
1	2	Workshop, Project
2	3	Library, Assignment
3	4	Seminar, Workshop
4	5	Assignment, Seminar
5	6	Workshop, Assignment
6	7	Project, Library
7	8	Library, Seminar, Project
8	9	Library, Seminar, Workshop

Preprocessed Transaction Data:

	Assignment	Library	Project	Seminar	Workshop
0	0	1	0	1	0
1	0	0	1	0	1
2	1	1	0	0	0
3	0	0	0	1	1
4	1	0	0	1	0
5	1	0	0	0	1
6	0	1	1	0	0
7	0	1	1	1	0
8	0	1	0	1	1

=====

Minimum Support: 0.2

Frequent Itemsets:

	support	itemsets
0	0.333333	Assignment
1	0.555556	Library
2	0.333333	Project
3	0.555556	Seminar
4	0.444444	Workshop
5	0.222222	Project, Library
6	0.333333	Library, Seminar

7 0.222222 Workshop, Seminar

Association Rules:

	antecedents	consequents	support	confidence	lift
0	Project	Library	0.222222	0.666667	1.20
1	Library	Seminar	0.333333	0.600000	1.08
2	Seminar	Library	0.333333	0.600000	1.08
3	Workshop	Seminar	0.222222	0.500000	0.90

=====

Minimum Support: 0.3

Frequent Itemsets:

	support	itemsets
0	0.333333	Assignment
1	0.555556	Library
2	0.333333	Project
3	0.555556	Seminar
4	0.444444	Workshop
5	0.333333	Library, Seminar

Association Rules:

	antecedents	consequents	support	confidence	lift
0	Library	Seminar	0.333333	0.6	1.08
1	Seminar	Library	0.333333	0.6	1.08

Slip 17

Q1. Write a Python program to create two DataFrames, student_details and academic_records, containing suitable student-related attributes. Perform data integration by combining the two DataFrames using a common attribute such as Student_ID. Display the individual DataFrames and the integrated DataFrame.

```
import pandas as pd

student_details = pd.DataFrame({
    "Student_ID": [101, 102, 103, 104, 105],
    "Name": ["Riya", "Amit", "Sneha", "Rahul", "Pooja"],
    "Age": [20, 21, 20, 22, 21],
    "Gender": ["Female", "Male", "Female", "Male", "Female"]
})

academic_records = pd.DataFrame({
    "Student_ID": [101, 102, 103, 104, 105],
    "Course": ["BCS", "BCS", "BCA", "BCS", "BCA"],
    "Marks": [85, 78, 92, 75, 88],
    "Grade": ["A", "B+", "A+", "B", "A"]
})

print("Student Details:")
print(student_details)

print("\nAcademic Records:")
print(academic_records)

integrated_data = pd.merge(
    student_details,
    academic_records,
    on="Student_ID",
    how="inner"
)

print("\nIntegrated Data:")
print(integrated_data)
```

Output

Student Details:

	Student_ID	Name	Age	Gender
0	101	Riya	20	Female
1	102	Amit	21	Male
2	103	Sneha	20	Female
3	104	Rahul	22	Male
4	105	Pooja	21	Female

Academic Records:

	Student_ID	Course	Marks	Grade
0	101	BCS	85	A
1	102	BCS	78	B+
2	103	BCA	92	A+
3	104	BCS	75	B

4	105	BCA	88	A
---	-----	-----	----	---

Integrated Data:

	Student_ID	Name	Age	Gender	Course	Marks	Grade
0	101	Riya	20	Female	BCS	85	A
1	102	Amit	21	Male	BCS	78	B+
2	103	Sneha	20	Female	BCA	92	A+
3	104	Rahul	22	Male	BCS	75	B
4	105	Pooja	21	Female	BCA	88	A

Q2. Create a dataset containing information about different cities with the attributes: City, Population, GDP, and Literacy Rate. Perform the following visualization tasks:

a. Generate a Treemap showing city-wise population distribution.

b. Create a 3D Scatter Plot using Population, GDP, and Literacy Rate.

```
import pandas as pd
import matplotlib.pyplot as plt
import squarify

data = {
    "City": ["Mumbai", "Pune", "Delhi", "Bengaluru", "Chennai",
            "Hyderabad"],
    "Population": [20.7, 7.4, 32.9, 13.6, 11.2, 10.8],
    "GDP": [310, 69, 293, 110, 78, 75],
    "Literacy Rate": [89, 88, 87, 88, 90, 84]
}

df = pd.DataFrame(data)

print("City Dataset:")
print(df)

plt.figure(figsize=(10, 6))
squarify.plot(
    sizes=df["Population"],
    label=df["City"],
    alpha=0.8
)
plt.title("City-wise Population Distribution")
plt.axis("off")
plt.show()

fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection="3d")

ax.scatter(
    df["Population"],
    df["GDP"],
    df["Literacy Rate"],
    s=100
)

ax.set_xlabel("Population (Million)")
ax.set_ylabel("GDP (Billion USD)")
```

```
ax.set_zlabel("Literacy Rate (%)")
ax.set_title("3D Scatter Plot of City Data")

plt.show()
```

Output

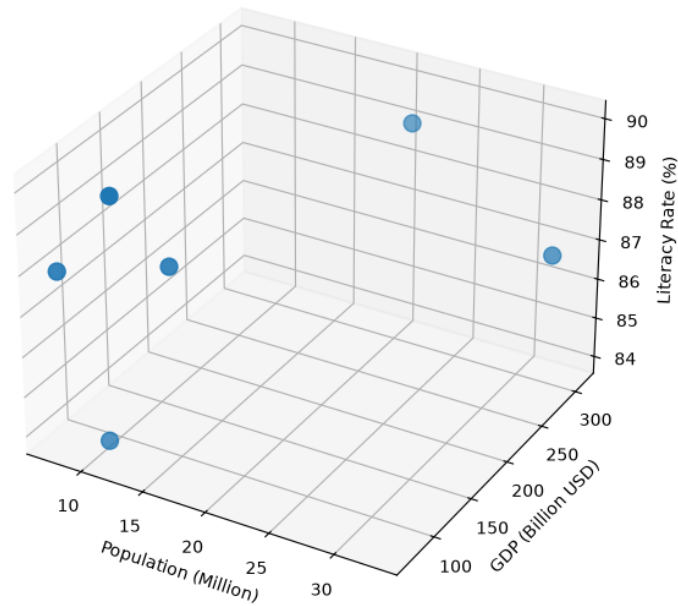
City Dataset:

	City	Population	GDP	Literacy Rate
0	Mumbai	20.7	310	89
1	Pune	7.4	69	88
2	Delhi	32.9	293	87
3	Bengaluru	13.6	110	88
4	Chennai	11.2	78	90
5	Hyderabad	10.8	75	84

City-wise Population Distribution



3D Scatter Plot of City Data



OR

Q2. Develop a Random Forest Classifier using the given Student Grading dataset to classify students into different performance categories. Perform the necessary data preprocessing, model training, and testing, and evaluate the classifier using appropriate classification metrics such as Confusion matrix, Accuracy, Precision, Recall, and F1-Score.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, accuracy_score,
precision_score, recall_score, f1_score

data = {
    "Study_Hours": [2, 3, 4, 5, 6, 7, 8, 9, 2, 4, 6, 8, 3, 5, 7, 9,
1, 4, 6, 8],
    "Attendance": [60, 65, 70, 75, 80, 85, 90, 95, 55, 72, 82, 92,
63, 78, 88, 96, 50, 74, 84, 91],
    "Assignment_Score": [50, 55, 60, 65, 72, 78, 85, 92, 45, 62, 75,
88, 52, 68, 80, 94, 40, 64, 77, 90],
    "Exam_Score": [45, 50, 58, 63, 70, 76, 84, 90, 40, 56, 73, 86,
48, 65, 78, 92, 35, 60, 74, 88],
    "Performance": [
        "Low", "Low", "Medium", "Medium", "Medium",
        "High", "High", "High", "Low", "Medium",
        "Medium", "High", "Low", "Medium", "High",
        "High", "Low", "Medium", "Medium", "High"
    ]
}

df = pd.DataFrame(data)

print("Student Grading Dataset:")
print(df)

print("\nMissing Values:")
print(df.isnull().sum())

X = df[["Study_Hours", "Attendance", "Assignment_Score",
"Exam_Score"]]
y = df["Performance"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

model.fit(X_train, y_train)
```

```

y_pred = model.predict(X_test)

cm = confusion_matrix(y_test, y_pred, labels=["Low", "Medium",
"High"])

print("\nConfusion Matrix:")
print(cm)

print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred,
average="weighted", zero_division=0))
print("Recall:", recall_score(y_test, y_pred, average="weighted",
zero_division=0))
print("F1-Score:", f1_score(y_test, y_pred, average="weighted",
zero_division=0))

```

Output

Student Grading Dataset:

	Study_Hours	Attendance	Assignment_Score	Exam_Score	Performance
0	2	60	50	45	Low
1	3	65	55	50	Low
2	4	70	60	58	Medium
3	5	75	65	63	Medium
4	6	80	72	70	Medium
5	7	85	78	76	High
6	8	90	85	84	High
7	9	95	92	90	High
8	2	55	45	40	Low
9	4	72	62	56	Medium
10	6	82	75	73	Medium
11	8	92	88	86	High
12	3	63	52	48	Low
13	5	78	68	65	Medium
14	7	88	80	78	High
15	9	96	94	92	High
16	1	50	40	35	Low
17	4	74	64	60	Medium
18	6	84	77	74	Medium
19	8	91	90	88	High

Missing Values:

```

Study_Hours      0
Attendance        0
Assignment_Score  0
Exam_Score        0
Performance       0
dtype: int64

```

Confusion Matrix:

```

[[2 0 0]
 [0 2 0]
 [0 0 2]]

```

```

Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1-Score: 1.0

```


Slip 18

Q1. Write a Python program to create an employee_data DataFrame with the attributes: Emp_ID, Age, Dept, Years_Experience, and Performance_Rating. Perform data cleaning by handling missing values, remove duplicate records and detecting outliers using the IQR method.

```
import pandas as pd

data = {
    "Emp_ID": [101, 102, 103, 104, 105, 105, 106, 107, 108, 109],
    "Age": [25, 28, 32, 35, 40, 40, 45, 50, 52, 100],
    "Dept": ["HR", "IT", "Sales", "IT", None, None, "HR", "Sales",
            "IT", "HR"],
    "Years_Experience": [2, 4, 7, 10, 15, 15, 20, 25, 28, 35],
    "Performance_Rating": [3, 4, 4, 5, 3, 3, 4, 5, 4, 2]
}

employee_data = pd.DataFrame(data)

print("Original Employee Data:")
print(employee_data)

print("\nMissing Values:")
print(employee_data.isnull().sum())

employee_data["Dept"] =
employee_data["Dept"].fillna(employee_data["Dept"].mode()[0])

employee_data = employee_data.drop_duplicates()

print("\nData After Removing Duplicates:")
print(employee_data)

Q1 = employee_data["Age"].quantile(0.25)
Q3 = employee_data["Age"].quantile(0.75)
IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

print("\nIQR:", IQR)
print("Lower Limit:", lower_limit)
print("Upper Limit:", upper_limit)

outliers = employee_data[
    (employee_data["Age"] < lower_limit) |
    (employee_data["Age"] > upper_limit)
]

print("\nDetected Outliers:")
print(outliers)

employee_data = employee_data[
    (employee_data["Age"] >= lower_limit) &
    (employee_data["Age"] <= upper_limit)
]
```

```
1
```

```
print("\nFinal Cleaned Employee Data:")
print(employee_data)
```

Output

Original Employee Data:

	Emp_ID	Age	Dept	Years_Experience	Performance_Rating
0	101	25	HR	2	3
1	102	28	IT	4	4
2	103	32	Sales	7	4
3	104	35	IT	10	5
4	105	40	NaN	15	3
5	105	40	NaN	15	3
6	106	45	HR	20	4
7	107	50	Sales	25	5
8	108	52	IT	28	4
9	109	100	HR	35	2

Missing Values:

```
Emp_ID      0
Age          0
Dept        2
Years_Experience  0
Performance_Rating  0
dtype: int64
```

Data After Removing Duplicates:

	Emp_ID	Age	Dept	Years_Experience	Performance_Rating
0	101	25	HR	2	3
1	102	28	IT	4	4
2	103	32	Sales	7	4
3	104	35	IT	10	5
4	105	40	HR	15	3
6	106	45	HR	20	4
7	107	50	Sales	25	5
8	108	52	IT	28	4
9	109	100	HR	35	2

IQR: 18.0

Lower Limit: 5.0

Upper Limit: 77.0

Detected Outliers:

	Emp_ID	Age	Dept	Years_Experience	Performance_Rating
9	109	100	HR	35	2

Final Cleaned Employee Data:

	Emp_ID	Age	Dept	Years_Experience	Performance_Rating
0	101	25	HR	2	3
1	102	28	IT	4	4
2	103	32	Sales	7	4
3	104	35	IT	10	5
4	105	40	HR	15	3
6	106	45	HR	20	4
7	107	50	Sales	25	5
8	108	52	IT	28	4

Q2. Consider the World Cities dataset containing attributes - city, country, latitude, longitude, and population information. Perform the following tasks:

a. Create a Geospatial Visualization showing the locations of cities on a world map, using marker size to represent population.

b. Create a Treemap to visualize the population distribution of cities.

```
import pandas as pd
import plotly.express as px
import matplotlib.pyplot as plt
import squarify

data = {
    "City": ["Mumbai", "Delhi", "New York", "London", "Tokyo",
            "Paris", "Sydney", "Dubai"],
    "Country": ["India", "India", "USA", "UK", "Japan", "France",
               "Australia", "UAE"],
    "Latitude": [19.0760, 28.6139, 40.7128, 51.5074, 35.6762,
                 48.8566, -33.8688, 25.2048],
    "Longitude": [72.8777, 77.2090, -74.0060, -0.1278, 139.6503,
                  2.3522, 151.2093, 55.2708],
    "Population": [20.7, 32.9, 8.3, 9.0, 14.0, 2.1, 5.3, 3.6]
}

df = pd.DataFrame(data)

print("World Cities Dataset:")
print(df)

# a) Geospatial Visualization
fig = px.scatter_geo(
    df,
    lat="Latitude",
    lon="Longitude",
    size="Population",
    hover_name="City",
    hover_data=["Country", "Population"],
    projection="natural earth",
    title="World Cities - Population Distribution"
)

fig.update_layout(
    geo=dict(showland=True)
)

fig.show()

# b) Treemap
plt.figure(figsize=(12, 7))

squarify.plot(
    sizes=df["Population"],
    label=df["City"],
    alpha=0.8
)
```

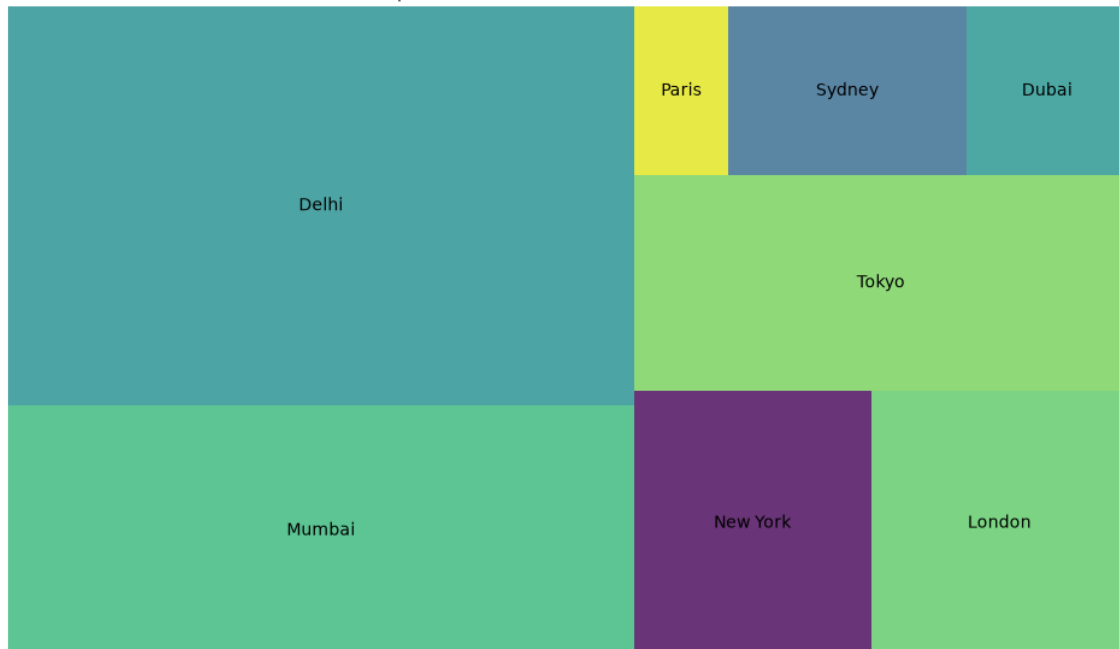
```
plt.title("Population Distribution of World Cities")
plt.axis("off")
plt.show()
```

Output

World Cities Dataset:

	City	Country	Latitude	Longitude	Population
0	Mumbai	India	19.0760	72.8777	20.7
1	Delhi	India	28.6139	77.2090	32.9
2	New York	USA	40.7128	-74.0060	8.3
3	London	UK	51.5074	-0.1278	9.0
4	Tokyo	Japan	35.6762	139.6503	14.0
5	Paris	France	48.8566	2.3522	2.1
6	Sydney	Australia	-33.8688	151.2093	5.3
7	Dubai	UAE	25.2048	55.2708	3.6

Population Distribution of World Cities



OR

Q2. Use the Crop Yield dataset to apply Simple Linear Regression for predicting crop yield from rainfall. Handle missing values, split the data into training and testing sets, and evaluate the model using MAE and R² Score. Create a Scatter Plot with a Regression Line to visualize the relationship.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

data = {
    "Rainfall": [400, 450, 500, 550, 600, 650, 700, 750, 800, 850,
    900, 950],
```

```

    "Crop_Yield": [2.1, 2.4, 2.6, 2.9, 3.1, 3.4, 3.6, 3.9, 4.2, 4.4,
4.7, 5.0]
}

df = pd.DataFrame(data)

print("Original Crop Yield Dataset:")
print(df)

print("\nMissing Values:")
print(df.isnull().sum())

df["Rainfall"] = df["Rainfall"].fillna(df["Rainfall"].mean())
df["Crop_Yield"] = df["Crop_Yield"].fillna(df["Crop_Yield"].mean())

X = df[["Rainfall"]]
y = df["Crop_Yield"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("\nModel Evaluation:")
print("MAE:", mae)
print("R2 Score:", r2)

print("\nRegression Equation:")
print("Crop Yield =", model.intercept_, "+", model.coef_[0], "*
Rainfall")

plt.figure(figsize=(8, 6))
plt.scatter(df["Rainfall"], df["Crop_Yield"], label="Actual Data")
plt.plot(df["Rainfall"], model.predict(X), label="Regression Line")
plt.xlabel("Rainfall (mm)")
plt.ylabel("Crop Yield (tons/hectare)")
plt.title("Rainfall vs Crop Yield")
plt.legend()
plt.show()

```

Output

Original Crop Yield Dataset:

	Rainfall	Crop_Yield
0	400	2.1
1	450	2.4
2	500	2.6
3	550	2.9

4	600	3.1
5	650	3.4
6	700	3.6
7	750	3.9
8	800	4.2
9	850	4.4
10	900	4.7
11	950	5.0

Missing Values:

Rainfall 0

Crop_Yield 0

dtype: int64

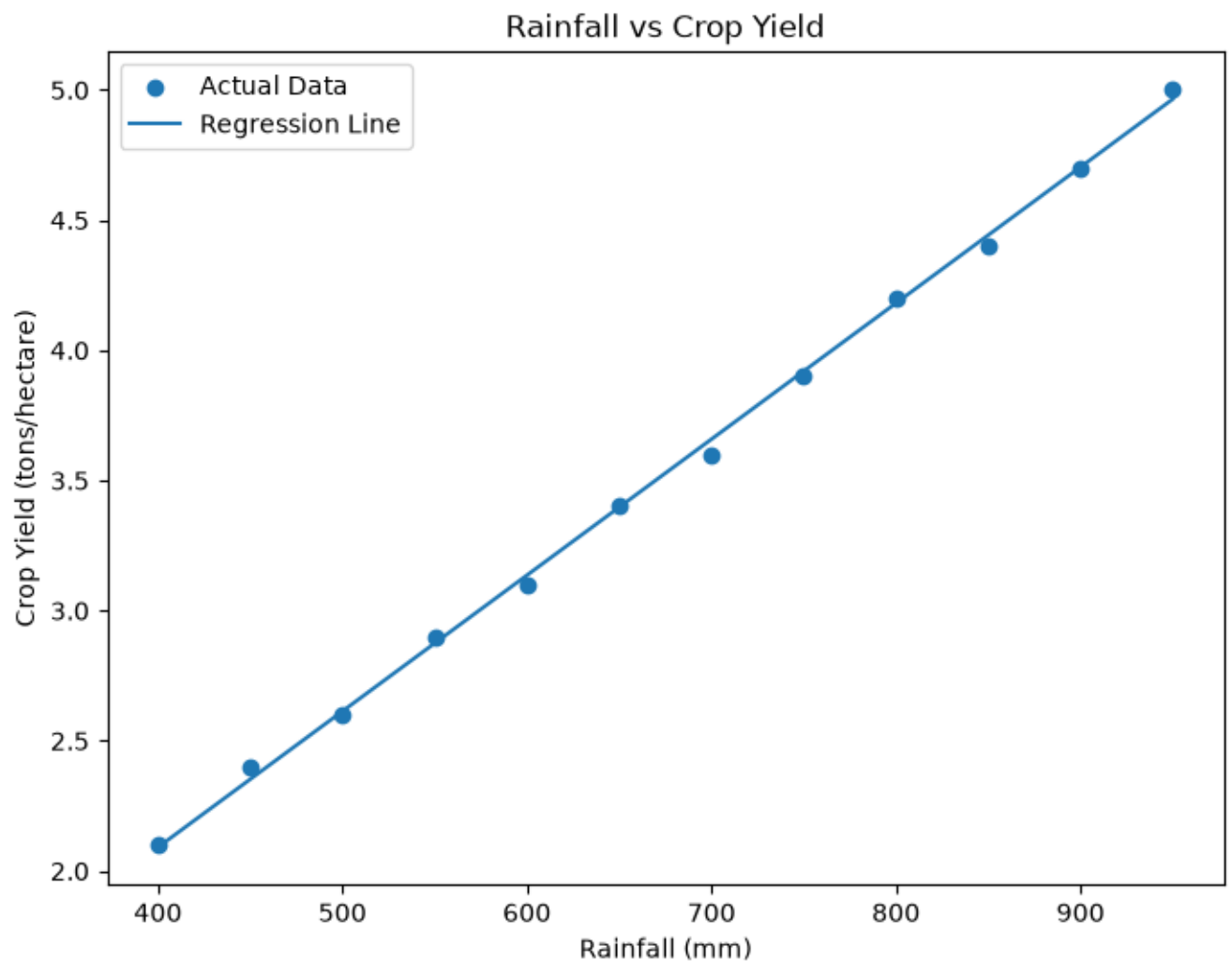
Model Evaluation:

MAE: 0.018621973929235924

R² Score: 0.999524224059711

Regression Equation:

Crop Yield = 0.00041899441340875754 + 0.005226256983240222 * Rainfall



Slip 19

Q1. Create a dataframe containing information about different vehicles such as Fuel_Efficiency, Engine_Power, Vehicle_Weight, Engine_Size, and Acceleration. Add 5 records in dataframe, including outlier. Create Box Plots for Fuel Efficiency and Engine Power and identify outliers.

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Fuel_Efficiency": [18, 20, 22, 19, 50],
    "Engine_Power": [100, 120, 140, 110, 300],
    "Vehicle_Weight": [1200, 1300, 1400, 1250, 1500],
    "Engine_Size": [1.2, 1.5, 1.8, 1.3, 2.5],
    "Acceleration": [12, 11, 10, 12, 8]
}

df = pd.DataFrame(data)

print("Vehicle Dataset:")
print(df)

plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.boxplot(df["Fuel_Efficiency"])
plt.title("Fuel Efficiency")
plt.ylabel("Fuel Efficiency")

plt.subplot(1, 2, 2)
plt.boxplot(df["Engine_Power"])
plt.title("Engine Power")
plt.ylabel("Engine Power")
```

```

plt.tight_layout()
plt.show()

Q1_fuel = df["Fuel_Efficiency"].quantile(0.25)
Q3_fuel = df["Fuel_Efficiency"].quantile(0.75)
IQR_fuel = Q3_fuel - Q1_fuel
lower_fuel = Q1_fuel - 1.5 * IQR_fuel
upper_fuel = Q3_fuel + 1.5 * IQR_fuel

fuel_outliers = df[
    (df["Fuel_Efficiency"] < lower_fuel) |
    (df["Fuel_Efficiency"] > upper_fuel)
]

Q1_power = df["Engine_Power"].quantile(0.25)
Q3_power = df["Engine_Power"].quantile(0.75)
IQR_power = Q3_power - Q1_power
lower_power = Q1_power - 1.5 * IQR_power
upper_power = Q3_power + 1.5 * IQR_power

power_outliers = df[
    (df["Engine_Power"] < lower_power) |
    (df["Engine_Power"] > upper_power)
]

print("\nFuel Efficiency Outliers:")
print(fuel_outliers["Fuel_Efficiency"].tolist())

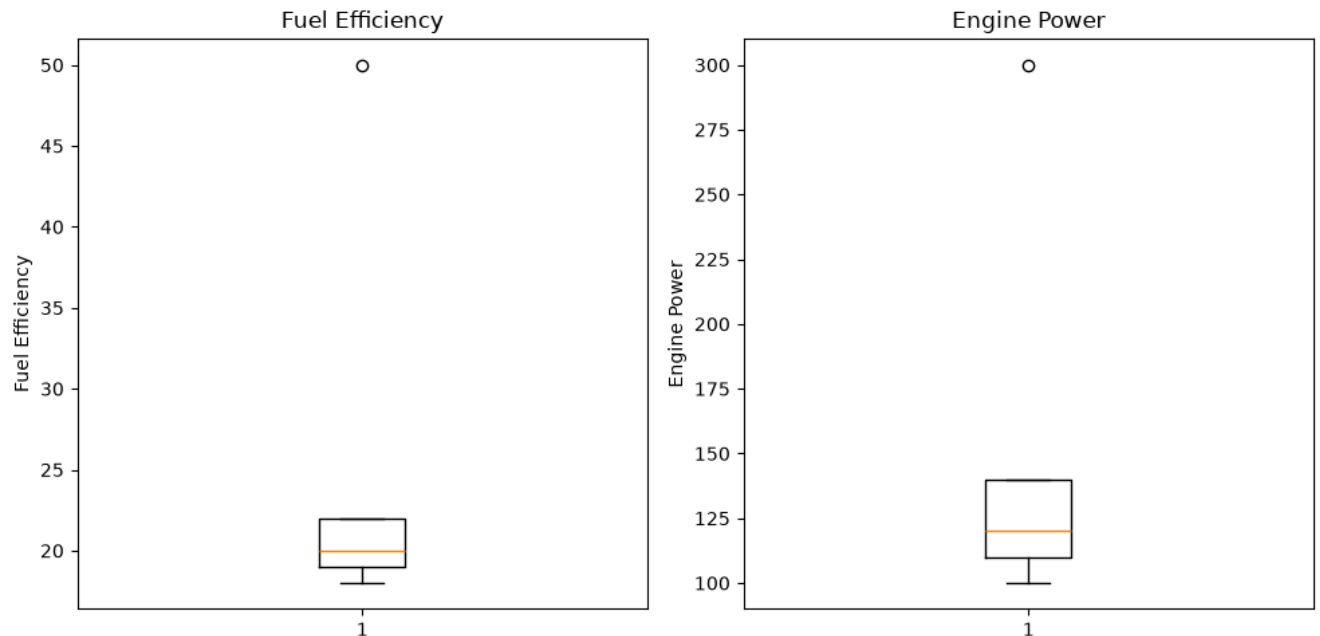
print("\nEngine Power Outliers:")
print(power_outliers["Engine_Power"].tolist())

```


Output

Vehicle Dataset:

	Fuel_Efficiency	Engine_Power	Vehicle_Weight	Engine_Size	Acceleration
0	18	100	1200	1.2	12
1	20	120	1300	1.5	11
2	22	140	1400	1.8	10
3	19	110	1250	1.3	12
4	50	300	1500	2.5	8



Fuel Efficiency Outliers:

[50]

Engine Power Outliers:

[300]

Q2. Write a Python program to create a DataFrame with the attributes Temperature, Humidity, Wind Speed, Pressure, and Rainfall. Apply the following data transformation techniques and display the transformed data:

a)Min-MaxScaling b)Standardization c) Normalization

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler,
Normalizer
```

```
data = {
    "Temperature": [20, 25, 30, 35, 40],
    "Humidity": [40, 50, 60, 70, 80],
    "Wind Speed": [5, 10, 15, 20, 25],
    "Pressure": [1000, 1005, 1010, 1015, 1020],
    "Rainfall": [10, 20, 30, 40, 50]
}
```

```
df = pd.DataFrame(data)
```

```

print("Original Data:")
print(df)

# a) Min-Max Scaling
minmax = MinMaxScaler()
minmax_data = pd.DataFrame(
    minmax.fit_transform(df),
    columns=df.columns
)

print("\nA) Min-Max Scaling:")
print(minmax_data)

# b) Standardization
standard = StandardScaler()
standard_data = pd.DataFrame(
    standard.fit_transform(df),
    columns=df.columns
)

print("\nB) Standardization:")
print(standard_data)

# c) Normalization
normalizer = Normalizer()
normalized_data = pd.DataFrame(
    normalizer.fit_transform(df),
    columns=df.columns
)

print("\nC) Normalization:")
print(normalized_data)

```

Output

Original Data:

	Temperature	Humidity	Wind Speed	Pressure	Rainfall
0	20	40	5	1000	10
1	25	50	10	1005	20
2	30	60	15	1010	30
3	35	70	20	1015	40
4	40	80	25	1020	50

A) Min-Max Scaling:

	Temperature	Humidity	Wind Speed	Pressure	Rainfall
0	0.00	0.00	0.00	0.00	0.00
1	0.25	0.25	0.25	0.25	0.25
2	0.50	0.50	0.50	0.50	0.50
3	0.75	0.75	0.75	0.75	0.75
4	1.00	1.00	1.00	1.00	1.00

B) Standardization:

	Temperature	Humidity	Wind Speed	Pressure	Rainfall
0	-1.414214	-1.414214	-1.414214	-1.414214	-1.414214
1	-0.707107	-0.707107	-0.707107	-0.707107	-0.707107
2	0.000000	0.000000	0.000000	0.000000	0.000000
3	0.707107	0.707107	0.707107	0.707107	0.707107
4	1.414214	1.414214	1.414214	1.414214	1.414214

C) Normalization:

	Temperature	Humidity	Wind Speed	Pressure	Rainfall
0	0.019979	0.039958	0.004995	0.998939	0.009989
1	0.024831	0.049662	0.009932	0.998210	0.019865
2	0.029621	0.059243	0.014811	0.997254	0.029621
3	0.034348	0.068695	0.019627	0.996080	0.039254
4	0.039008	0.078015	0.024380	0.994696	0.048760

OR

Q2. Implement the Random Forest Classification algorithm on the given Heart Disease dataset to predict the presence or absence of heart disease. Perform the necessary data preprocessing, model training, and testing, and evaluate the model using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, confusion_matrix

data = {
    "Age": [63, 37, 41, 56, 57, 57, 56, 44, 52, 57, 54, 48, 49, 64,
58],
    "Sex": [1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0],
    "ChestPain": [3, 2, 1, 1, 0, 0, 1, 2, 2, 0, 2, 1, 1, 3, 0],
    "RestBP": [145, 130, 130, 120, 120, 140, 140, 120, 172, 150,
140, 130, 130, 110, 150],
    "Cholesterol": [233, 250, 204, 236, 354, 192, 294, 263, 199,
168, 239, 275, 266, 211, 283],
    "MaxHR": [150, 187, 172, 178, 163, 148, 153, 173, 162, 174, 160,
139, 171, 144, 162],
    "ExerciseAngina": [0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0],
    "Oldpeak": [2.3, 3.5, 1.4, 0.8, 0.6, 0.4, 1.3, 0.0, 0.5, 1.6,
1.2, 0.2, 0.6, 1.8, 1.0],
    "HeartDisease": [1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1]
}

df = pd.DataFrame(data)

print("Heart Disease Dataset:")
print(df)

print("\nMissing Values:")
print(df.isnull().sum())

X = df.drop("HeartDisease", axis=1)
y = df["HeartDisease"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

model = RandomForestClassifier(
```

```

        n_estimators=100,
        random_state=42
    )

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, zero_division=0)
recall = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)
cm = confusion_matrix(y_test, y_pred)

print("\nModel Evaluation:")
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1-Score:", f1)

print("\nConfusion Matrix:")
print(cm)

```

Output

Heart Disease Dataset:

	Age	Sex	ChestPain	RestBP	Cholesterol	MaxHR	ExerciseAngina	Oldpeak	\
0	63	1	3	145	233	150	0	2.3	
1	37	1	2	130	250	187	0	3.5	
2	41	0	1	130	204	172	0	1.4	
3	56	1	1	120	236	178	0	0.8	
4	57	0	0	120	354	163	1	0.6	
5	57	1	0	140	192	148	0	0.4	
6	56	1	1	140	294	153	0	1.3	
7	44	1	2	120	263	173	0	0.0	
8	52	1	2	172	199	162	0	0.5	
9	57	1	0	150	168	174	0	1.6	
10	54	1	2	140	239	160	0	1.2	
11	48	0	1	130	275	139	0	0.2	
12	49	1	1	130	266	171	0	0.6	
13	64	1	3	110	211	144	1	1.8	
14	58	0	0	150	283	162	0	1.0	

HeartDisease

0	1
1	0
2	0
3	1
4	1
5	0
6	1
7	0
8	0
9	1
10	1
11	0
12	0
13	1

14 1

Missing Values:

Age	0
Sex	0
ChestPain	0
RestBP	0
Cholesterol	0
MaxHR	0
ExerciseAngina	0
Oldpeak	0
HeartDisease	0

dtype: int64

Model Evaluation:

Accuracy: 0.8

Precision: 1.0

Recall: 0.6666666666666666

F1-Score: 0.8

Confusion Matrix:

```
[[2 0]
 [1 2]]
```

Slip 20

Q1. Write a Python program to create a DataFrame containing the attributes Age, Salary, Experience, Working_Hours, and Performance_Score. Apply the Standardization data transformation technique and display the transformed data.

```
import pandas as pd

from sklearn.preprocessing import StandardScaler

data = {
    "Age": [22, 25, 28, 30, 35],
    "Salary": [25000, 30000, 35000, 40000, 50000],
    "Experience": [1, 2, 4, 6, 10],
    "Working_Hours": [8, 8, 9, 9, 10],
    "Performance_Score": [60, 65, 70, 80, 90]
}

df = pd.DataFrame(data)

print("Original Data:")
print(df)

scaler = StandardScaler()

standardized_data = pd.DataFrame(
    scaler.fit_transform(df),
    columns=df.columns
)

print("\nStandardized Data:")
print(standardized_data)
```

Output

```
Original Data:
   Temperature  Humidity  Wind Speed  Pressure  Rainfall
```

0	20	40	5	1000	10
1	25	50	10	1005	20
2	30	60	15	1010	30
3	35	70	20	1015	40
4	40	80	25	1020	50

A) Min-Max Scaling:

	Temperature	Humidity	Wind Speed	Pressure	Rainfall
0	0.00	0.00	0.00	0.00	0.00
1	0.25	0.25	0.25	0.25	0.25
2	0.50	0.50	0.50	0.50	0.50
3	0.75	0.75	0.75	0.75	0.75
4	1.00	1.00	1.00	1.00	1.00

B) Standardization:

	Temperature	Humidity	Wind Speed	Pressure	Rainfall
0	-1.414214	-1.414214	-1.414214	-1.414214	-1.414214
1	-0.707107	-0.707107	-0.707107	-0.707107	-0.707107
2	0.000000	0.000000	0.000000	0.000000	0.000000
3	0.707107	0.707107	0.707107	0.707107	0.707107
4	1.414214	1.414214	1.414214	1.414214	1.414214

C) Normalization:

	Temperature	Humidity	Wind Speed	Pressure	Rainfall
0	0.019979	0.039958	0.004995	0.998939	0.009989
1	0.024831	0.049662	0.009932	0.998210	0.019865
2	0.029621	0.059243	0.014811	0.997254	0.029621
3	0.034348	0.068695	0.019627	0.996080	0.039254
4	0.039008	0.078015	0.024380	0.994696	0.048760

Q2. Create a dataset containing information about different books with the attributes Book Title, Author, Publication Year, Average Rating, and Number of Ratings.

Perform the following tasks on created dataset:

a. Create a Venn Diagram comparing the authors of books published in two different time periods.

b. Generate a Word Cloud using the book titles.

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from matplotlib_venn import venn2
```

```
from wordcloud import WordCloud
```

```
data = {
```

```
    "Book Title": [
```

```
        "Python Programming",
```

```
        "Data Science Basics",
```

```
        "Machine Learning Guide",
```

```
        "Python for Beginners",
        "Artificial Intelligence",
        "Web Development",
        "Data Analytics",
        "Machine Learning with Python",
        "Deep Learning",
        "Programming Fundamentals"
    ],
    "Author": [
        "James",
        "Robert",
        "James",
        "David",
        "Robert",
        "Michael",
        "Sarah",
        "James",
        "Sarah",
        "David"
    ],
    "Publication Year": [
        2015, 2016, 2018, 2019, 2020,
        2021, 2022, 2023, 2024, 2025
    ],
    "Average Rating": [
        4.2, 4.0, 4.5, 4.1, 4.3,
        3.9, 4.4, 4.6, 4.7, 4.0
    ],
    "Number of Ratings": [
        1200, 950, 1500, 800, 1300,
        700, 1100, 1600, 1400, 600
    ]
}
```



```

}

df = pd.DataFrame(data)

print("Book Dataset:")
print(df)

# a) Venn Diagram
period1 = set(df[df["Publication Year"] <= 2020]["Author"])
period2 = set(df[df["Publication Year"] > 2020]["Author"])

print("\nAuthors of Books Published up to 2020:")
print(period1)

print("\nAuthors of Books Published after 2020:")
print(period2)

plt.figure(figsize=(8, 6))

venn2(
    [period1, period2],
    set_labels=("Up to 2020", "After 2020")
)

plt.title("Comparison of Authors in Two Time Periods")
plt.show()

# b) Word Cloud
titles = " ".join(df["Book Title"])

wordcloud = WordCloud(
    width=1000,

```

```

        height=500,

        background_color="white"
    ).generate(titles)

plt.figure(figsize=(12, 6))

plt.imshow(wordcloud, interpolation="bilinear")

plt.axis("off")

plt.title("Word Cloud of Book Titles")

plt.show()

```

Output

Book Dataset:

	Book Title	Author	Publication Year	Average Rating	\
0	Python Programming	James	2015	4.2	
1	Data Science Basics	Robert	2016	4.0	
2	Machine Learning Guide	James	2018	4.5	
3	Python for Beginners	David	2019	4.1	
4	Artificial Intelligence	Robert	2020	4.3	
5	Web Development	Michael	2021	3.9	
6	Data Analytics	Sarah	2022	4.4	
7	Machine Learning with Python	James	2023	4.6	
8	Deep Learning	Sarah	2024	4.7	
9	Programming Fundamentals	David	2025	4.0	

Number of Ratings

0	1200
1	950
2	1500
3	800
4	1300
5	700
6	1100
7	1600
8	1400
9	600

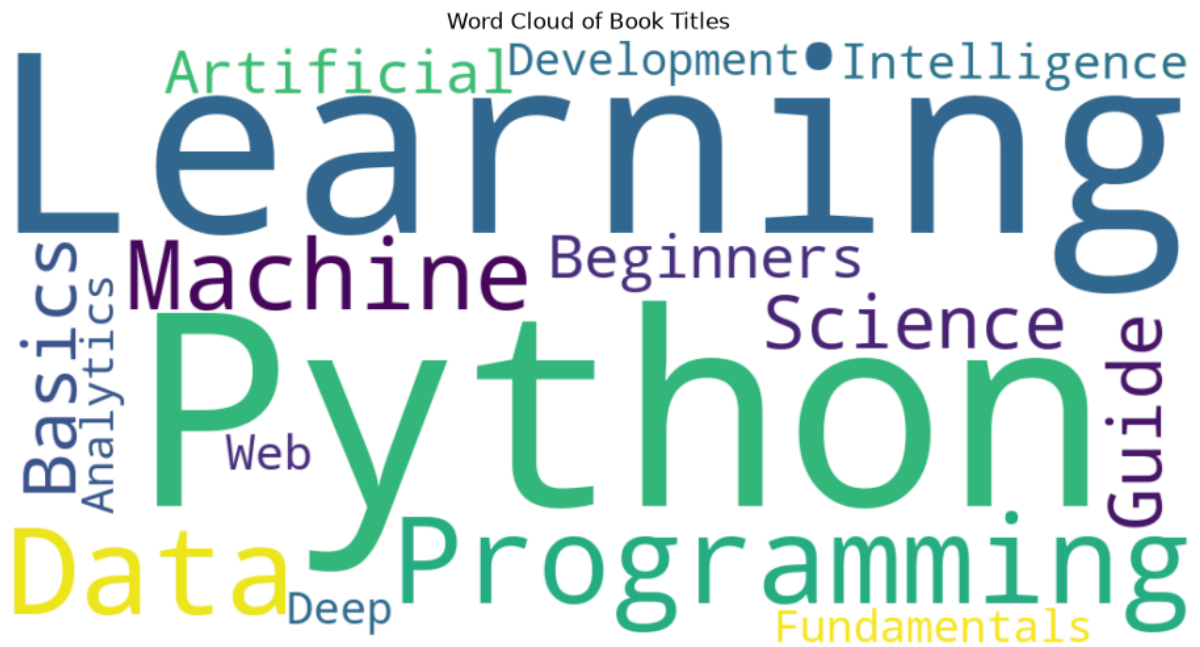
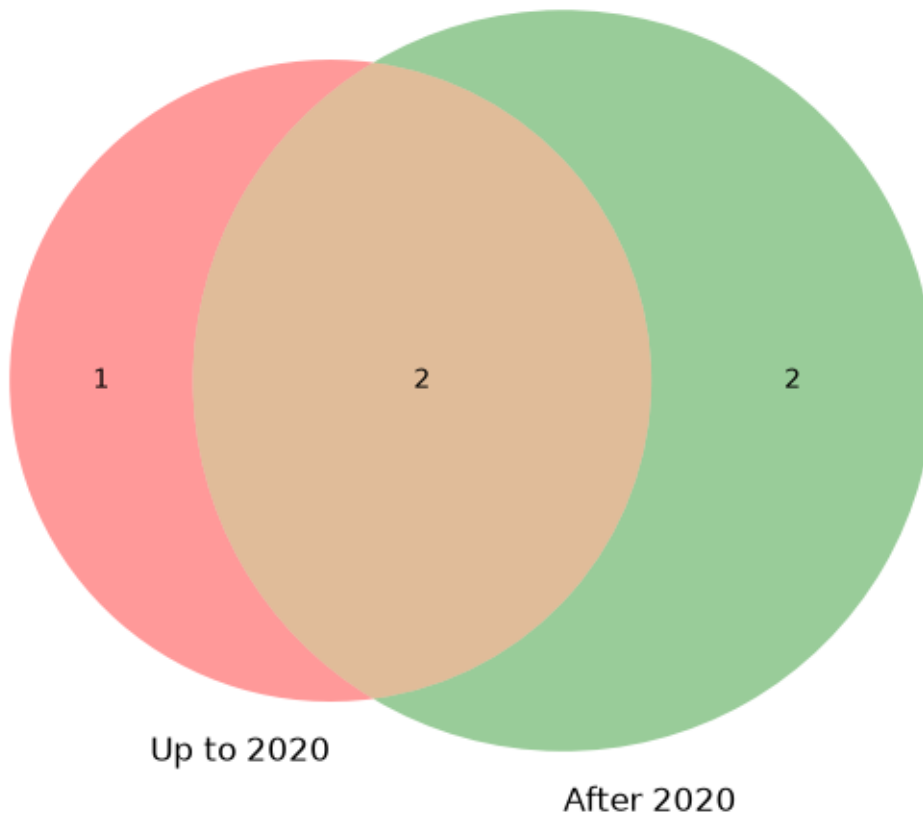
Authors of Books Published up to 2020:

```
{'James', 'David', 'Robert'}
```

Authors of Books Published after 2020:

```
{'Michael', 'David', 'Sarah', 'James'}
```

Comparison of Authors in Two Time Periods



OR

Q2. Apply the Apriori algorithm to the following student activity dataset to identify frequent itemsets and generate association rules among the activities. Repeat the analysis using minimum support values 0.2 and 0.3.

TID	Items
1.	Library , Seminar
2.	Workshop, Project
3.	Library, Assignment
4.	Seminar, Workshop
5.	Assignment, Seminar
6.	Workshop, Assignment
7.	Project, Library
8.	Library, Seminar, Project
9.	Library,Seminar,Workshop

```
import pandas as pd

from mlxtend.preprocessing import TransactionEncoder

from mlxtend.frequent_patterns import apriori, association_rules


data = {
    "TID": [1, 2, 3, 4, 5, 6, 7, 8, 9],
    "Items": [
        "Library, Seminar",
        "Workshop, Project",
        "Library, Assignment",
        "Seminar, Workshop",
        "Assignment, Seminar",
        "Workshop, Assignment",
        "Project, Library",
        "Library, Seminar, Project",
        "Library, Seminar, Workshop"
    ]
}
```

```

df = pd.DataFrame(data)

print("Original Dataset:")
print(df)

transactions = df["Items"].apply(
    lambda x: [item.strip() for item in x.split(",")]
)

te = TransactionEncoder()
encoded = te.fit(transactions).transform(transactions)

basket = pd.DataFrame(
    encoded,
    columns=te.columns_
).astype(int)

print("\nPreprocessed Transaction Data:")
print(basket)

for support in [0.2, 0.3]:

    print("\n" + "=" * 60)
    print("Minimum Support:", support)

    frequent_itemsets = apriori(
        basket,
        min_support=support,
        use_colnames=True
    )

```

```

print("\nFrequent Itemsets:")

if frequent_itemsets.empty:
    print("No frequent itemsets found.")
    continue

display_itemsets = frequent_itemsets.copy()
display_itemsets["itemsets"] =
display_itemsets["itemsets"].apply(
    lambda x: ", ".join(x)
)

print(display_itemsets)

rules = association_rules(
    frequent_itemsets,
    metric="confidence",
    min_threshold=0.5
)

print("\nAssociation Rules:")

if rules.empty:
    print("No association rules found.")
else:
    rules["antecedents"] = rules["antecedents"].apply(
        lambda x: ", ".join(x)
    )

    rules["consequents"] = rules["consequents"].apply(
        lambda x: ", ".join(x)
    )

```

```

print(
    rules[
        ["antecedents", "consequents",
        "support", "confidence", "lift"]
    ]
)

```

Output

Original Dataset:

	TID	Items
0	1	Library, Seminar
1	2	Workshop, Project
2	3	Library, Assignment
3	4	Seminar, Workshop
4	5	Assignment, Seminar
5	6	Workshop, Assignment
6	7	Project, Library
7	8	Library, Seminar, Project
8	9	Library, Seminar, Workshop

Preprocessed Transaction Data:

	Assignment	Library	Project	Seminar	Workshop
0	0	1	0	1	0
1	0	0	1	0	1
2	1	1	0	0	0
3	0	0	0	1	1
4	1	0	0	1	0
5	1	0	0	0	1
6	0	1	1	0	0
7	0	1	1	1	0
8	0	1	0	1	1

=====
Minimum Support: 0.2

Frequent Itemsets:

	support	itemsets
0	0.333333	Assignment
1	0.555556	Library
2	0.333333	Project
3	0.555556	Seminar
4	0.444444	Workshop
5	0.222222	Library, Project
6	0.333333	Library, Seminar
7	0.222222	Workshop, Seminar

Association Rules:

	antecedents	consequents	support	confidence	lift
0	Project	Library	0.222222	0.666667	1.20
1	Library	Seminar	0.333333	0.600000	1.08
2	Seminar	Library	0.333333	0.600000	1.08
3	Workshop	Seminar	0.222222	0.500000	0.90

=====
Minimum Support: 0.3

Frequent Itemsets:

	support	itemsets
0	0.333333	Assignment
1	0.555556	Library
2	0.333333	Project
3	0.555556	Seminar
4	0.444444	Workshop
5	0.333333	Library, Seminar

Association Rules:

	antecedents	consequents	support	confidence	lift
0	Library	Seminar	0.333333	0.6	1.08
1	Seminar	Library	0.333333	0.6	1.08